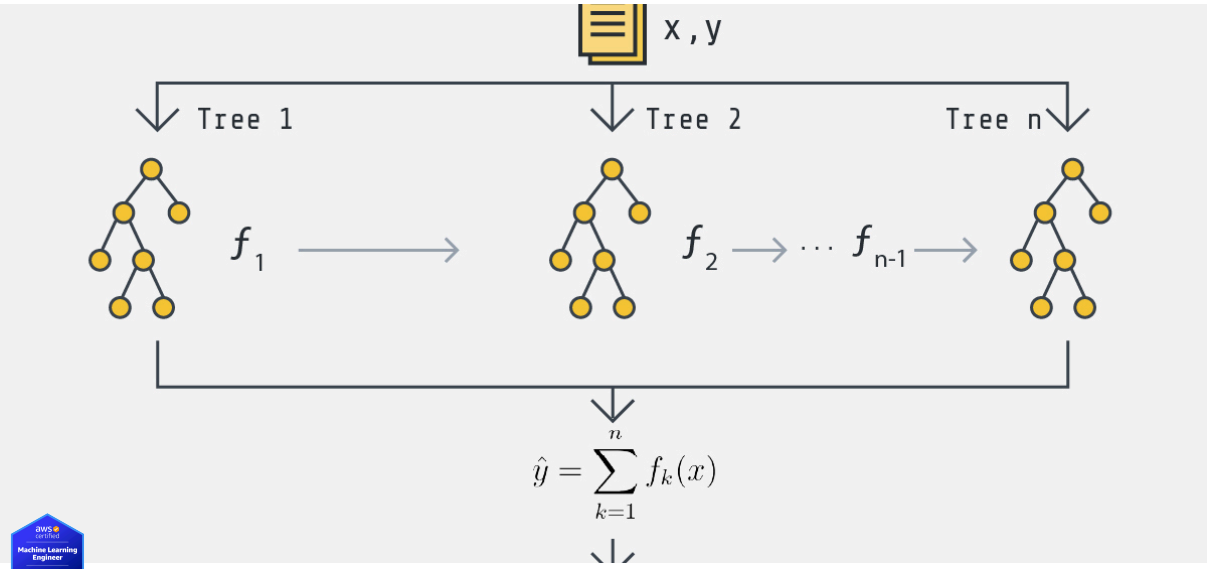


AWS MLE

Section I

- Data
- Amazon S3
- Amazon EBS, EFS & FSx
- Data Streams
- Data Transformation, Integrity & Feature Engineering

Section II

- AWS Managed AI Services
- SageMaker Built-In Algorithms
- Model Training, Tuning & Evaluation
- Generative AI on AWS

Section III

- MLOPs with AWS
- Security, Identity & Compliance
- Management & Governance
- ML Best Practices
- Good To Know

Section I

Data

▼ **Data Types.**

Structured - organized in defined manner or schema, typically found in relational databases (CSV, database tables). It is easily queryable, organized in rows and columns, and has consisent structure.

Unstructured - data that doesnt have a predefined structure or schema (text files, image, videos, audio). It is not easily queryable without preprocessing and may come in various formats.

Semi-structured - data that is not as organized as structured data but has some level of structure in the form of tags or hierarchies (XML, JSON, log files). It's more flexible than structured data but not as chaotic as unstructured.

▼ Properties of Data.

Volume - refers to the amount of size of data that organizations are dealing with at any given time.

It may range from gigabytes to petabytes or even more. There are challenges in storing, processing and analyzing high volumes of data.

Velocity - refers to the speed at which new data is generated, collected and processed.

High velocity required real time or near-real-time processing capabilities. Rapid ingestion and processing can be critical for certain apps.

Variety - refers to the different types, structures and sources of data. Data can come from multiple sources and in various formats.

▼ Data Warehouses, Lakes & Lakehouses.

Data Warehouse - centralized repository optimized for analysis where data from different sources is stored in a structured format.

- Designed for complex queries and analysis.
- Typically uses star or snowflake schema.
- Optimized for read-heavy operations.

Usage

- When have structured data sources and require fast and complex queries.
- Data integration from different sources is essential.
- BI and analytics are the primary use case.

Data Lake - storage repository that holds vast amounts of raw data in its native format.

- Can store large volumes of raw data without schema.
- Supports batch, real-time and stream processing.

Usage

- We have mix of data formats.
- Need scalable and cost-effective solution to store massive amounts of data.
- Advanced analytics, ML or data discovery.

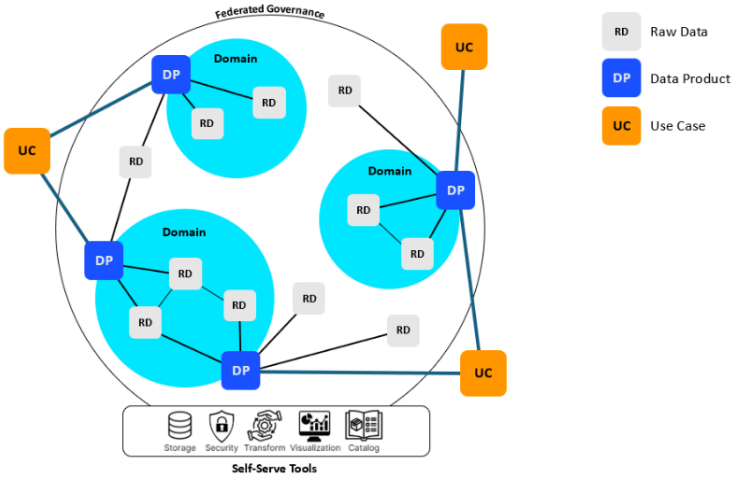
Feature	Data Warehouse	Data Lake
Schema	Schema-on-write (defined before storing data)	Schema-on-read (defined when reading data)
Process Model	ETL (Extract → Transform → Load)	ELT (Extract → Load → Transform)
Data Types	Mostly structured data	Structured + unstructured data
Agility	Less flexible due to predefined schema	More flexible, accepts raw data
Processing	Data transformed before storage	Data transformed after loading
Cost	Usually more expensive (optimized queries)	Cheaper storage but processing can increase cost

Data Lakehouse - hybrid data architecture that combines the best features of data lakes and data warehouses.

▼ Data Mesh.

Data Mesh is a data architecture approach where data ownership and management are decentralized and handled by domain teams, instead of being controlled by a single central data team.

Instead of one centralized data platform, each business domain manages its own data as a product.



▼ Comon Data Formats.

- **CSV** - text-based format that represents data in a tabular form where each line corresponds to a row and values within a row are separated by commas.

  Usage: small/medium datasets, data interchange between systems, importing/exporting data from databases or spreadsheets.

- **JSON** - lightweight, text-based, and human-readable data interchange format that represents structured or semi-structured data based on key-value pairs.

  Usage: data interchange between a web server and a web client, configurations and settings for apps, cases where flexible schema is needed.

- **Avro** - binary format that stores both the data and its schema, allowing it to be processed later with different systems.

  Usage: big data and real-time processing systems, when schema evolution is needed, fast serialization.

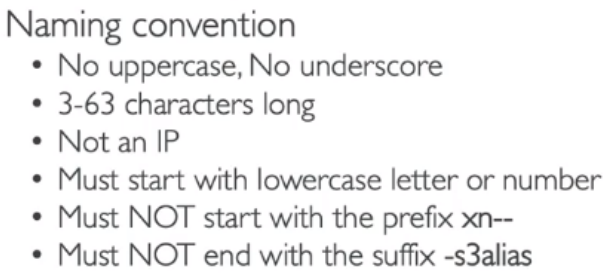
- **Parquet** - columnar storage format optimized for analytics. Allows for efficient compression and encoding schemes.

  Usage: analyzing large datasets with analytics engines, storing data on distributed systems.

Amazon S3

S3 Basics.

Amazon S3 allows people to store objects (files) in **buckets** (directories). Buckets must have a globally unique name (across all regions all accounts). Buckets are defined at the region level (S3 looks like a global service but buckets are created in a region).



Objects have a key. The key is FULL path (for example s3://my-bucket/folder1/folder2/file.txt). The key is composed of prefix+object name. There is no concept of "directories" within buckets, just keys with very long names that contain slashes.

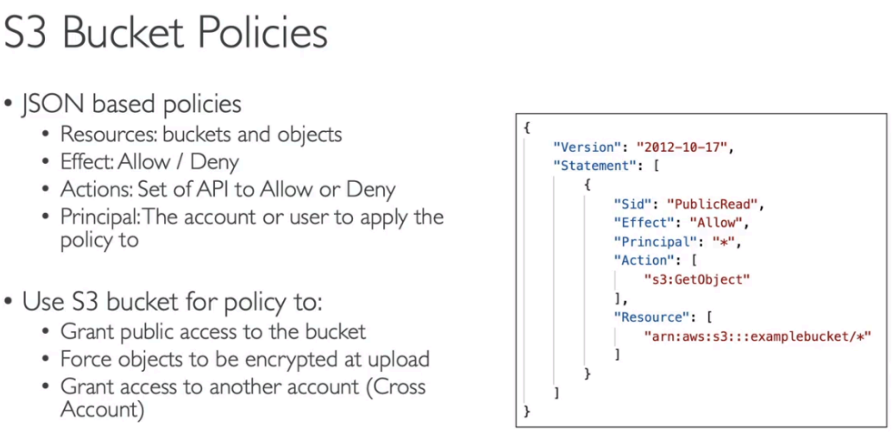
Object values are content of the body. Max object size is 5TB, if we upload more than 5GB, must use "multi-part upload". Object can have also **metadata** (list of text key/value pairs), **tags** (unicode key/value pair, useful for security/lifecycle), **version ID**.

S3 Security.

- **User-Based**

  IAM Policies - which API calls should be allowed for a specific user from IAM.

- **Resource-Based**
  - Bucket Policies - bucket wide rules from the S3 console (allows cross account).



  - Object Access Control List (ACL) - finer grain (can be disable).
  - Bucket Access Control List (ACL) - less common (can be disabled).

An IAM principal can access an S3 object if the user IAM permissions ALLOW it OR the resources policy ALLOWS it AND there is no explicit DENY.

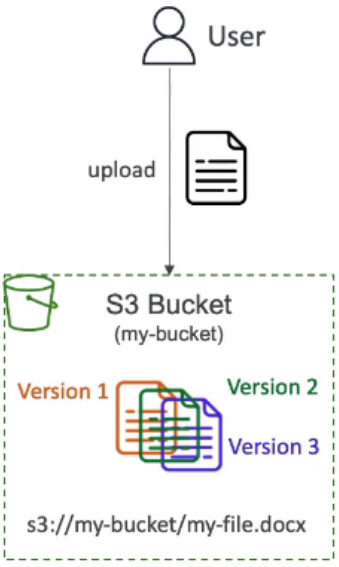
By default, an S3 object is owned by the AWS account that uploaded it even though the bucket is owned by another account. Object owner must explicitly grant the bucket owner access using a bucket policy to require external users to grant *bucket-owner-full-control* when uploading objects.

There are bucket settings for block public access. These settings were created to prevent company data leaks. If we know our bucket should never be public, we should leave these on (can be set at the account level).

> If we host static website and get 403 forbidden error, then we should check if bucket policy allows public reads.

S3 Versioning.

Version for files can be enabled in Amazon S3. It is enabled at the bucket level. Same key overwrite will change the versions.



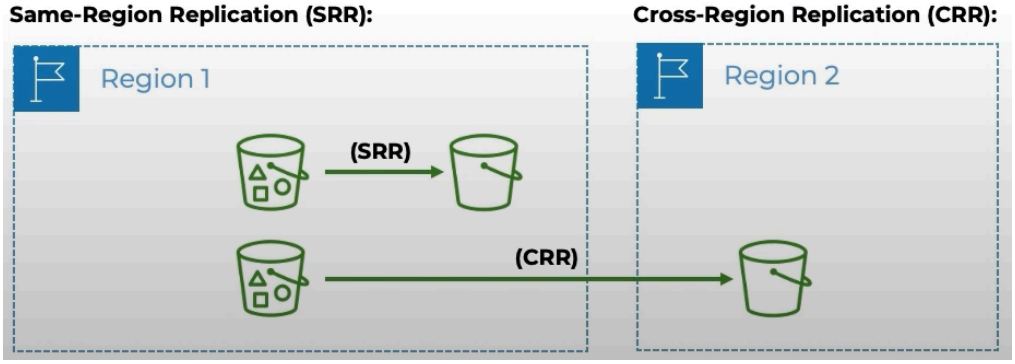
It's best practice to version our buckets. It protects us against unintended deletes (can restore a version) and we can roll back to previous version easily.

Any file that is not versioned prior to enabling versioning will have "null" version. Suspending versioning does not delete the previous versions.

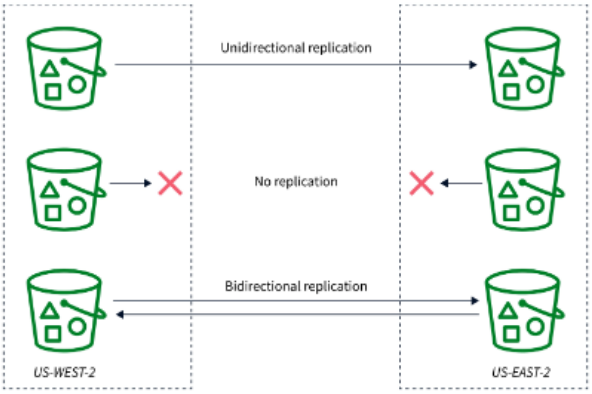
S3 Replication.

To perform replication we must enable versioning in source and destination buckets. Buckets can be in different AWS accounts. Copying is asynchronous and we must give proper IAM permissions to S3.

- **Cross-Region Replication (CRR)** - replicates objects between buckets in different AWS regions. It's useful for disaster recovery, latency reduction and compliance.
- **Same-Region Replication (SRR)** - replicates objects between buckets within the same AWS region. It's useful for log aggregation and live replication between production and test accounts.



- **One-way Replication** - replicate objects from a source bucket to a destination bucket, but not back to the source.
- **Two-way Replication** - allows for bi-directional **syncing** between two S3 buckets. Changes made in either bucket can be reflected in the other.



After we enable replication, only new objects are replicated. We can replicate existing objects using **S3 Batch Replication**.

When versioning is enabled, deleting an object creates a Delete Marker instead of removing the object. The object appears deleted, but older versions still exist. Can be replicated if **Delete Marker Replication** is enabled.

Version ID Deletes - deletes a specific version of an object permanently using its version ID. Not replicated to the destination bucket.

▼ **S3 Storage Classes.**

- **S3 Standard - General Purpose**

  Used for frequently accessed data. Has low latency and high throughput. Sustain 2 concurent facility failures. It's used for big data analytics, mobile and gaming applications, content distribution...

IA storage classes are used for data that is less frequently accessed, but requires rapid access when needed. They have lower cost than S3 Standard.

- **S3 Standard-IA**

  It's used for disaster recovery and backups. Has 99,9% availability.

- **S3 One Zone-IA**

It's used for storing secondary backup copies of on-premise data, or data we can recreate. Has high durability in a single AZ. If AZ is destroyed data is lost.

S3 Glacier storage classes are low-cost object storage meant for archiving and backup. Pricing consist of price for storage + object retrieval cost.

- **S3 Glacier Instant Retrieval**

  Has millisecond retrieval time, great for data accessed once a quarter. Minimum storage duration is 90 days.

- **S3 Glacier Flexible Retrieval**
  - Expedited - 1 to 5 minutes retrieval time.

    Provisioned capacity ensures that our retrieval capacity for expedited retrievals is available when we need it. Each unit of capacity provides that at least three expedited retrievals can be performed every five minutes and provides up to 150 MB/s of retrieval throughput.
  - Standard - 3 to 5 hours retrieval time.
  - Bulk - 5 to 12 hours retrieval time (free).

  Minimum storage duration is 90 days.

- **S3 Glacier Deep Archive**
  - Standard - 12 hours.
  - Bulk - 48 hours.

  Minimum storage duration is 180 days.

- **S3 Intelligent Tiering**

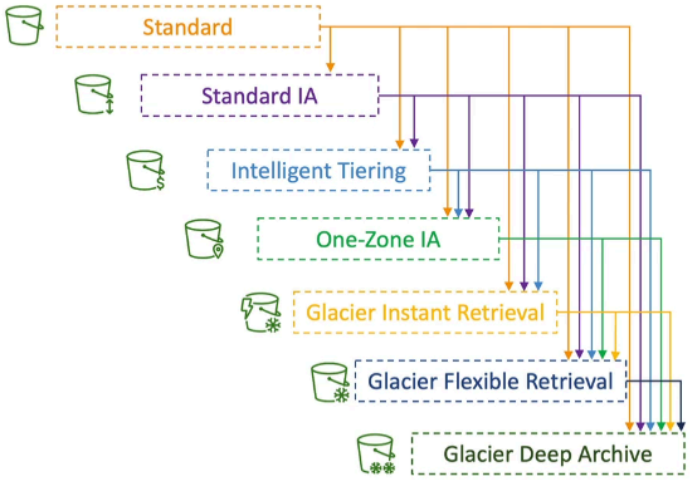
  Allows us to move objects between existing tiers based on usage pattern. We need to pay small montly monitoring and auto-tiering fee. There are no retrieval charges in S3 Intelligent-Tiering.
  - Frequent Access tier (automatic) - default tier.
  - IA tier (automatic) - objects not accessed for 30 days.
  - Archive Instant Access tier (automatic) - objects not accessed for 90 days.
  - Archive Access tier (optional) - configurable from 90 days to 700+ days.
  - Deep Archive Access tier (optional) - configurable from 180 days to 700+ days.

	Standard	Intelligent-Tiering	Standard-IA	One Zone-IA	Glacier Instant Retrieval	Glacier Flexible Retrieval	Glacier Deep Archive
Durability	99.999999999% == (11 9's)						
Availability	99.99%	99.9%	99.9%	99.5%	99.9%	99.99%	99.99%
Availability SLA	99.9%	99%	99%	99%	99%	99.9%	99.9%
Availability Zones	>= 3	>= 3	>= 3	1	>= 3	>= 3	>= 3
Min. Storage Duration Charge	None	None	30 Days	30 Days	90 Days	90 Days	180 Days
Min. Billable Object Size	None	None	128 KB	128 KB	128 KB	40 KB	40 KB
Retrieval Fee	None	None	Per GB retrieved	Per GB retrieved	Per GB retrieved	Per GB retrieved	Per GB retrieved

	Standard	Intelligent-Tiering	Standard-IA	One Zone-IA	Glacier Instant Retrieval	Glacier Flexible Retrieval	Glacier Deep Archive
Storage Cost (per GB per month)	$0.023	$0.0025 - $0.023	%0.0125	$0.01	$0.004	$0.0036	$0.00099
Retrieval Cost (per 1000 request)	**GET:** $0.0004 **POST:** $0.005	**GET:** $0.0004 **POST:** $0.005	**GET:** $0.001 **POST:** $0.01	**GET:** $0.001 **POST:** $0.01	**GET:** $0.01 **POST:** $0.02	**GET:** $0.0004 **POST:** $0.03 **Expedited:** $10 **Standard:** $0.05 **Bulk:** free	**GET:** $0.0004 **POST:** $0.05 **Standard:** $0.10 **Bulk:** $0.025
Retrieval Time	Instantaneous					**Expedited** (1 – 5 mins) **Standard** (3 – 5 hours) Bulk (5 – 12 hours)	**Standard** (12 hours) **Bulk** (48 hours)
Monitoring Cost (pet 1000 objects)		$0.0025					

▼ **S3 Lifecycle Rules.**

Lifecycle Rules in S3 are a set of actions that we can configure to manage the lifecycle of objects in our bucket. These rules enable us to automatically transition objects to different storage classes or delete them after a specified period. Lifecycle rules help us reduce costs and simplify storage management.



Transition Actions - used to configure objects to transition to another storage class.

Expiration Actions - used to configure objects to expire (delete) after some time.

- Can be used to delete old versions of files (if versioning is enabled).
- Can be used to delete incomplete Multi-Part uploads.

Rules can be created for a certain prefix or for certain objects tags.

▼ Lifecycle Rules Question - Scenario I

Our application on EC2 creates images thumbnails after profile photos are uploaded to S3. These thumbnails can be easily recreated and only need to be kept for 60 days. The source images should be able to be immediately retrieved for these 60 days, and afterwards, the user can wait up to 6 hours. How to design this?

- S3 source images can be on **Standard** with a lifecycle configuration to transition them to Glacier after 60 days.
- S3 thumbnails can be on **One-Zone IA** with a lifecycle configuration to expire them after 60 days.

▼ Lifecycle Rules Question - Scenario II

A rule in our company states that we should be able to recover our deleted S3 objects immediately for 30 days, although this may happen rarely. After this time, and for up to 365 days, deleted objects

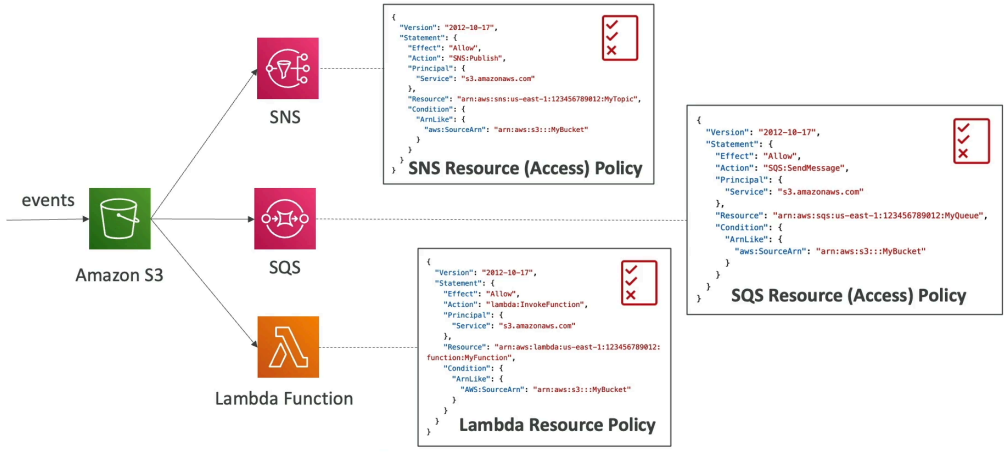
should be recoverable within 48 hours.

- Enable S3 Versioning in order to have objects version.
- Transition the "noncurrent versions" of the object to Standard IA.
- Transition afterwards the "noncurrent versions" to Glacier Deep Archive.

S3 Analytics - helps us decide when to transition objects to the right storage class. Recommendations are only for Standard and Standard IA. Report is updated daily and we need to wait 24 to 48 hours to start seeing data analysis.

S3 Event Notifications.

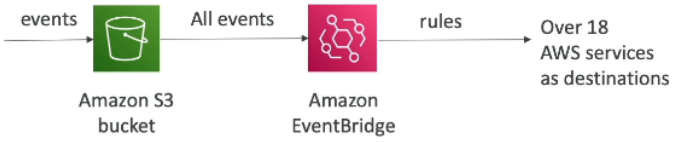
S3 Event Notifications allow us to receive notifications when certain events happen to objects within our S3 buckets. We can create as many S3 events as desired.



S3 event notifications typically deliver events in seconds but can sometimes take a minute or longer.

We can directly send S3 events to EventBridge, expanding the range of destinations and enabling more complex event handling and routing scenarios. SQS FIFO queues aren't supported as an S3 event notification destination.

- Advanced filtering options with JSON rules.
- Multiple destinations (Step Functions, Kinesis Streams)
- Archive and replay events.

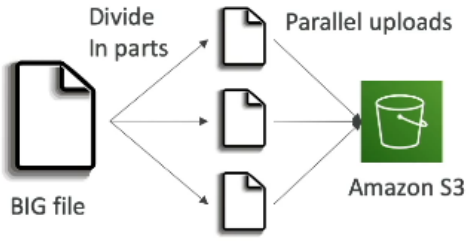


S3 Performance.

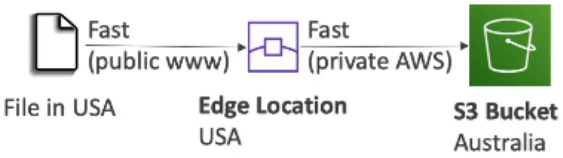
S3 automatically scales to high request rates, latency 100-200ms. Our application can achieve at least 3500 PUT/COPY/POST/DELETE or 5500 GET/HEAD requests per second per prefix in a bucket. If we spread reads across all four prefixes evenly, we can achieve 22000 requests per second for GET and HEAD.

Performance Boosts

- **Multi-Part Upload** - recommended for files < 100MB, must use for files >5GB. It can help parallelize uploads.



- **S3 Transfer Acceleration** - increase transfer speed by transferring file to an AWS Edge Location which will forward the data to the S3 bucket in the target region. It is compatible with multi-part upload.



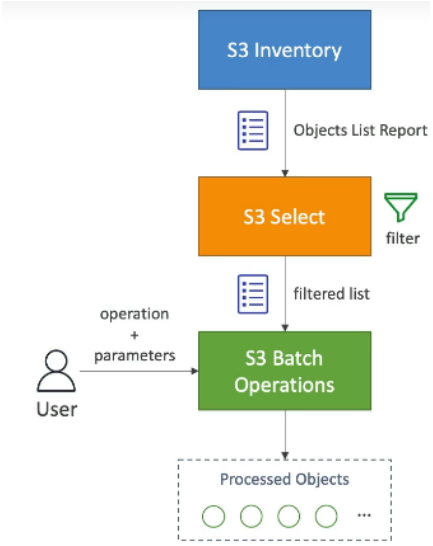
- **S3 Byte-Range Fetches** - parallelizes GETs by requesting specific byte ranges. It has better resilience in case of failures. Can be used to speed up downloads and to retrieve only partial data.

▼ S3 Batch Operations.

With **S3 Batch Operations** we can perform bulk operations on existing S3 objects with a single request. A job consists of a list of objects, the action to perform and optional parameters. S3 Batch Operations manages retries, tracks progress, sends completion notifications and generate reports.

Use cases:

- Modify object metadata & properties.
- Copy objects between S3 buckets.
- **Encrypt un-encrypted objects.**
- Modify ACLs
- Restore objects from S3 Glacier
- Invoke Lambda function to perform custom action on each object.

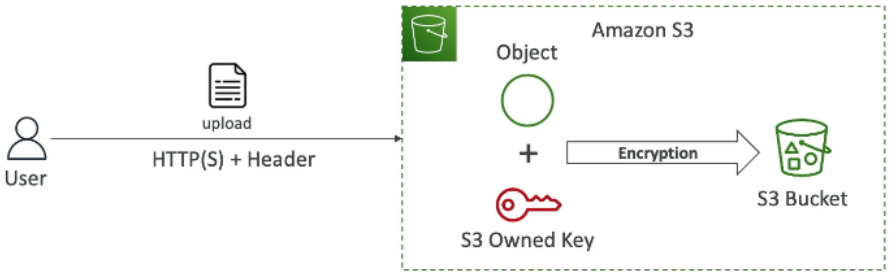


We can use S3 Inventory to get object list and use S3 Select to filter our objects.

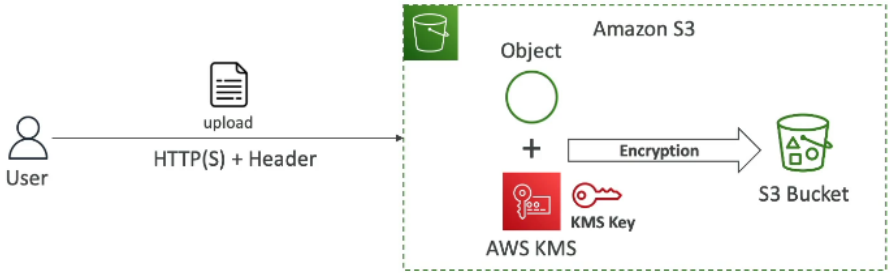
▼ **S3 Encryption.**

- **Server-Side Encryption (SSE)**
  - **SSE with S3-Managed Keys (SSE-S3)** - it is enabled by default and encrypts S3 objects using keys handled, managed and owned by AWS.

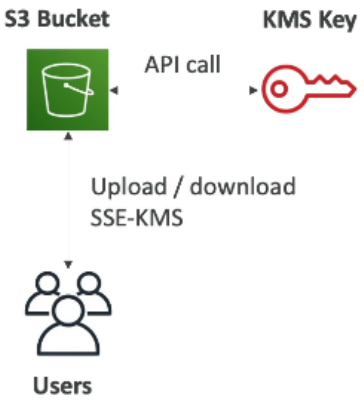
    We must set header *"x-amz-server-side-encryption":"AES256"*.



  - **SSE with KMS Keys stored in AWS KMS (SSE-KMS)** - leverages KMS to manage encryption keys.
  - We must set header *"x-amz-server-side-encryption":"aws:kms"*.

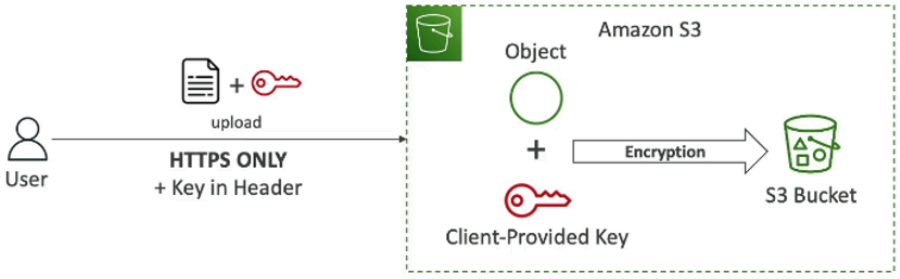


If we use SSE-KMS we may be impacted by the KMS limits. When we upload, it calls the GenerateDataKey KMS API and when we download, it calls the Decrypt KMS API.



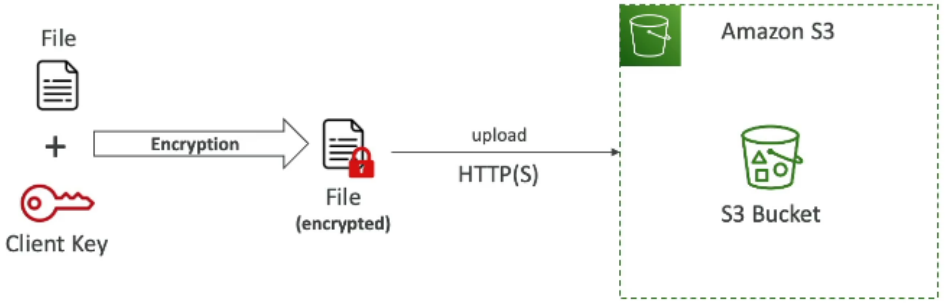
- **SSE with Customer-Provided Keys (SSE-C)** - when we want to manage our own encryption keys.

  We must use HTTPS for this type of encryption. Encryption key must be provided in HTTP headers, for every HTTP request made.



- **Client-Side Encryption**

  This type of encryption leverages client libraries such as S3 Client-Side Encryption Library. Clients must encrypt (decrypt) data themselves before sending (retrieving) to (from) S3. Customer fully manages the keys and encryption cycle.



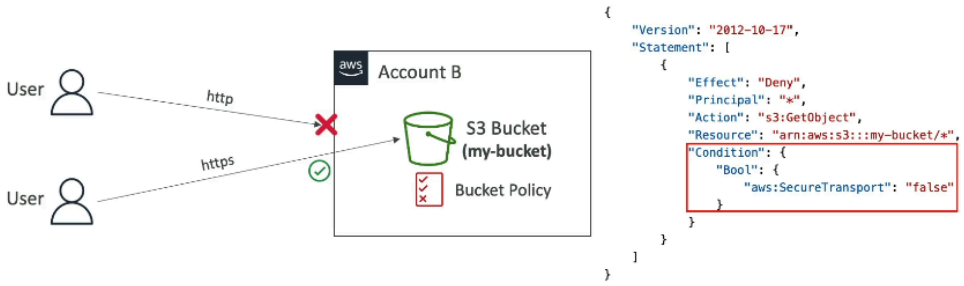
- **Encryption in Transit**

  S3 exposes two endpoints:

  - HTTP Endpoint (not encrypted)
  - HTTPS Endpoint (encryption in flight)

  HTTPS is mandatory for SSE-C.

  Force Encryption in Transit



SSE-S3 encryption is automatically applied to new objects stored in S3 buckets. Optionally, we can force encryption using a bucket policy and refuse any API call to PUT an S3 object without encryption headers (SSE-KMS or SSE-C).

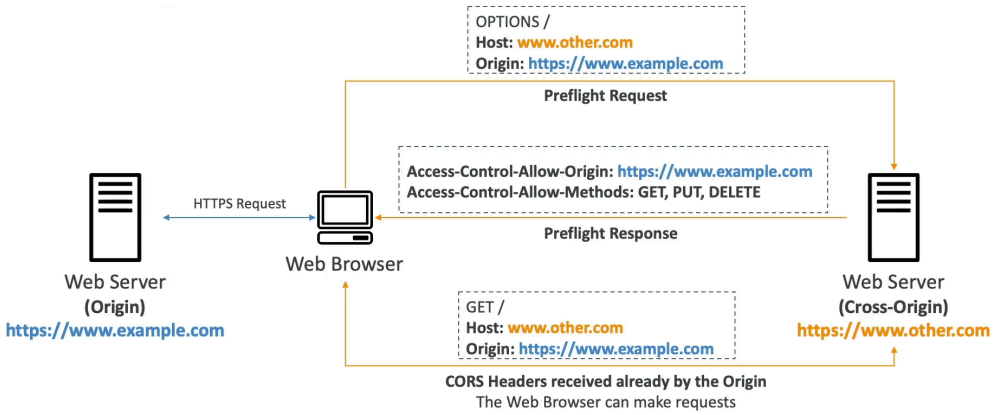
```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Deny",
            "Action": "s3:PutObject",
            "Principal": "*",
            "Resource": "arn:aws:s3:::my-bucket/*",
            "Condition": {
                "StringNotEquals": {
                    "s3:x-amz-server-side-encryption": "aws:kms"
                }
            }
        }
    ]
}
```

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Deny",
            "Action": "s3:PutObject",
            "Principal": "*",
            "Resource": "arn:aws:s3:::my-bucket/*",
            "Condition": {
                "Null": {
                    "s3:x-amz-server-side-encryption-customer-algorithm": "true"
                }
            }
        }
    ]
}
```

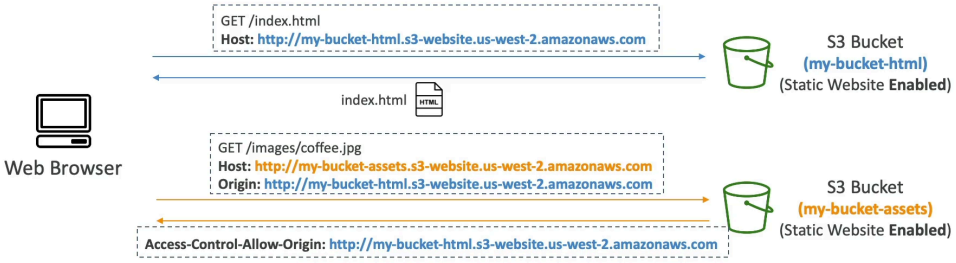
Bucket Policies are evaluated before “Default Encryption”.

S3 CORS.

CORS - Web Browser based mechanism to allow requests to other origins while visiting the main origin. The request wont be fulfilled unless the other origin allows for the requests, using CORS Headers (Access-Control-Allow-Origin).



If a client makes a cross-origin request on our S3 bucket, we need to enable the correct CORS headers. We can allow for a specific origin or * for all origins.



S3 MFA Delete.

MFA Delete is a feature designed to add an extra layer of security to protect our S3 bucket's objects from accidental or malicious deletion. It requires users to provide two or more forms of verification before they can perform deletion.

To use MFA Delete, Versioning must be enabled on the bucket and only the bucket owner (root account) can enable/disable MFA Delete.

S3 Access Logs.

S3 Access Logs are used for audit purpose, we may want to log all access to S3 buckets. Any request made to S3, from any account, authorized or denied will be logged into another S3 bucket. Target logging bucket must be in the same AWS region.

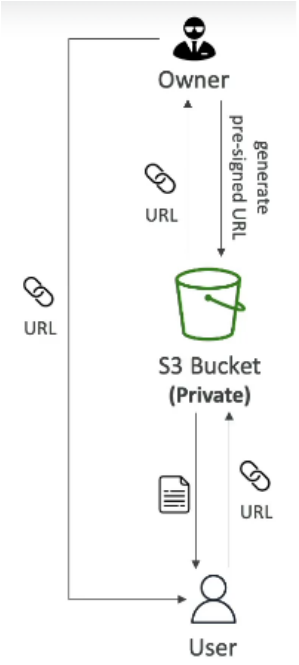
> We should not set our logging bucket to be the monitored bucket. It will create a logging loop and our bucket will grow exponentially.

S3 Pre-Signed URLs.

We can generate pre-signed URLs using the S3 Console, AWS CLI or SKD. Users given a pre-signed URL inherit the permissions of the user that generated the URL for GET and PUT methods.

Use cases:

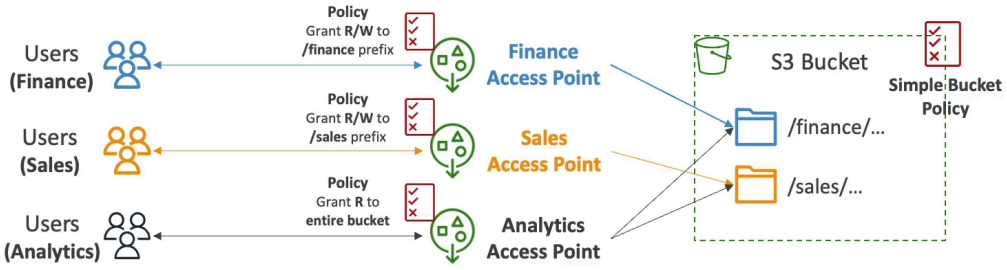
- Allow only logged-in users to download a premium video from our S3 bucket.
- Allow an ever-changing list of users to download files by generating URLs dynamically.
- Allow temporarily a user to upload a file to a precise location in our S3 bucket.



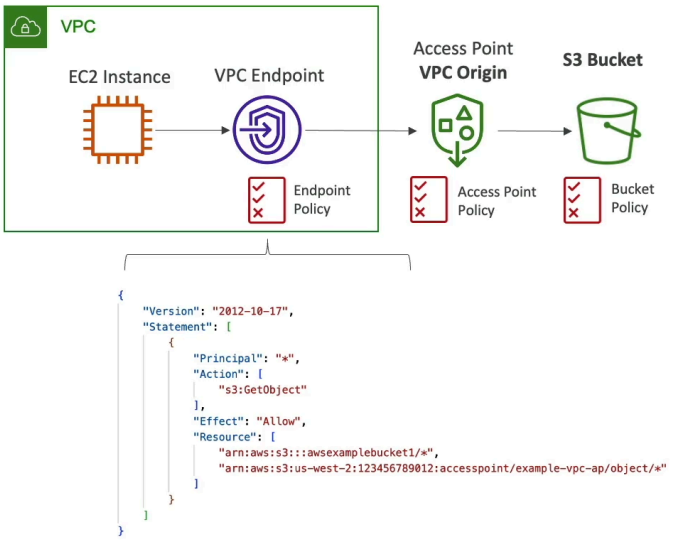
Pre-signed URLs **allow users to interact directly with the cloud storage service rather than routing through our application server**. This can reduce latency.

▼ **S3 Access Points.**

Access Points simplify security management for S3 Buckets. Each Access Point has its own DNS name (Internet Origin or VPC Origin) and access point policy (similar to bucket policy).



We **can define the access point to be accessible only from within the VPC**. To do that we must create a VPC Endpoint to access the Access Point and the VPC Endpoint must allow access to the target bucket and Access Point.

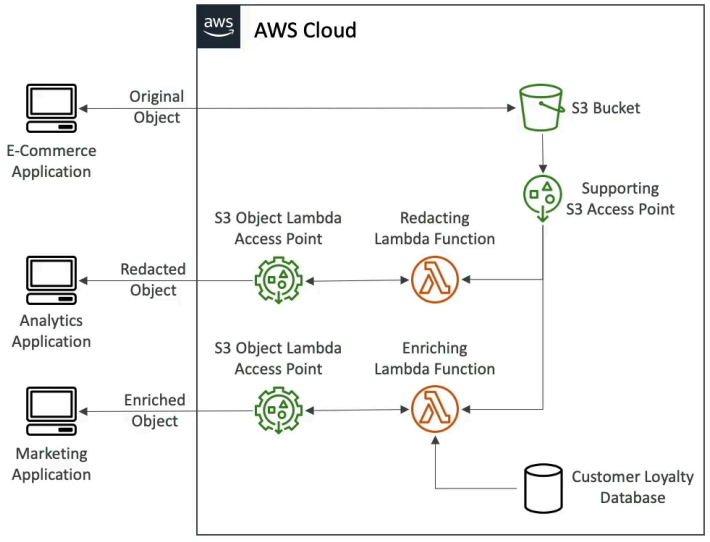


S3 Multi-Region Access Points - provide a global endpoint that applications can use to fulfill requests from S3 buckets located in multiple AWS Regions. We can use Multi-Region Access Points to build multi-Region applications with the same simple architecture used in a single Region, and then run those applications anywhere in the world.

They provide built-in network resilience with acceleration of internet-based requests to Amazon S3. Application requests made to a Multi-Region Access Point global endpoint use AWS Global Accelerator.

S3 Object Lambda.

We can use Lambda functions to change the object before it is retrieved by the caller application. Only one S3 bucket is needed, on top of which we create S3 Access Point and S3 Object Lambda Access Points.



Use cases:

- Redacting PII information from analytics or non-productiuon environments.
- Converting across data formats, such as converting XML to JSON.
- Resizing and watermarking images on the fly using caller-specific details, such as the user who requested the object.

Amazon EBS, EFS & FSx

EBS.

EBS Volume (think of them as a "network USB stick") is a network drive we can attach to our instances (cloud only) while they run. It allows our instances to persist data even after their termination. EBS volumes support live configuration changes while in production which means that we can modify the volume type, volume size, and IOPS capacity without service interruptions.

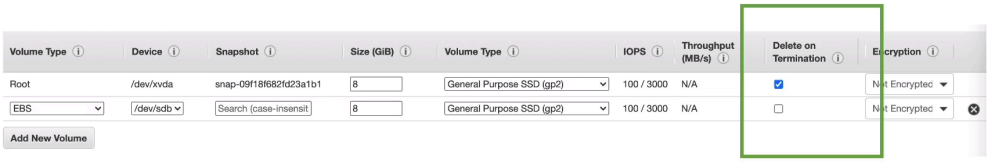
EBS can only be mounted to one instance at a time. They are bound to a specific AZ.

- It's a network drive (i.e. not a physical drive)
  - It uses the network to communicate the instance, which means there might be a bit of latency
  - It can be detached from an EC2 instance and attached to another one quickly
- It's locked to an Availability Zone (AZ)
  - An EBS Volume in us-east-1a cannot be attached to us-east-1b
  - To move a volume across, you first need to snapshot it
- Have a provisioned capacity (size in GBs, and IOPS)
  - You get billed for all the provisioned capacity
  - You can increase the capacity of the drive over time

Data Lifecycle Manager (DLM) - automate the creation, retention, and deletion of snapshots taken to back up our EBS volumes.

Delete on Termination

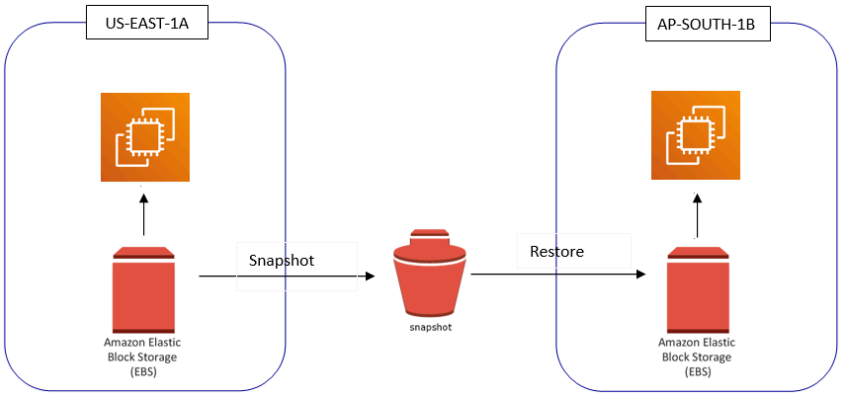
- Controls the EBS behaviour when an EC2 instance terminates.
  - By default, the root EBS volume is deleted (attribute enabled).
  - By default, any other attached EBS volume is not deleted (attribute disabled).
- This can be controlled by the AWS console / AWS CLI.
- Use case: preserve root volume when instance is terminated.



EBS Elastic Volumes is a feature that let us change the size, performance, or type of an EBS volume without stopping or detaching it from an instance.

EBS Snapshots.

It allows us to make a backup of EBS volume at a point in time. We can copy snapshots across AZ or Region. The EBS volume can be used while the snapshot is in progress.



EBS Snapshot Features:

- **EBS Snapshot Archive**

  We can move a snapshot to an "archive tier" that is 75% cheaper. It takes within 24 to 72 hours for restoring the archive.

- **Recycle Bin for EBS Snapshots**

  Setup rules to retain deleted snapshots so we can recover them after an accidental deletion.

EC2 Instance Store.

EBS volumes have "limited" performance. If we need high performance hardware disk, we should use **EC2 Instance Store**.

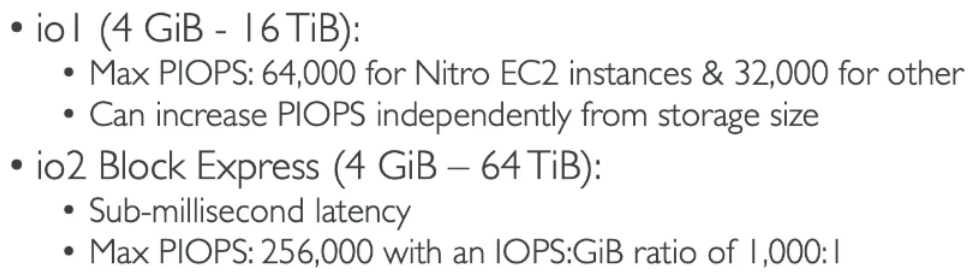
EC2 Instance Store has better I/O performance and they are good for buffer, **cache**, scratch data or temporary content.

EC2 Instance Store lose their storage if they are stopped (ephemeral). So we have risk of data loss if hardware fails. Backups and replication are our responsibility.

EBS Volume Types.

EBS Volumes are characterized in size, throughput and IOPS. Only gp2/gp3 and io1/io2 Block Express can be used as boot volumes.

- **gp2 / gp3 (SSD)** - general purpose SSD volume that balances price and performance for a wide variety of workloads. Small gp2 volume can burst IOPS to 3000. Gp3 is cheaper.
- **io1 / io2 Block Express (SSD)** - highest performance SSD volume for mission-critical low-latency or high-throughput workloads. Supports multi-attach. It's great for database workloads.



- **st1 (HDD)** - low cost HDD volume designed for frequently accessed, throughput-intensive workloads.
- **sc1 (HDD)** - lowest cost HDD volume designed for less frequently accessed workloads.

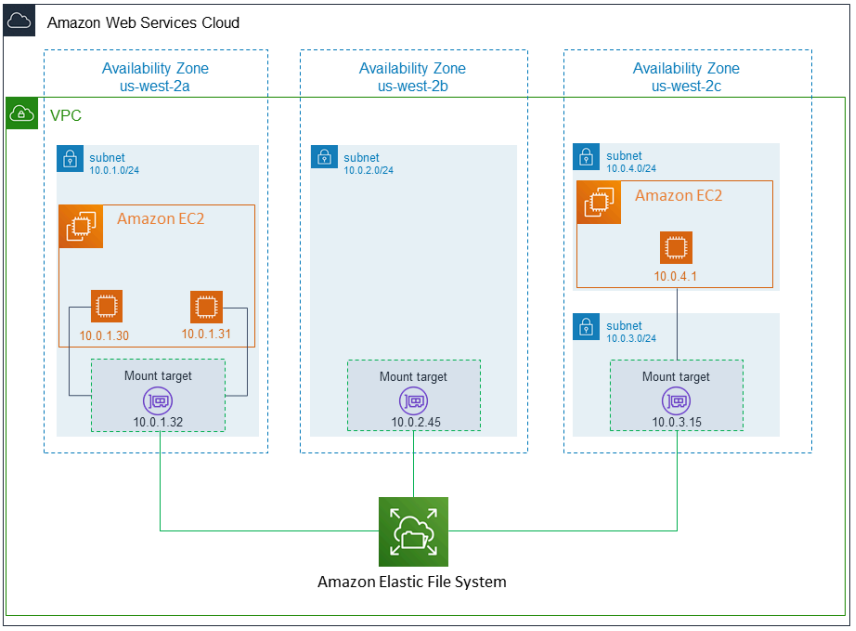
Volume Type	General Purpose SSD	Provisioned IOPS SSD	Throughput Optimized HDD	Cold HDD	EBS Magnetic HDD
Description	Balance price and performance for a wide variety of transactional workloads	Designed for mission critical applications which requires high performance	Low cost. Designed for frequently accessed throughput intensive workloads	The lowest cost. Designed for less frequently accessed workloads	Previous generation HDD
API Name	gp2, gp3	Io1, io2	St1	Sc1	Standard
Use Case	Most workloads	Large Database workloads	Big Data, Warehouses, Log processing	File Servers	Workloads with infrequently accessed data
Size	1GB - 16TB	4GB – 16TB	500GB - 16TB	500GB - 16TB	1GB - 1TB
Max IOPS	16,000	64,000	500	250	40-200
Can be a boot volume?	Yes	Yes	No	No	No
Bursting	gp2 = Yes gp3 = No	No	Yes	Yes	No

▼ **EFS.**

EFS is managed NFS (network file system) that can be mounted on hundreds of EC2. EFS works with **Linux** (uses POSIX file system) EC2 instances in multi-AZ. It uses NFSv4.1 protocol.

We can enable encryption at rest using KMS.

EFS is highly available, scalable, expensive, pay per use and has no capacity planning.



EFS can handle 1000s of concurrent NFS clients. Grows to petabyte-scale network file system automatically.

Performance Classes

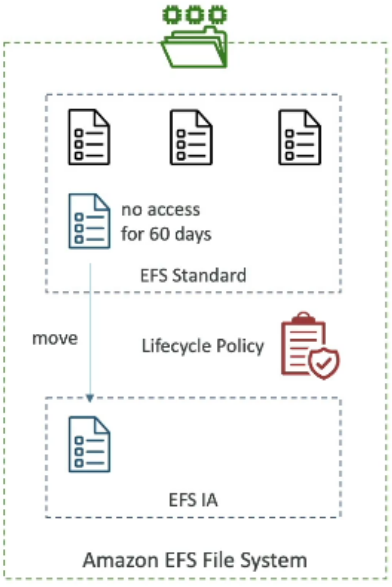
- **Performance Mode**
  - **General Purpose (default)** - best for latency-sensitive applications, such as web serving, content management systems, and development environments.
  - **Max I/O** - ideal for applications requiring high levels of aggregate throughput and parallelism, such as big data and media processing.
- **Throughput Mode**

- **Bursting Throughput** - throughput scales with the size of the file system. Suitable for most file systems, offering a balance between cost and performance.
- **Provisioned Throughput** - allows us to specify the desired throughput level, which can exceed what is available via the burst mode. Best for applications requiring a consistent and predictable level of throughput.
- **Elastic Troughtput** - automatically scales up or down based on our workloads. Used for unpredictable workloads.

Storage Classes

- **Standard** - for frequently accessed files. Data is redundantly stored across multiple Availability Zones within an AWS Region.
- **Infrequent Access (IA)** - for files that are accessed less frequently but require the same durability and availability as the EFS Standard class. It has lower price to store, but has retrieval fee.
- **Archive** - for rarelly accessed data (few times each year). It is 50% cheaper.

We need to implement lifecycle policies to move files between storage tiers.

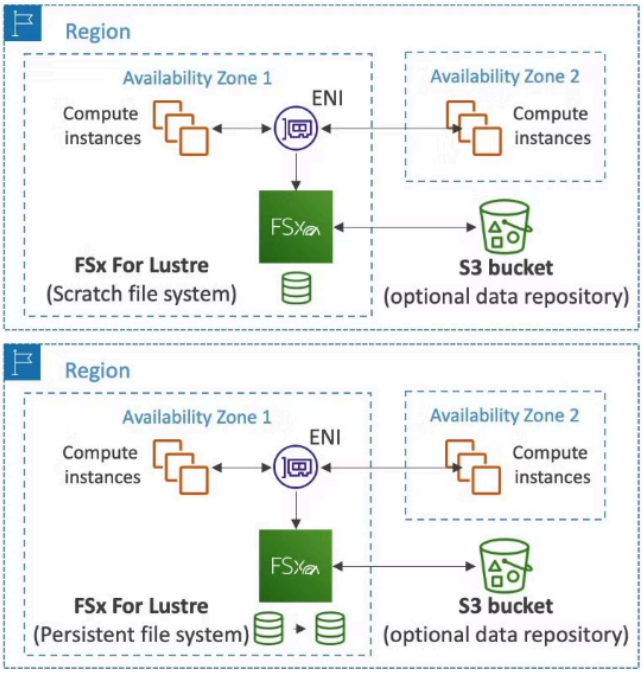


▼ **Amazon FSx.**

Amazon FSx is a fully managed service that lets us launch third party high-performance file systems on AWS.

FSx System Deployment Options

- **Scratch File System** - temporary storage. Data is not replicated (doesn't persist if file server fails). It has high burst and it is used for short-term processing.
- **Persistent File System** - long-term storage. Data is replicated within same AZ. It replaces failed files within minutes. Used for long-term processing and sensitive data.

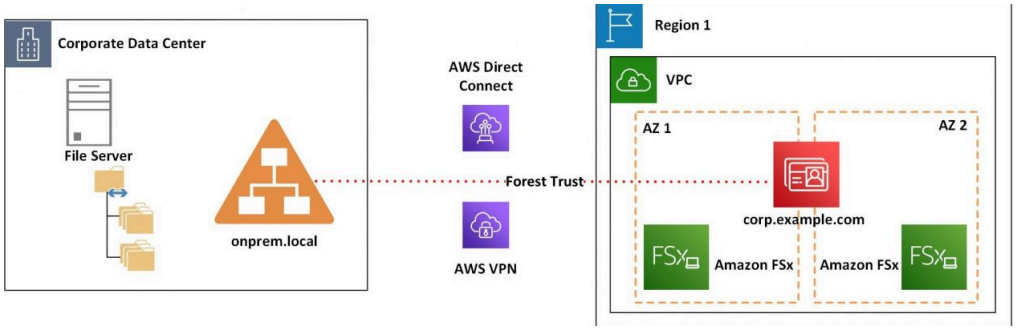


▼ **FSx - Windows File Server.**

FSx for Windows File Server - supports SMB and NTFS protocols. It can be mounted on Linux EC2 instances. We can integrate it with Microsoft AD, ACLs and user quotas. It supports Microsoft's Distributed File System (DFS) Namespaces.

We can choose SSD or HDD for storage options. FSx for Windows can be accessed from our on-premises infrastructure (VPN or DX). Can be configured to be Multi-AZ and all data is backed-up daily to S3.

FSx for Windows File Server **can be joined to an existing on-premises Active Directory**. This allows the file system to continue using the same AD groups and policies to restrict access to the shares, folders, and files, just as it did on-premises.

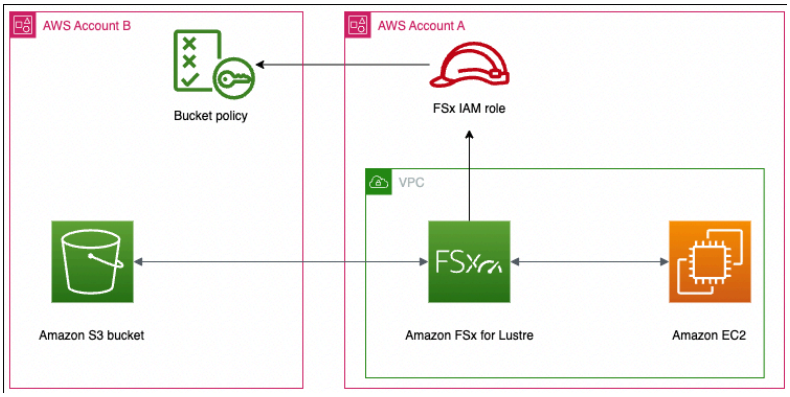


▼ **FSx - Lustre.**

FSx for Lustre - type of **parallel** distributed file system, for large-scale computing. It is commonly used for HPC and uses ML. We can chose between SSD and HDD storage options. Can be used from on-premises servers (VPN or Direct Connect).

FSx for Lustre uses a striped architecture, where data is distributed across multiple disks to optimize performance.

We can read S3 as a file system through FSx and write the output of the computations back to S3 (through FSx).



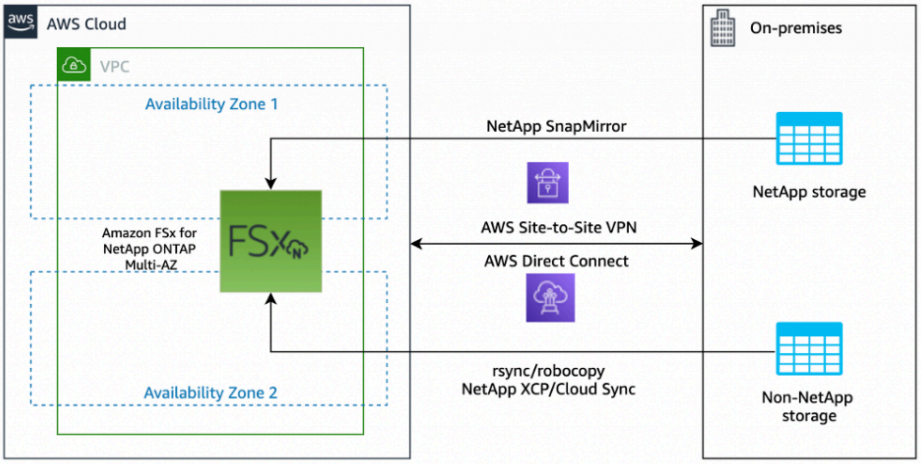
▼ **FSx - NetApp ONTAP & OpenZFS.**

FSx for NetApp ONTAP - managed NetApp ONTAP on AWS. It moves workloads running on ONTAP or NAS to AWS. It is compatible with NFS, SMB, iSCSI protocol.

Its storage shrinks or grows automatically and we can do snapshots, replication, data de-duplication and compression. We can do point-in-time instantaneous cloning (helpful for testing new workloads).

FSx for NetApp ONTAP offers a feature called Auto Tiering, which helps optimize storage performance and cost by automatically managing data across different storage tiers based on usage patterns.

ONTAP SnapMirror - a replication technology in ONTAP that allows us to asynchronously replicate data between two storage systems.



FSx for OpenZFS - managed OpenZFS file system on AWS. It is compatible with NFS and moves workloads running on ZFS to AWS. We can do point-in-time instantaneous cloning. Also can do snapshots and compression.

Data Streams

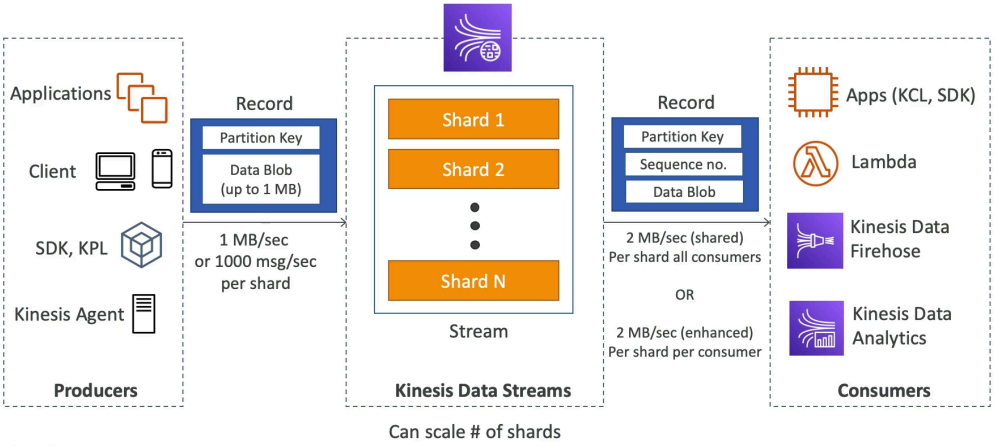
▼ **Kinesis Data Streams (KDS).**

KDS is a real-time data streaming service designed to handle massive streams of data in real-time. We have ability to reprocess data. Once data is inserted in Kinesis, it cant be deleted.

Key Concepts

- **Stream** - a collection of records that are distributed across multiple shards. Each stream has a sequence of data records.

- **Shard** - the fundamental unit of capacity within a stream. A stream can have multiple shards, allowing for scaling up.
- **Record** - a unit of data in a stream. It consists of:
  - Data Blob - data we want to store.
  - Partition Key - string used to group records by shard within a stream.
  - Sequence Number - an identifier assigned to each record, unique per shard, that is used to maintain order.
- **Producers** - clients or services that write data to the KDS (AWS SDK, KPL, Kinesis Agent).
- **Consumers** - applications or services that process or consume the data (AWS SDK, KCL, AWS Lambda, KDF, KDA).



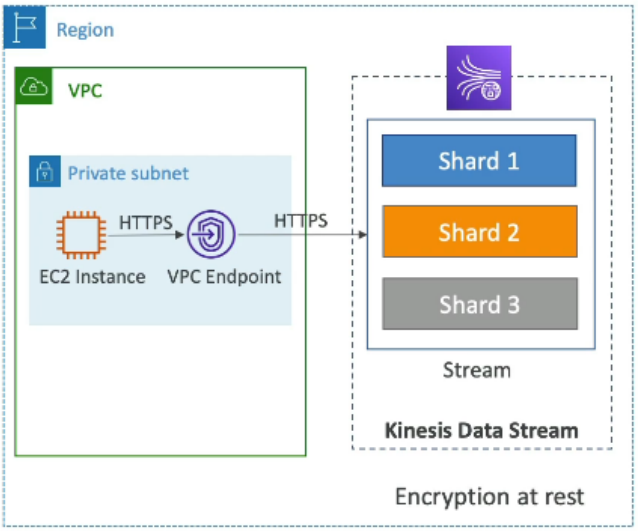
KDS Capacity Modes

- **Provisioned mode** - we choose the number of shards provisioned, scale manually or using API. We pay per shard provisioned per hour.
- **On-demand mode** - no need to provision or manage the capacity. Scales automatically based on observed throughput peak during the last 30 days. We pay per stream per hour & data in/out per GB.

KDS Security

We can control access/authorization using IAM policies. Encryption in flight is done using HTTPS endpoints and at rest using KMS. Also we can implement encryption/decryption of data on client side.

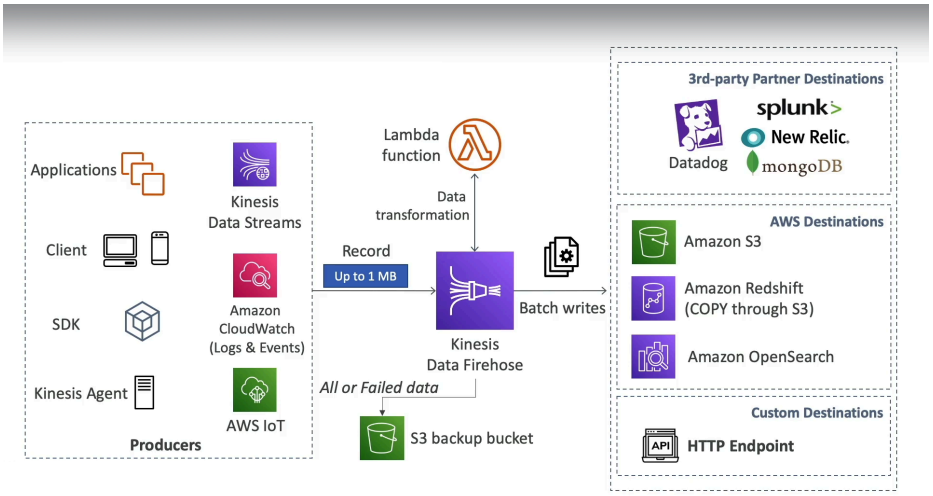
We can use VPC Endpoints for Kinesis to access within VPC. To monitor API calls we use CloudTrail.



▼ **Kinesis Data Firehose (KDF).**

KDF is a fully managed service that makes it easy to reliably load streaming data into data lakes, data stores, and analytics services. We pay for data going through Firehose.

It supports many data formats, conversions, transformations and compresion. It supports custom data transformations using AWS Lambda. Firehose can send failed or all data to a backup S3 bucket.



Because we write data in batches, KDF is near real time. If we set buffer interval to zero seconds (no buffering) it will be still considered near real time.

Kinesis Data Streams	Kinesis Data Firehose
• Streaming service for ingest at scale	• Load streaming data into S3 / Redshift / OpenSearch / 3rd party / custom HTTP
• Write custom code (producer / consumer)	• Fully managed
• Real-time (~200 ms)	• Near real-time
• Manage scaling (shard splitting / merging)	• Automatic scaling
• Data storage for 1 to 365 days	• No data storage
• Supports replay capability	• Doesn't support replay capability

Kinesis Troubleshooting.

Troubleshooting KDS Producers

1. **Writing is too slow**

   Possible Cause: Service limits exceeded or shard throughput limits.

   Solution: Check throughput exceptions and ensure partition keys distribute writes evenly across shards.

2. **Large producers (thousands of events per second)**

   Possible Cause: Sending many individual records.

   Solution: Batch records using Kinesis Producer Library (KPL) or `PutRecords`, or aggregate records into larger batches.

3. **Small producers (generate tiny pieces of data)**

   Possible Cause: Inefficient single record writes.

   Solution: Use `PutRecords` or Kinesis Recorder in AWS Mobile SDK.

4. **Stream returns 500 or 503**

   Possible Cause: Temporary Kinesis service errors.

   Solution: Implement retry logic when `AmazonKinesisException` occurs.

5. **Connection errors from Flink to Kinesis**

   Possible Cause: VPC/network configuration issues or insufficient resources.

   Solution: Verify VPC configuration and resource availability in Flink environment.

6. **Timeout errors from Flink to Kinesis**

   Possible Cause: Request timeout or queue limits.

   Solution: Increase `RequestTimeout` and adjust `setQueueLimit` in `FlinkKinesisProducer`.

7. **Throttling errors**

   Possible Cause: Too many requests to a shard.

   Solution: Use exponential backoff, improve partition key distribution, monitor for hot shards, and apply rate limiting.

Troubleshooting KDS Consumers

1. **Records skipped (KCL)**

   Possible Cause: Unhandled exception in `processRecords`.

   Solution: Check and handle exceptions inside `processRecords`.

2. **Same shard processed by multiple processors**

   Possible Cause: Worker failover occurred.

   Solution: Adjust failover time and handle shutdown reason **ZOMBIE.**

3. **Reading is too slow**

   Possible Cause: Too few shards or inefficient consumer.

   Solution: Increase shard count, increase `maxRecords`, optimize consumer code.

4. **Shard iterator expires**

   Possible Cause: Low DynamoDB capacity for KCL metadata.

   Solution: Increase write capacity for DynamoDB shard table.

5. **Processing falling behind**

   Possible Cause: Insufficient resources or high data rate.

   Solution: Increase retention period, add resources, monitor `IteratorAgeMilliseconds` and `MillisBehindLatest`.

6. **Lambda consumer not invoked**

   Possible Cause: Permission issue or timeout.

   Solution: Check execution role permissions and Lambda timeout.

7. **Concurrency limits exceeded (Lambda)**

   Possible Cause: Too many parallel invocations.

   Solution: Increase concurrency limits or reduce event rate.

8. **ProvisionedThroughputExceededException**

   Possible Cause: Shard read throttling.

   Solution: Reshard stream, reduce `GetRecords` calls, **use enhanced fan-out**, apply retries with exponential backoff.

9. **High latency**

   Possible Cause: Insufficient shards or compute resources.

   Solution: Monitor `GetRecords.Latency` and `IteratorAge`, increase shards or resources.

10. **500 errors**

    Possible Cause: High service error rate.

    Solution: Implement retry mechanism.

11. **KCL application stuck / blocked**

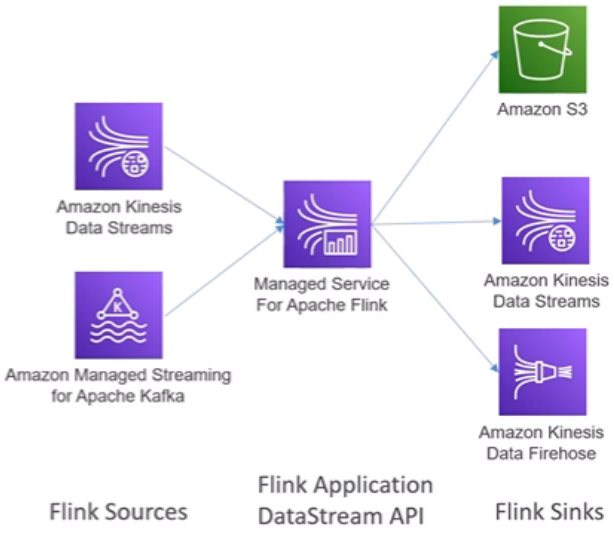
    Possible Cause: Inefficient record processing.

    Solution: Optimize `processRecords`, increase `maxLeasesPerWorker`, enable KCL debug logs.

▼ **Amazon Managed Service for Apache Flink (MSAF).**

MSAF was formerly KDA for Apache Flink. KDA always used Flink under the hood. It integrates Flink with AWS.

> Flink is a open-source stream processing framework that provides advanced capabilities for managing state, fault tolerance, and complex event processing.



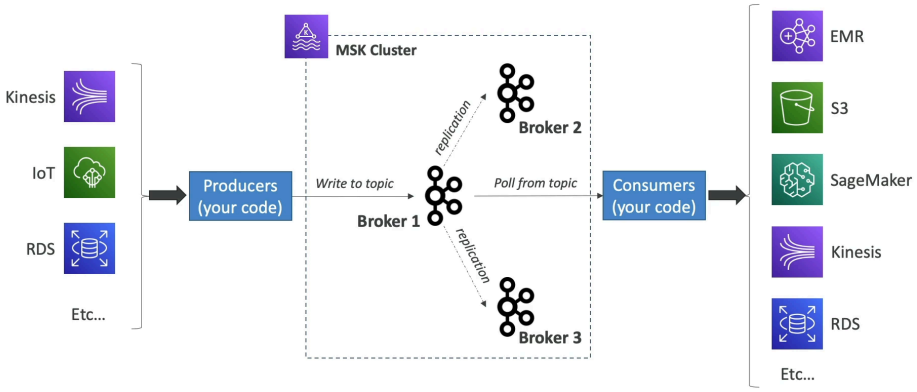
Usage: Streaming ETL, continous metric generation, responsive analytics.

Amazon Managed Service for Kafka (MSK).

MSK is an alternative to Amazon Kinesis. It is fully managed Apache Kafka on AWS. It allows us to create, update and delete clusters.

MSK creates and manages Kafka brokers nodes and Zookeeper nodes for us. We can deploy the MSK cluster in our VPC and multi-AZ. Data is stored in **EBS** volumes for as long as we want.

Default message size is 1MB. It is possible ot send larger messages into Kafka after custom configuration.



MSK Security

- Optional in-flight using TLS between the brokers.
- Optional in-flight using TLS between the clients.
- Encryption at rest using KMS.

We can autorize specific security groups for our Kafka clients.

Category	Method	Description	Typical Use
Authentication	**IAM AuthN**	Clients authenticate using IAM users or roles from AWS Identity and Access Management	AWS services (EC2, Lambda, containers) connecting to MSK
Authentication	**TLS / mTLS**	Client authenticates using a TLS certificate	High-security environments or enterprise integrations

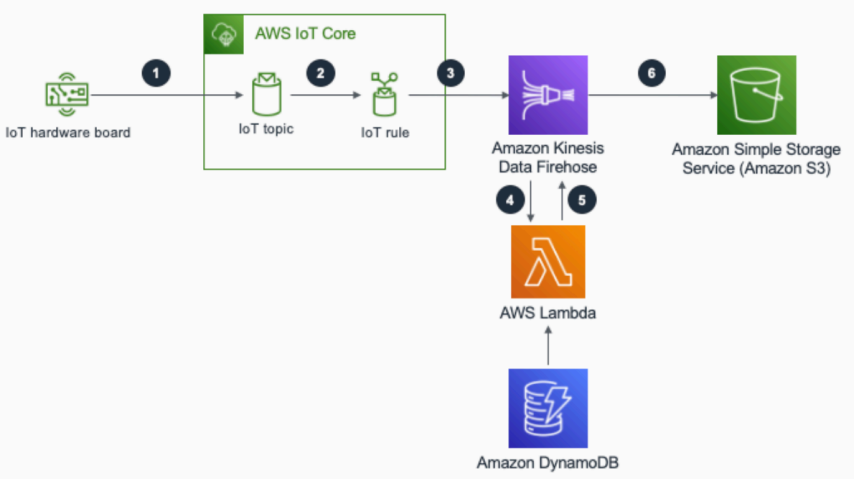
Category	Method	Description	Typical Use
Authentication	**SASL/SCRAM**	Username/password authentication stored in AWS Secrets Manager	Traditional Kafka clients
Authorization	**IAM AuthZ**	Controlled using IAM policies (produce, consume, describe cluster)	Best for AWS-native architectures
Authorization	**Kafka ACLs**	Define permissions on topics, consumer groups, and cluster operations	Standard Kafka permission model

MSK Serverless - run Apache Kafka on MSK without managing the capacity.

Kinesis Data Streams vs MSK

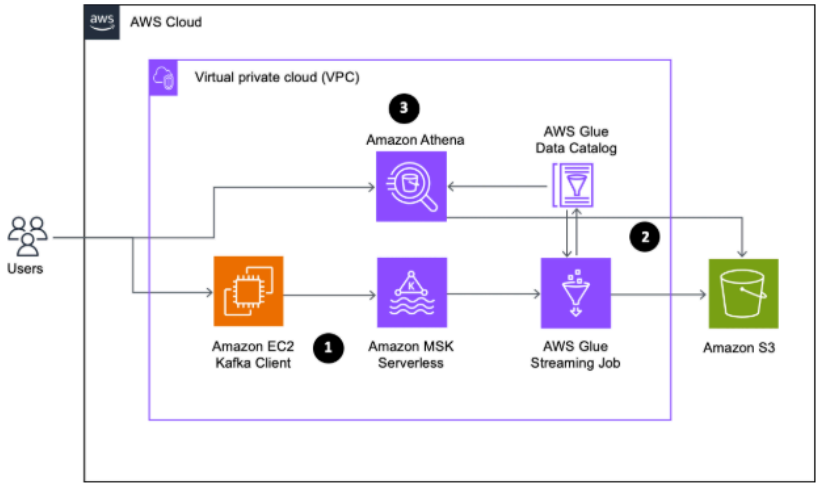
Feature	MSK	KDF
Main Purpose	Real-time event streaming	Deliver streaming data to storage/analytics
Message Size Limit	**> 1MB** per record	**1 MB** per record
Consumers	Multiple consumer groups can read same topic	No consumers - data delivered directly to destination
Data Retention	Configurable (hours to days)	Temporary buffering only
Destinations	Applications, stream processors	S3, Redshift, OpenSearch, Splunk
Processing	Supports complex processing (Kafka Streams, Flink, apps)	Limited transformation (Lambda)
Infrastructure	Requires Kafka cluster	Fully serverless
Scaling	Manual/automatic broker scaling	Fully automatic
Latency	Very low (milliseconds)	Higher due to buffering

▼ **Solution Architecture - Near-Real-Time Ingestion**



▼ **Solution Architecture - Stateful Streams - MSK Serverless + Glue ETL Pipeline**

- Glue Streaming ETL is good choice for stateful stream processing.
- If we have to reuse the transformation logic from historical backfills, Apache Spark is good choice for Glue Job.



Data Transformation, Integrity & Feature Engineering

EMR.

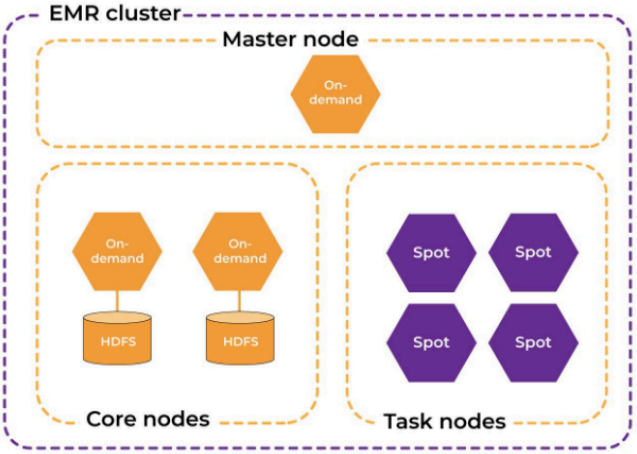
EMR helps creating Hadoop clusters (Big Data) to analyze and process vast amount of data. The clusters can be made of hundreds of EC2 instaces. It takes care of all the provisioning and configuration. It has auto-scaling and it's integrated with Spot instances.

EMR supports Apache Spark, HBase, Presto, Flink...

Usage: data processing, log analysis, ML, web indexing, big data ...

EMR Node Types

- **Master Node** - manages and coordinates the cluster (long running).
- **Core Node** - runs tasks and stores data (long running).
- **Task Node** (optional) - just to run tasks (short running).



EMR Cluster Types

- **Long-running Cluster** - designed for ongoing data processing needs. They remain active for extended periods, allowing users to submit multiple jobs over time without the overhead of cluster startup and configuration.
- **Transient Cluster** - designed for short-lived, one-off jobs. They are created for specific processing tasks and terminated immediately after the job is complete.

Purchasing Options

- **On-demand** - reliable, predictable, wont be terminated.
- **Reserved** (min 1 year) - cost savings (EMR will automatically use if available).
- **Spot Instances** - cheaper, can be terminated, less reliable.

EMR Storage

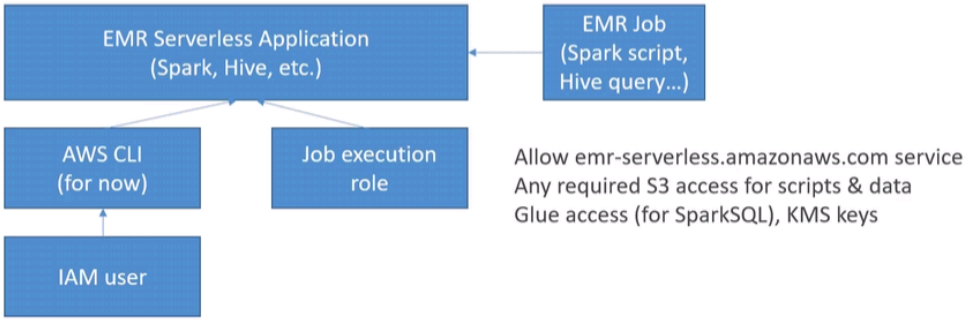
- **HDFS** - used when need very fast processing during the cluster lifetime. Data is lost if the cluster is terminated.
- **EMRFS (S3 as HDFS)** - use when data must persist beyond the cluster lifetime. Multiple clusters can access the same data.

  EMRFS Consistent View is a feature that ensures consistent reads and writes when EMR clusters access data stored in S3 through EMRFS.
- Local File System - use when applications need node-local temporary storage. Data is stored on the instance's local disk and not distributed.
- EBS for HDFS - use when we need larger or more reliable storage for HDFS.

▼ EMR Serverless.

Amazon EMR Serverless is a deployment option for Amazon EMR that abstracts away the underlying infrastructure. It is beneficial for customers who prioritize ease of operation for their open-source analytics applications.

- **Application** - a logical entity that represents a specific runtime environment. We define the Amazon EMR release version and the open-source framework.
- **Job** - a request submitted to an EMR Serverless application to execute a script or a query. EMR Serverless executes these jobs asynchronously.
- **Workers** - the underlying compute resources that EMR Serverless uses to execute workloads.



EMR Serverless supports popular open-source frameworks like Apache Spark and Apache Hive.

Script

This Bash starts a serverless Spark job on AWS EMR Serverless using the AWS CLI, and stores the response in a variable called JOB_RUN_OUTPUT.

```
JOB_RUN_OUTPUT=$(aws emr-serverless start-job-run \
  --application-id "$APPLICATION_ID" \
  --execution-role-arn "$JOB_ROLE_ARN" \
  --job-driver '{
    "sparkSubmit": {
      "entryPoint": "s3://us-east-1.elasticmapreduce/emr-containers/samples/wo
```

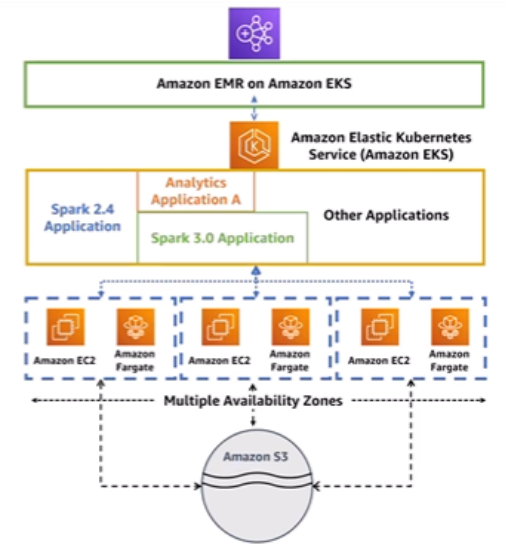
```
rdcount/scripts/wordcount.py",
      "entryPointArguments": ["'"$S3_BUCKET"'/output/wordcount_results"],
      "sparkSubmitParameters": "--conf spark.executor.cores=4 --conf spark.dri
ver.cores=2 --conf spark.driver.memory=4g --conf spark.executor.memory=8g --co
nf spark.executor.instances=3"
    }
  }' \
  --configuration-overrides '{
    "monitoringConfiguration": {
      "s3MonitoringConfiguration": {
        "logUri": "'"$S3_BUCKET"'/logs/"
      }
    }
  }')
```

Spark adds 10% overhead to memory requested for drivers and executors. We must be sure to **set initial capacity at least 10% more than requested by the job**.

```
aws emr-serverless create-application \
    --type "SPARK" \
    --name "my_application_name" \
    --release-label "emr-6.5.0-preview" \
    --initial-capacity '{
        "DRIVER": {
            "workerCount": 5,
            "resourceConfiguration": {
                "cpu": "2vCPU",
                "memory": "4GB"
            }
        },
        "EXECUTOR": {
            "workerCount": 50,
            "resourceConfiguration": {
                "cpu": "4vCPU",
                "memory": "8GB"
            }
        }
    }' \
    --maximum-capacity '{
        "cpu": "400vCPU",
        "memory": "1024GB"
    }'
```

EMR on EKS

We can submit Spark job on EKS without provisioning cluster, and share resources between Spark and other apps on Kubernetes.



▼ Apache Spark on EMR.

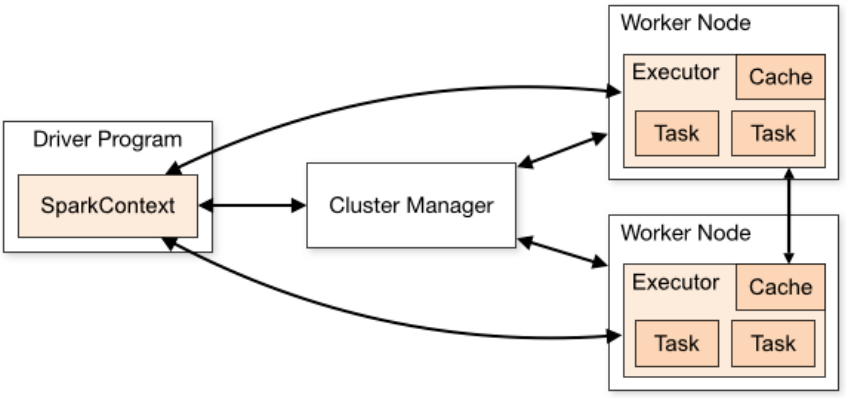
Spark Components

Spark Driver - the process that runs the main() function of our application. It converts the user's code into a logical plan, creates the SparkContext, breaks the program into tasks, and schedules these tasks across the executors.

> If the Driver crashes, the entire application fails. If the Driver is overloaded, the job will fail.

Spark Executors (Workers) - processes that run on worker nodes in the cluster. They execute the tasks assigned by the Driver, store data in memory or on disk, and report results back to the Driver.

Cluster Manager - an external service that manages resources (CPU/RAM) on the physical or virtual machines in the cluster.



Execution Model

Spark does not execute code immediately. Instead, it builds a DAG of operations. When we define a map() or filter(), Spark just records the intent.

- **Transformations**: (map, filter, join) are lazy. They do not trigger execution.
- **Actions**: (collect, count, saveAsTextFile, write) are eager. They trigger the execution of the DAG.

Spark **handles node failure automatically**. If an executor dies, the Driver uses the Lineage (a record of how the data was created) to recompute only the lost data partitions.

The Ecosystem

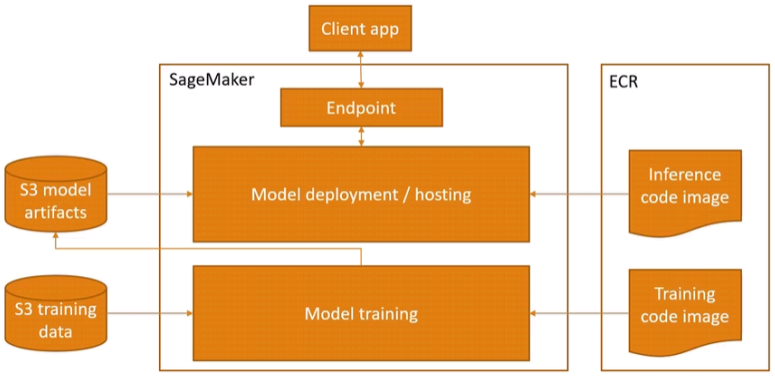
Spark SQL - a module for working with structured and semi-structured data. It introduces the DataFrame and Dataset APIs. It allows us to run SQL queries on your data and provides an optimized execution engine.

Spark Streaming - a component for processing real-time data streams. It enables us to process data as it arrives (from sources like Kafka, Kinesis, or TCP sockets).

MLlib - Spark's scalable machine learning library. It provides a suite of common ML algorithms designed to run on a distributed cluster. It also includes tools for feature engineering, pipeline construction, and model evaluation.

Amazon SageMaker.

Amazon SageMaker is a comprehensive, fully managed platform that provides the infrastructure and tools for every step of the ML lifecycle.



We can use SageMaker's built-in algorithms, or we can bring our own frameworks (PyTorch, TensorFlow) and even our own custom Docker containers.

SageMaker Domain - organizational entity we must create to use SageMaker Studio. It acts as a logical container (organization). Users within the same domain share notebook files and artifacts with one another.

AWS creates a managed EFS volume attached to the domain (persistent storage for all users within that domain). Each user gets their own private home directory on this volume. We manage our team by creating User Profiles within the domain. Each profile represents a specific person who will be accessing studio.

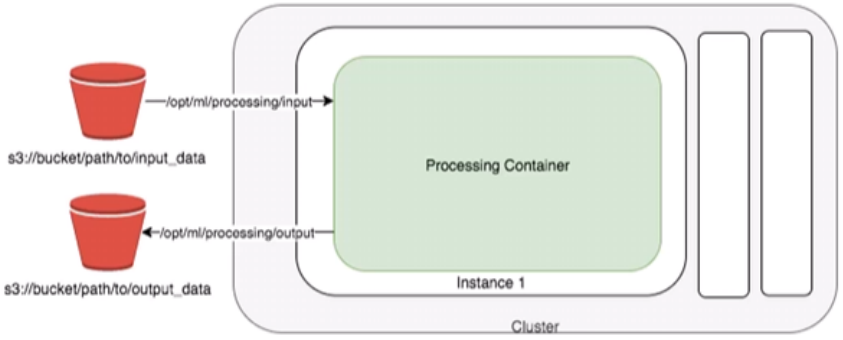
We can configure the domain to run within our own VPC, allowing us to restrict network access. By default, a domain has two VPCs (one managed by SageMaker, and another our own).

Data Processing, Training & Deployment with SageMaker.

Every step in the cycle reads data from S3, does its job on temporary compute instances, and writes the results back to S3.

Data Processing

We use a feature called **SageMaker Processing Jobs**. The raw data sits in an S3 bucket. We write a Python script to clean the data. Then trigger a Processing Job. SageMaker automatically spins up EC2 instances, loads a Docker container, downloads raw data from S3, and runs our script. The script outputs the cleaned, ready-to-train data.



SageMaker saves this cleaned data back to a new S3 location and shuts down the EC2 instances so we stop paying for them.

Training

We use **SageMaker Training Jobs**. We configure a Training Job by pointing SageMaker to three things:

1. The S3 location of processed data.
2. The algorithm to use (a Docker container in Amazon ECR). This can be a SageMaker built-in algorithm, a framework, or our own custom container.
3. The compute resources we want.

Deployment

SageMaker offers multiple deployment options depending on our latency needs.

- **Real-Time Endpoints (for live web apps)** - SageMaker provisions persistent EC2 instances behind a secure HTTPS endpoint. It loads our model.tar.gz from S3 into the instance's memory.

  Payload Size: 25MB, Max Time: 60s

- **Batch Transform (for bulk, offline predictions)** - SageMaker spins up instances, downloads the data, runs the predictions, saves the results to a new S3 folder, and shuts the instances down. No endpoint is created.

  Example: We have a massive file of new data in S3 and we want predictions for all of them by tomorrow morning.

- **Serverless Inference** - good for unpredictable traffic with idle periods (scales to zero, we pay per invocation).

- **Asynchronous Inference** - good for large payloads or models that take a long time to process (like deep learning image processing).

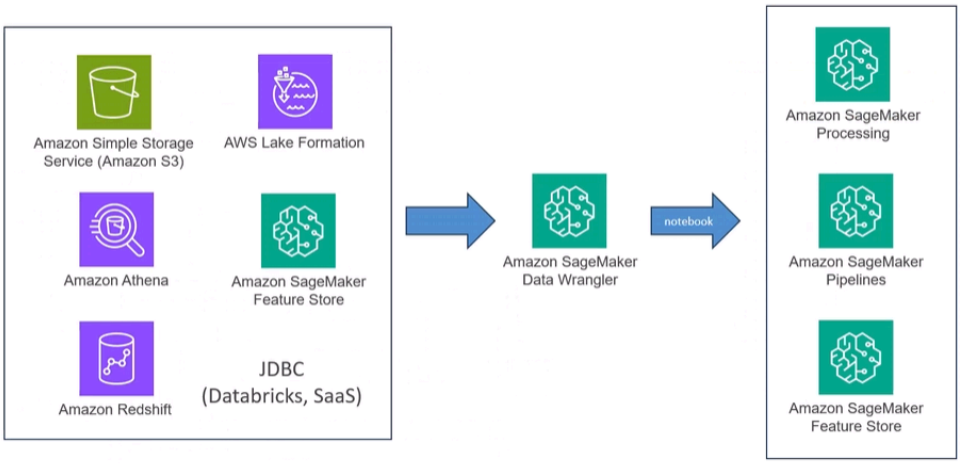
  Payload Size: 1GB, Max Time: 1h

SageMaker Data Wrangler.

Data Wrangler is a visual tool for preparing and transforming data before we use it to train ML models. It lets us clean, explore, and transform datasets from multiple sources without writing much code. It can directly import data from Amazon S3, Amazon Athena, Amazon Redshift, Snowflake, and Databricks.

It includes over 300 built-in transformations. If a built-in transform doesn't exist, we can add custom Pandas, PySpark, or PySparkSQL snippets.

It includes built-in analyses that help us generate visualizations and data analyses.



It can automatically detect Target Leakage (when a feature in dataset accidentally reveals the target variable, making model look perfectly accurate during training).

Sampling & Ingestion Options

When importing files, Data Wrangler provides sampling controls to manage memory and performance:

- **First K rows**: Reads the first K rows of the dataset (fastest).
- **Random sample**: Samples a specified percentage or fixed row count randomly across the dataset.
- **Stratified sample**: Samples proportionally based on a specified categorical column to preserve class distribution.
- **Full dataset**: Imports the entire dataset without sampling.

SageMaker Feature Store.

SageMaker Feature Store is a centralized repository to store, share, and manage features for ML. It solves the problem where different teams recreate the same data processing logic multiple times.

Different teams can discover and use existing features. It also allows us to "travel back in time" to see what a feature looked like at a specific timestamp. This prevents data leakage

- **Online Store** - high-throughput, low-latency. Used for real-time inference (powered by **DynamoDB** or Amazon MemoryDB).
- **Offline Store** - low-cost, high-volume storage in S3. Used for batch training and historical analysis (data is stored in Parquet, searchable via Athena).

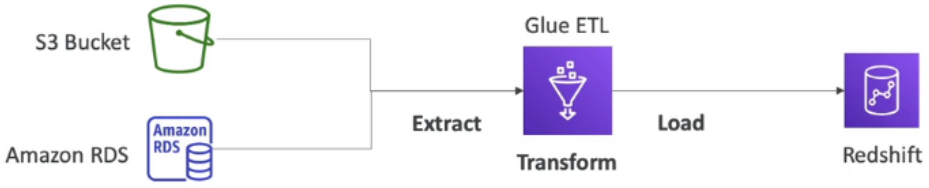
It uses AWS KMS for encryption at rest and IAM for access control.

SageMaker Canvas.

SageMaker Canvas is a no-code, visual interface that allows business analysts to build ML models without writing a single line of code.

Amazon Glue.

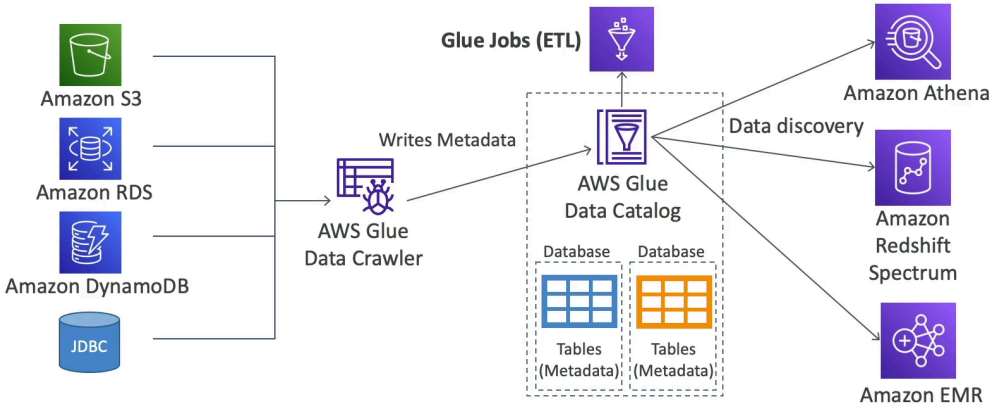
AWS Glue is a serverless fully managed **ETL** service. It is useful for preparing and transforming data for analytics.



Glue Components

- **Data Catalog** - central repository that stores metadata about datasets in our AWS environment. Can be used by Athena, Redshift or EMR.
- **ETL Engine** - automatically generates Python or Scala code to extract data from various sources, transform it, and load it into target data stores.
- **Crawlers** - automated processes in AWS Glue that scan and infer the schema of data stored in various sources.

  Glue crawler will extract partitions based on how our S3 data is organized.
- **Glue Jobs** - ETL scripts that users create to transform data. These jobs can be scheduled and monitored within AWS Glue.
- **Glue DynamicFrames** - glue's custom version of a Spark DataFrame. It is specifically designed to handle messy data with inconsistent schemas without crashing.



Glue Advanced Features

Glue Job Bookmarks - save and track the data that has already been processed, prevent re-processing old data.

Glue Elastic Views - combine and replicate data across multiple data stores using SQL. No custom code is needed, Glue monitors for changes in the source data (serverless).

Glue DataBrew - a visual data preparation tool. It has over 250 read-made transformations. We can create recipes of transformations that can be saved as jobs within a larger project, and create datasets

with custom SQL from Redshift and Snowflake. DataBrew is **strictly for batch data**. It cannot connect to a stream.

Glue Studio - used to create, run and monitor ETL jobs in Glue.

Glue Streaming ETL - run Glue jobs as streaming jobs. compatible with KDS, Kafka, MSK

Glue FindMatches - designed to identify duplicate or matching records in datasets without requiring labeled training data. It uses machine learning to find fuzzy matches.

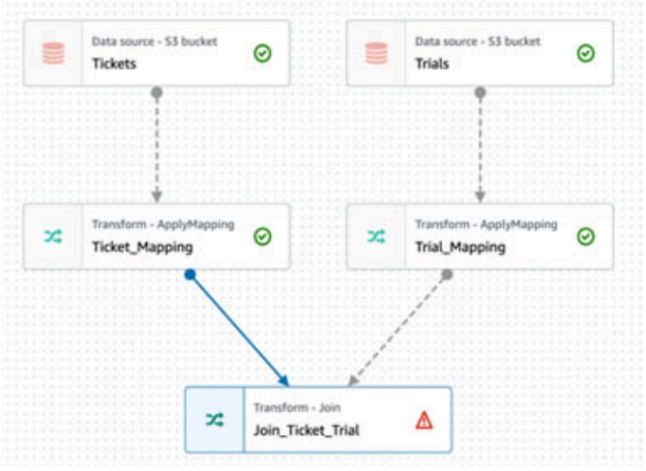
Glue PySpark Jobs vs Lambda

PySpark provides distributed processing via Apache Spark, which makes Glue ideal for handling large datasets and complex transformations.

Feature	AWS Glue PySpark Jobs	AWS Lambda
Data Size	Large-scale data (GBs-TBs)	Small-medium data (up to a few GBs, constrained by memory and timeout limits)
Execution Time	Long-running jobs (up to hours)	Short-lived tasks (maximum 15 minutes)
Processing Type	Batch processing, complex ETL	Real-time, event-driven processing
Memory	Large memory and distributed computing available via Spark	Limited (up to 10 GB of memory)
Scalability	Automatically scales for large, distributed workloads	Automatically scales but limited by function constraints
Cost	Typically cost-effective for large datasets and long-running jobs	Cost-effective for short-lived, low-latency tasks
Integration with Other Services	Tight integration with S3, Redshift, RDS, Athena	Integration with many AWS services, especially for event-driven triggers

▼ **Glue Studio & Data Quality.**

Glue Studio is the visual interface for building, running, and monitoring Glue ETL jobs. It creates a DAG of our data flow. We see boxes for sources, transform, and target. Glue Studio generates the Scala or Python (PySpark) code for us in the background.



Glue Data Quality Rules - conditions we define to automatically validate and monitor the quality of data. Results can be used to fail the job or just be reported to CloudWatch.

Amazon Athena.

Athena is a serverless query service which is used to analyze data stored in S3. It uses standard SQL to query the files and supports CSV, JSON, ORC, Avro and Parquet. It is commonly used with Quicksight for reporting/dashboards.

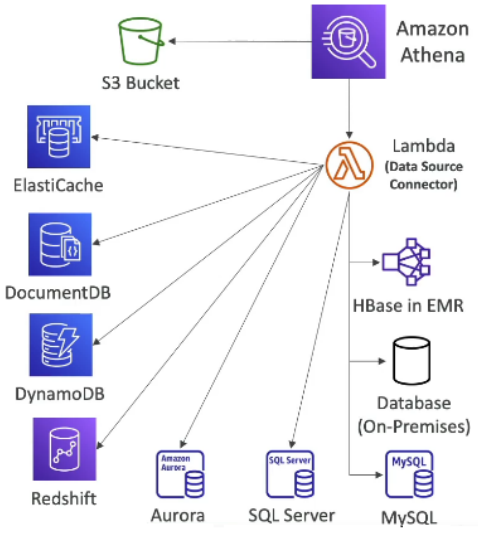
It uses the Glue Data Catalog as its central dictionary to understand the schema of the S3 files.

Use cases: business intelligence, analytics, reporting, analyzing querying VPC Flow Logs, CloudTrail trails, etc...

Athena Workgroups - allows us to organize, manage, and control queries for different teams, projects, or use cases. It integrates with IAM, CloudWatch and SNS.

We can set a limit on how much data a Workgroup can scan per query, per hour, or per day. If a query exceeds the limit, Athena cancels it automatically. Also we can force all users in a Workgroup to publish their query results to a specific, encrypted S3 bucket.

Athena Federated Query - allows us to run SQL queries across data stored in relational, non-relational, object and custom data sources. It uses Data Source Connectors that run on AWS Lambda and it stores results back in S3.



Athena Performance Improvement

- Use columnar data for cost-savings (less scan) - Apache Parquet or ORC is recommended. We can use Glue to convert our data to Parquet or ORC.

Dataset	Size on Amazon S3	Query Run time	Data Scanned	Cost
Data stored as CSV files	1 TB	236 seconds	1.15 TB	$5.75
Data stored in Apache Parquet format*	130 GB	6.78 seconds	2.51 GB	$0.01
Savings / Speedup	87% less with Parquet	34x faster	99% less data scanned	99.7% savings

- Compress data - for smaller retrievals.
- Partition datasets in S3 - easy querying on virtual columns (s3://athena-ex/flight/parquet/year=1995/month=1/day=1).
- Use larger files (> 128MB) to minimize overhead.

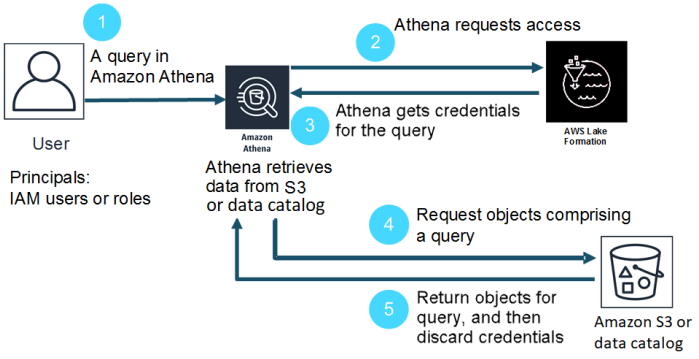
Athena Cost Model.

We pay for data scaned (usually ~5$ per TB). But there are edge cases:

1. **If a query fails due to an error we are NOT charged.**
2. If we start a massive query and then cancel it halfway through, we are charged for the amount of data scanned up to the exact moment of cancellation.
3. DDL Statements are FREE.
4. We pay Athena for the query, but we also pay standard S3 rates for GET requests and storing the query results in S3 bucket.

Athena Security.

To query data, an IAM user needs permission to use the Athena and permission to read the underlying S3. If they have Athena access but S3 explicitly denies them, the query fails.



We can encrypt output results in S3 using SSE-S3, SSE-KMS, or CSE-KMS.

Athena ACID Transactions.

Data Lakes were originally append-only. We couldn't easily update or delete a single row in S3 without rewriting the entire file.

Athena supports open-source transaction formats, specifically Apache Iceberg, Apache Hudi, and Linux Foundation Delta Lake. Concurrent users can safely make row-level modifications.

We can recover data recently deleted with a SELECT statement (time travel operations).

It is compatible with EMR, Spark, anything that supports Iceberg table format.

Handling PII in DataBrew Transformations.

Technique	Function Name(s)	Description	Typical Use Case
Substitution	`REPLACE_WITH_RANDOM`	Replaces original values with random ones	Anonymizing names, emails
Shuffling	`SHUFFLE_ROWS`	Randomly reorders rows to break linkage	De-identifying datasets while preserving distribution
Deterministic Encryption	`DETERMINISTIC_ENCRYPT`	Encrypts values consistently	Joining datasets securely
Probabilistic Encryption	`ENCRYPT`	Encrypts values with randomness	Stronger privacy protection
Decryption	`DECRYPT`	Reverses encryption to retrieve original data	Authorized data recovery
Nulling / Deletion	`DELETE`	Removes or nullifies sensitive fields	Dropping unnecessary PII
Masking	`MASK_CUSTOM`, `MASK_DATE`, `MASK_DELIMITER`, `MASK_RANGE`	Partially hides data	Displaying limited info
Hashing	`CRYPTOGRAPHIC_HASH`	Converts data into irreversible hash values	Identity matching without revealing raw data

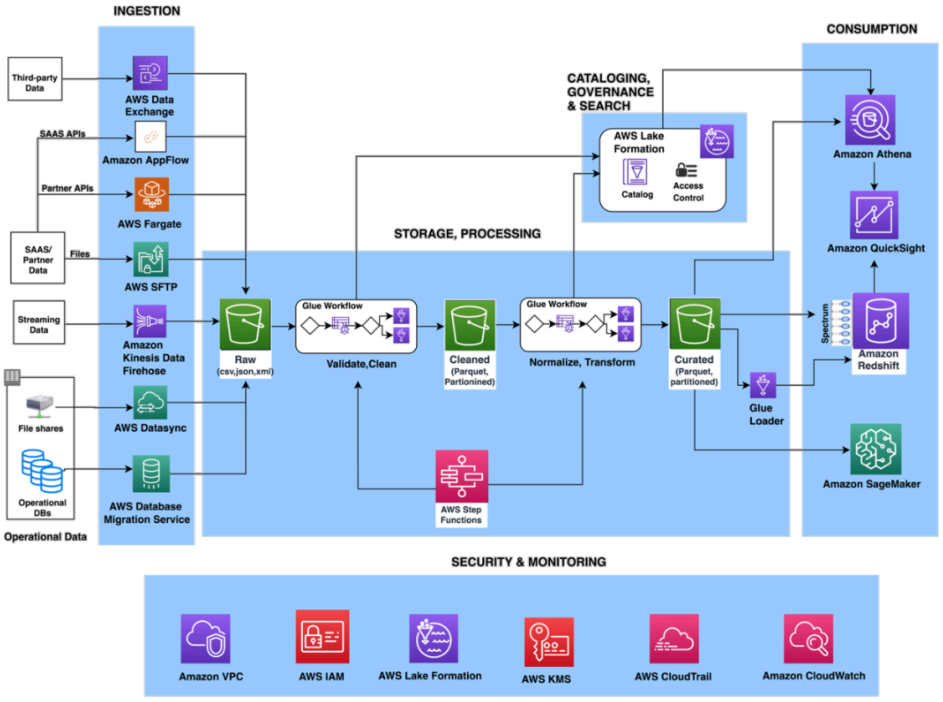
Athena CREATE TABLE AS SELECT.

CTAS (Create Table As Select) - a single SQL command that runs a query, saves the results to S3 in a new format, and automatically creates a new table in the Glue Data Catalog.

```
CREATE TABLE ml_features
WITH (
    format = 'PARQUET',
    external_location = 's3://bucket/features/'
) AS
SELECT col1, col2, col3 FROM raw_csv_table WHERE year = '2023';
```

It transforms data, converts formats, and partitions data all in one step, without needing to boot up a Glue Spark job.

▼ Solution Architecture - Serverless Data Lake & Discovery



▼ Visual Tools Cheat Sheet

Tool	Who is it for?	Best for...
Glue Studio	Data Engineers / MLEs	Building complex Spark-based ETL pipelines without writing manual code.
Glue DataBrew	Data Analysts	Cleaning data with a spreadsheet-like interface (250+ built-in transforms).
SageMaker Data Wrangler	ML Engineers	Preparing features **specifically for ML** inside SageMaker Studio (e.g., SMote, One-Hot Encoding).
SageMaker Canvas	Business Analysts	The entire ML lifecycle (AutoML), not just the ETL part.

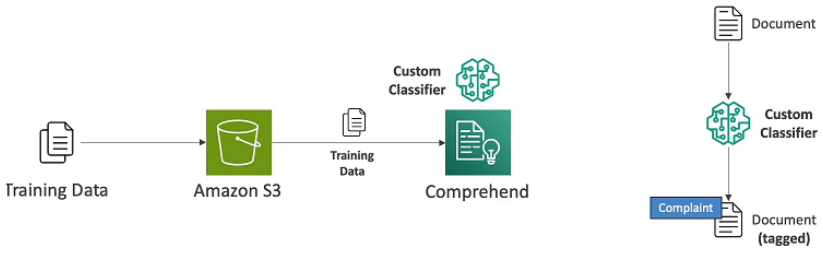
Section II

AWS Managed AI Services

▼ Amazon Comprehend.

Comprehend is a fully managed and serverless service used for NLP. It is used for extracting key phrases, places, people... It analyzes text using tokenization and parts of speech, and automatically organizes a collection of text files by topic.

Comprehend uses LDA (Latent Dirichlet Allocation) for unsupervised topic modeling. Because it is an unsupervised, it does not assign pre-defined human labels to the topics. It groups related documents together by identifying statistical clusters of words that frequently occur.



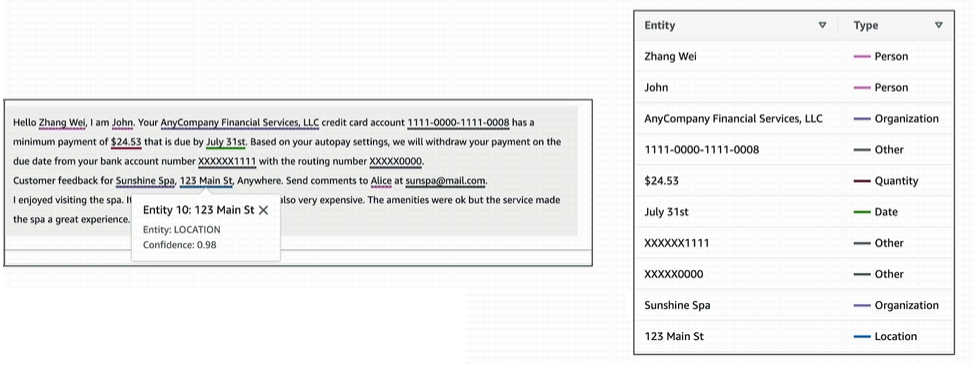
We can create custom models and share them between accounts, but both accounts must be in same region.

Synchronous (Real-Time) Analysis - call the API, the connection stays open, and we get the result back in milliseconds. To use real-time analysis with a Custom Model, we must create an endpoint.

> We provision throughput for these endpoints in Inference Units. If application is lagging or dropping requests, we should increase the number of IUs, or enable autoscaling for the endpoint.

Asynchronous (Batch) Analysis - Comprehend pulls the data from S3, processes it in the background, and writes the results back to a different S3 bucket.

Named Entity Recognition (NER) - a feature of Comprehend that takes a sentence and tags specific words with a category.

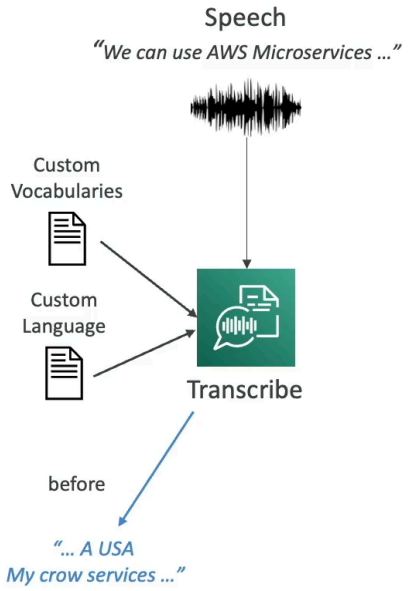


▼ Amazon Translate & Transcribe.

Translate provides us natural and accurate language translation. Allows us to localize content, such as websites and applications for international users.

Transcribe automatically converts speech to text. It uses DL process called automatic speech recognition to convert speech to text quickly and accurately.

- It can remove PII using redaction.
- It has ML-powered voice-based toxicity detection.
- We can create custom vocabularies (add specific words, phrases, domain-specific terms, etc...).
- We can create custom language models (train transcribe model on our own text data).



Amazon Polly.

Polly turns text into speech using DL. We can create applications that talk.

Lexicons - custom pronunciation dictionaries used with Polly to control how specific words or phrases are spoken.

Example: "AWS = Amazon Web Services".

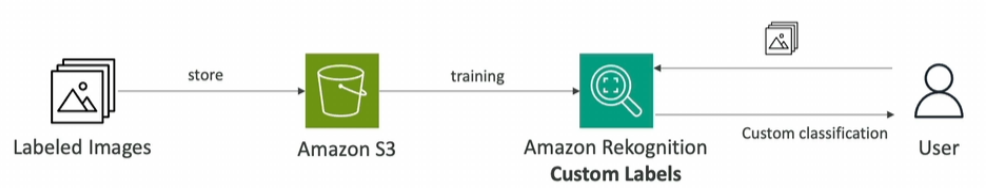
SSML (Speech Synthesis Markup Language) - language we can use to indicate Polly how to pronounce text.

Example: "Hello <break> how are you?".

Amazon Rekognition.

Rekognition finds objects, people, text, secenes in images and videos using ML. Can be used for object detection, facial recognition, facial analysis, content moderation, OCR, etc...

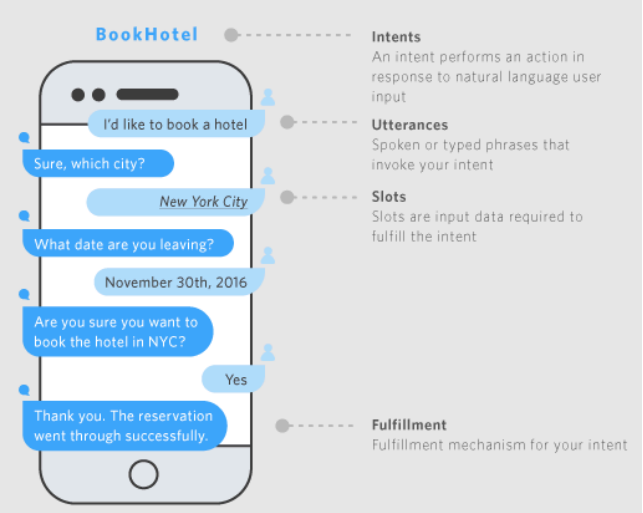
Rekognition Custom Labels - if we need to recognize something highly specific to our business. It uses Transfer Learning under the hood (AutoML for vision). We only need a small dataset to train it.



Rekognition Content Moderation - detects explicit, violent, or suggestive content. We can extend rekognition capabilities by providing our own labeled set of images (Custom Moderation Adaptors).

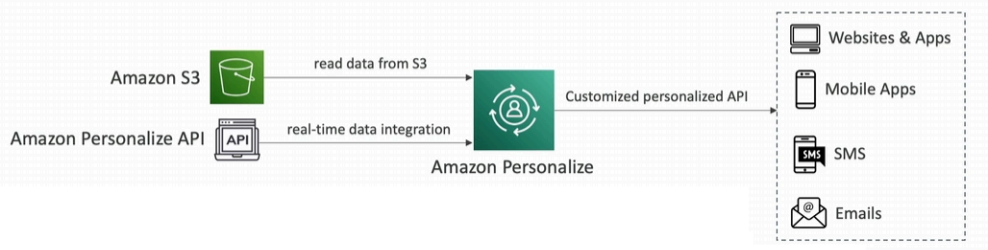
Amazon Lex.

Lex is a fully managed service for building chatbots into applications using voice and text.



▼ Amazon Personalize.

Personalize is a fully managed ML service to build apps with real-time personalized.



To use Personalize, we must provide data in these three specific schemas:

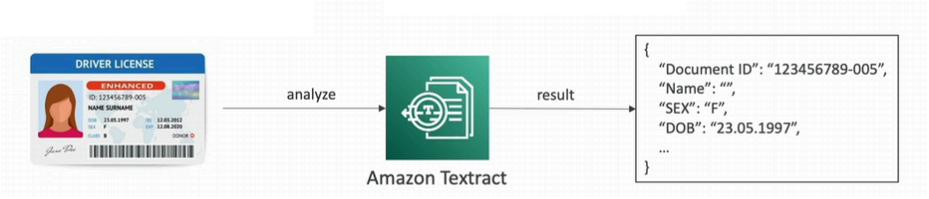
1. **Users Dataset (Optional)**: Metadata about users (age, gender, location, membership level).
2. **Items Dataset (Optional)**: Metadata about items (genre, price, brand, category).
3. **Interactions Dataset**: Historical data of who did what and when. The timestamp is critical.

Recipes - algorithms that are prepared for specific use cases. We must provide the training configuration on top of the recipe.

- **User-Personalization** - for "recommended for you" homepages. It predicts the items a specific user will interact with.
- **Personalized-Ranking** - takes a specific list of items and re-orders them specifically for a certain user.
- **Similar-Items** - for "customers who bought this also bought..." or "related items."
- **Next-Best-Action** - recommends the best action for a user.

▼ Amazon Textract.

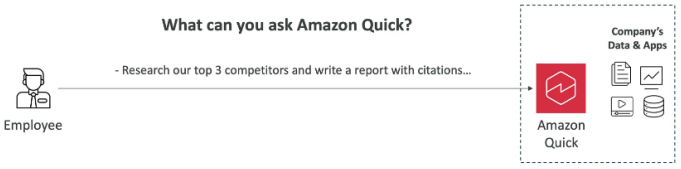
Textract automatically extracts text, handwriting and data from any scanned documents using ML. It does not have a native, built-in PII masking or redaction feature. We must must combine Textract with Comprehend.



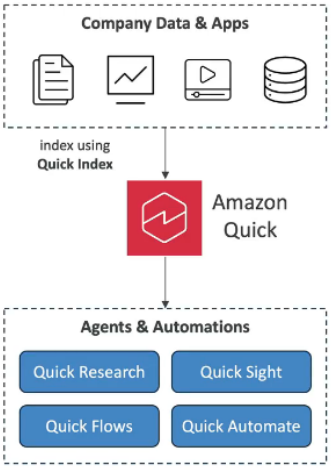
Amazon Quick.

Amazon Quick is an agentic AI assistant for our employees. It can answer, research, analyze and act. It is built on Bedrock, so our data is never used to train models.

We can build and publish web applications without coding by describing them in natural language.

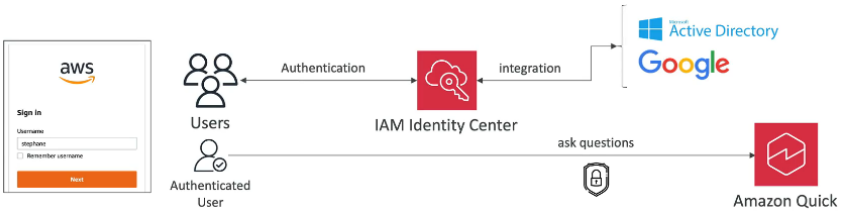


- **Quick Index** - unified, searchable knowledge base of our company data (fully managed RAG).
- **Quick Research** - deep research agent, combines internal data and the internet.
- **Quick Flows** - simple automations built with natural language.
- **Quick Automate** - multi-step agentic workflows across systems, with human approvals.
- **Quick Sight** - AI-powered BI dashboards.
- **Quick Spaces** - shared workspace of documents, dashboards and agents for a team.



Security

Users can be authenticated through IAM Identity Center. Users receive responses generated only from the documents they have access to.



Integrations

Quick can be integrated with Knowledge Bases (to index our company data into Quick Index) and Actions (to allow agents to perform tasks in third party services).

Amazon Hardware for AI.

Trainium and **Inferentia** are AWS-designed custom chips (ASICs) created specifically to reduce the high cost of training and deploying deep learning models.

AWS Inferentia - high-performance, low-latency ML Inference at the **lowest cost** in the cloud. **Use it when model is already trained, and we need to put it into production** as cheaply as possible without sacrificing speed.

AWS Trainium - high-performance ML Training for deep learning models. **Use it when training massive models** and want to save money.

AWS Neuron SDK - to run on Trainium or Inferentia, we must compile model using the Neuron Compiler (neuron-cc).

SageMaker Built-In Algorithms

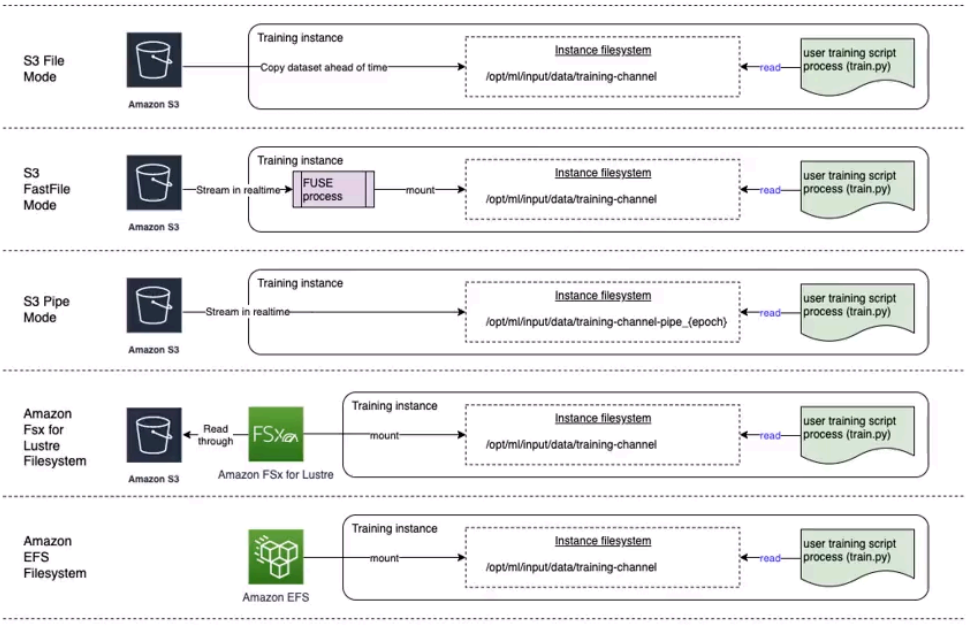
SageMaker Input Modes.

S3 File Mode (The Default) - SageMaker downloads the entire dataset from S3 onto the encrypted EBS volume of training instance before the training algorithm starts.

> Training won't start until the download is complete!

Pipe Mode - data is streamed directly from S3 into the training container using Unix Pipes (FIFO).

FastFile Mode - uses a FUSE-based system to mount the S3 bucket as a local file system. It looks like file mode, but it behaves like pipe mode.



Alternative High-Performance Storage Options

S3 Express One Zone - up to 10x faster than S3 Standard and consistent single-digit ms latency. It is the **only storage class that natively supports File, Fast File, and Pipe** modes with this level of performance.

FSx for Lustre - a high-performance parallel file system. It caches the S3 data onto its high-speed SSDs. It scales to 100's of GB of throughput and millions of IOPS with low latency. It **requires VPC**.

EFS - a shared file system. The **data must already be in EFS**. Moving data from S3 to EFS just for a training job is usually a waste of time, and **VPC must be configured**.

Comparison

Feature	File Mode	Pipe Mode	FastFile Mode
Startup Time	Slow (waits for download)	**Instant**	**Instant**
Disk Space Needed	**Full Dataset Size**	Minimal	Minimal
Access Pattern	Random Access (Anywhere)	**Sequential Only**	Random Access (Simulated)
Ease of Implementation	Very Easy	Complex (Requires SDK/Format)	Very Easy
Data Persistence	Stored on EBS	Not Stored	Cached On-Demand

▼ SageMaker Linear Learner.

Linear Learner fits a linear line (for regression) or a hyperplane (for classification) to the data. Unlike standard Scikit-Learn linear models, SageMaker's Linear Learner trains **multiple models in parallel** with different hyperparameters and then selects the best one based on the validation set.

Linear Learner expected formats: RecordIO-protobuf (Float32 only), CSV, File or Pipe Mode.

Linear Learner automatically normalizes data by default (`normalize_data=True`). We should shuffle our data before training.

Hyperparameters

Hyperparameter	Description	Typical Usage / Notes
`predictor_type`	Specifies the type of prediction task.	Must be set to `binary_classifier`, `multiclass_classifier`, or `regressor`.
`normalize_data`	Normalizes features to have mean 0 and standard deviation 1.	Default is `True`
`balance_multiclass_weights`	Adjusts class weights for imbalanced datasets.	Set to `True` when some classes are underrepresented.
`learning_rate`	Controls how quickly the model updates during training.	/
`mini_batch_size`	Number of samples used per training step.	Larger batches = faster computation, but may reduce generalization.
`wd`	L2 regularization parameter to prevent overfitting.	Penalizes large weights; helps improve generalization.
`l1`	L1 regularization parameter.	Encourages sparsity (useful for feature selection).
`binary_classifier_model_selection_criteria`	Defines the metric used to choose the best binary classification model.	Set to `precision_at_target_recall` or `recall_at_target_precision`.
`target_recall`	Minimum acceptable recall threshold.	Example: `0.8`; use when missing positive cases is costly (e.g., fraud detection, medical diagnosis).
`target_precision`	Minimum acceptable precision threshold.	Example: `0.9`; use when false positives are costly (e.g., spam filters, alerts).

Instance Types & Performance

It can scale across multiple GPUs on a single instance but does not natively support multi-node training for a single model in the same way algorithms like XGBoost do.

▼ XGBoost in SageMaker.

SageMaker XGBoost handles non-linear relationships much better than Linear Learner. It is highly robust against outliers and works exceptionally well with tabular data.

SageMaker XGBoost data formats: CSV (**no header, target variable must be in the first column**), Parquet/RecordIO-protobuf, LibSVM.

> Unlike Linear Learner, **XGBoost automatically handles missing values.**

Hyperparameters

Hyperparameter	Description	Typical Usage / Notes
`max_depth`	Maximum depth of each tree.	Higher values increase model complexity and risk of overfitting. Typical: 3–10.
`eta` (learning rate)	Step size shrinkage used during updates.	Smaller values improve robustness but require more `num_round`. Typical: 0.01–0.3.
`gamma`	Minimum loss reduction required to split a node.	Higher values make the algorithm more conservative (fewer splits).
`alpha` (L1)	L1 regularization term on weights.	Encourages sparsity; useful for feature selection.

Hyperparameter	Description	Typical Usage / Notes
`lambda` (L2)	L2 regularization term on weights.	Helps reduce overfitting by penalizing large weights.
`subsample`	Fraction of training data used per tree.	Typical: 0.5–1.0; lower values help prevent overfitting.
`num_round`	Number of boosting rounds (trees).	More rounds = better learning but higher risk of overfitting.
`objective`	Defines the learning task and output type.	`reg:squarederror`, `binary:logistic`, `multi:softmax`, etc.
`early_stopping_rounds`	Stops training if no improvement after N rounds.	Requires validation set; prevents overfitting and reduces training time.
`scale_pos_weight`	Balances positive vs negative class weights.	Useful for imbalanced datasets; often set to (#negative / #positive).
`eval_metric`	Metric used to evaluate model performance during training.	Examples: `rmse`, `logloss`, `auc`, `error`; should match the objective.

Instance Types & Performance

XGBoost is CPU-bound (memory intensive). Use M5 or C5 instances. However, SageMaker XGBoost does support GPU training (e.g., P2/P3) for large datasets by setting the `tree_method` to `gpu_hist`.

XGBoost **can run on multiple instances**. It uses a parameter server approach to aggregate tree structures.

▼ LightGBM in SageMaker.

LightGBM is a gradient boosting framework. SageMaker provides managed training infrastructure, distributed training support and easy deployment.

SageMaker LightGBM data formats: Text/CSV & Parquet.

Hyperparameters

Hyperparameter	Description	Typical Usage / Notes
`learning_rate`	Step size shrinkage used in each boosting step.	Smaller values (0.01–0.1) improve generalization but require more iterations (`num_iterations`).
`num_leaves`	Maximum number of leaves per tree.	Higher values increase model complexity. Common range: 20–150. Should be tuned with `max_depth`.
`feature_fraction`	Fraction of features randomly selected for each tree.	Typical: 0.6–1.0. Helps reduce overfitting and speeds up training.
`bagging_fraction`	Fraction of data sampled for each iteration (row sampling).	Typical: 0.5–1.0. Lower values help prevent overfitting. Requires `bagging_freq` > 0.
`bagging_freq`	Frequency (in iterations) of performing bagging.	Set > 0 to enable bagging. Common values: 1–10.
`max_depth`	Maximum depth of each tree.	Limits tree growth. Typical: 3–10. Use with `num_leaves` to control complexity.
`min_data_in_leaf`	Minimum number of samples required in a leaf node.	Larger values reduce overfitting. Typical: 20–100+.

Instance Types & Performance

We can do single or multi-instance CPU training (define instances with `instance_count`).

LightGBM is **memory-bound** algorithm (choose general purpose over compute-optimized). Chose M5, not C5.

Seq2Seq in SageMaker.

Seq2Seq model is type where input is a sequence of tokens, output is a sequence of tokens. It is used for machine translation, text summarization, speech to text, etc...

SageMaker Seq2Seq requires RecordIO-Protobuf as input (**tokens must be integers**). We start with tokenized text files, then convert it to protobuf. Also we must provide training data, validation data and vocabulary files (JSON).

Training for machine translation can take days. We should use pre-trained models if we can.

Hyperparameters

Hyperparameter	Description	Typical Usage / Notes
`batch_size`	Number of training samples processed in one forward/backward pass.	Common: 16–128. Larger values speed up training but require more memory; smaller values improve generalization.
`optimizer_type`	Optimization algorithm used to update model weights.	`adam` (most common), `sgd`, `rmsprop`.
`learning_rate`	Step size for updating weights during training.	/
`num_layers_encoder`	Number of layers in the encoder network.	Typical: 1–6. More layers increase representation power but risk overfitting and slower training.
`num_layers_decoder`	Number of layers in the decoder network.	Usually similar to encoder. Deeper decoders improve generation but increase latency.
`accuracy`	Measures percentage of correct predictions.	Less informative for sequence generation; useful mainly for token-level evaluation.
`BLEU_score`	Metric comparing generated sequences with reference translations.	Standard for translation tasks. Higher is better; sensitive to n-gram overlap and phrasing.
`perplexity`	Measures how well the model predicts a sequence (based on cross-entropy).	Lower is better. Common for language models; correlates with fluency but not always with semantic quality.

Instance Types & Performance

It can only use GPU instance types (P3 for example). It is **limited to use only single machine** for training (we can use multi-GPU's on one machine).

DeepAR in SageMaker.

DeepAR is an Amazon-developed algorithm used for forecasting one-dimensional time series data. It uses RNNs and allows us to train the same model over several related time series.

DeepAR data formats: JSON Lines or Parquet. Each record must contain start timestamp and target (time series values).

Each record can contain:

- `dynamic_feat` (dynamic features like promotion applied or product pruchases).
- `cat` (categorical features).

We always must include entire time series for training, testing and inference. Also entire dataset as test set (remove last time points for training).

> Dont use very large values for prediction length.

Instance Types & Performance

We can use CPU or GPU instances (single or multi machine). Always start with CPU and move to GPU if necessary (only helps with larger models or with large mini batch sizes).

For inference we can use only CPU instances.

BlazingText in SageMaker.

BlazingText combines two different NLP tasks into a single built-in algorithm.

- **Word2Vec (Unsupervised):** Generates embeddings of words. It maps words to a mathematical space where words with similar meanings are close together, which is often used as a preprocessing step for downstream NLP tasks.
- **Text Classification (Supervised):** A optimized classifier used to assign categories to sentences or documents.

> It is **used for sentences, not for entire documents**!

Text Classification Input: a text file where each line is a single training sentence. Each line must begin with a label prefixed exactly by `__label__`. It also supports the Augmented Manifest format (JSON Lines), which is required if we want to stream the text classification data using Pipe mode.

Word2Vec Input: a simple text file with space-separated tokens. Each line contains a single sentence. No labels are required.

Word2vec Modes

1. **Skip-Gram** - predict context words from a target word. It is better for rare words and it captures fine-grained semantics.
2. **Continuous Bag of Words (CBOW)** - predict target word from context words. It works well for frequent words.
3. **Batch Skip-Gram** - an optimized variant of the standard Skip-Gram model designed for high-speed, large-scale training. Instead of processing one (target, context) pair at a time, it processes many pairs together in batches using matrix operations.

   This is the only mode that supports distributed training across multiple CPU instances, but it cannot utilize GPUs.

Hyperparameters

Hyperparameter W2v	Description	Typical Usage / Notes
`mode`	Training architecture used for learning word embeddings.	Options: `cbow` , `skipgram` , `batch_skipgram` .
`learning_rate`	Step size for updating model weights.	/
`window_size`	Context window size around a target word.	Larger values capture broader context but may add noise. Typical: 5–10.
`vector_dim`	Dimensionality of word embeddings.	Higher values capture more semantic detail but increase computation. Typical: 100–300.
`negative_samples`	Number of negative samples used in training.	Improves efficiency of training. Typical: 5–20.

Hyperparameter TC	Description	Typical Usage / Notes
`epochs`	Number of passes over the training dataset.	/
`learning_rate`	Step size for model updates.	/
`word_ngrams`	Maximum length of word n-grams used as features.	Higher values capture phrases but increase complexity. Typical: 1–3.

Hyperparameter TC	Description	Typical Usage / Notes
`vector_dim`	Size of feature or embedding vectors.	Higher values improve representation but increase memory/computation. Typical: 50–300.

Instance Types & Performance

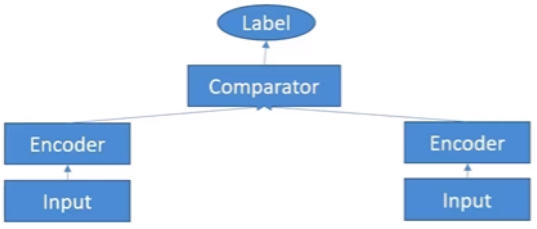
For cbow and skipgram it is recommended a single P3 instance.

Object2Vec in SageMaker.

Object2Vec creates a low-dimensional densce embeddings of high-dimensional objects (generalized Word2Vec).

The primary input format for training is **JSON Lines**. Input data must be tokenized into integers. Training data consists of pairs of tokens and/or sequences of tokens.

Object2Vec trains with two input channels, two encoders, and a comparator. Comparator is followed by a feed-forward NN.



Object2Vec Encoder Types

- **Pooled Embedding** - the fastest to train, but offers the lowest accuracy for complex sequences. Best suited for simple, discrete categorical tokens.
- **Hierarchical CNN** - faster training than RNNs due to parallelization, while still capturing local, short-distance context. A great middle-ground for sequence data.
- **BiLSTM** - the slowest to train, but yields the highest accuracy for sequential inputs (like sentences or paragraphs).

Instance Types & Performance

We can **only train on a single machine** (CPU or GPU, multi GPU). It is recommended to do inference on P3 instance with `INFERENCE_PREFERRED_MODE` env variable to optimize for encoder embeddings rather than classification or regression.

Object Detection in SageMaker.

SageMaker's Object Detection has two available variants:

- **MXNet Variant** - uses CNN with SSD algorithm. The base CNN can be VGG-16 or ResNet-50. We also can use transfer learning mode. It has built-in data augmentation.
- **Tensorflow Variant** - pulls models directly from the TensorFlow Model Garden.

MXNet expects RecordIO or image format (jpg/png). With image format we must supply a JSON file for anotation data for each image.

Instance Types & Performance

It is must to use GPU instances for training (multi-GPU and multi-machine) like P2, P3, G4, G5. For inference it is okay CPU or GPU (M5, P2, P3).

Image Classification in SageMaker.

SageMaker's Image Classification has two available variants:

- **MXNet Variant** - we can do full training mode or use transfer learning mode. Default image size is 3-channel 224×224.
- **Tensorflow Variant** - pulls models directly from the TensorFlow Model Garden.

MXNet expects RecordIO or image format (jpg/png). If we use image format, we must provide list files for both the training and validation channels.

Instance Types & Performance

It is must to use GPU instances for training (multi-GPU and multi-machine) like P2, P3, G4, G5. For inference it is okay CPU or GPU (M5, P2, P3).

Semantic Segmentation in SageMaker.

Semantic Segmentation provides **pixel-level classification**. It assigns a class label to every single pixel in the image.

It is built on MXNet Gluon and Gluon CV. We can chose between 3 algorithms: FCN, PSP or DeepLabV3. For all three of these algorithms, we can choose **ResNet** as the backbone network.

It expects jpg images and png annotations for both training and validation. Annotation images must be provided as single-channel png images (segmentation masks).

Instance Types & Performance

It is must to use GPU instances for training (multi-GPU and multi-machine) like P2, P3, G4, G5. For inference it is okay CPU or GPU (C5, M5, P2, P3).

Random Cut Forest in SageMaker.

RCF is unsupervised learning algorithm. It creates a forest of trees and randomly cuts the data dimensions to isolate data points. **Anomalies are easier to isolate**. If a data point can be isolated with very few cuts, it is highly likely to be an anomaly.

It accepts **RecordIO-protobuf and CSV** as input formats. There are optional test channel for computing accuracy, precision, recall and F1 on labeled data.

Hyperparameters

Hyperparameter	Description	Typical Usage / Notes
`num_trees`	Number of trees in forest.	Increasing number of trees reduces noise.
`num_samples_per_tree`	Number of samples that each tree should have.	Should be chosen such that 1/num approximates the ratio of anomalous to normal data.

Instance Types & Performance

It doesnt take advandate of GPUs. It is recommended to use M4, C4 or C5 for training.

Neural Topic Model in SageMaker.

NTM is **unsupervised** algorithm that organizes documents into topics. It is used to classify or summarize documents based on topics.

We must preprocess text manually before passing it to SageMaker (**tokenize it into integers**). Every document must contain a count for every word in the vocabulary in CSV.

The algorithm accepts RecordIO-protobuf or CSV for inputs.

Instance Types & Performance

We can train it on GPU (recommended) or CPU (cheaper).

Latent Dirichlet Allocation in SageMaker.

LDA is **unsupervised** algorithm (not deep learning) that organizes documents into topics. It is used to classify or summarize documents based on topics. It uses a **probabilistic** graphical model to find topics.

We must preprocess text manually before passing it to SageMaker (**tokenize it into integers**). Every document must contain a count for every word in the vocabulary in CSV.

The algorithm accepts RecordIO-protobuf or CSV for inputs.

Instance Types & Performance

It support single instance CPU training.

KNN in SageMaker.

KNN can be used for both **classification** (taking a majority vote of neighbors) and **regression** (averaging the values of neighbors).

SageMaker KNN introduces a training step that builds an **index** of the data. It uses dimensionality reduction to group similar data points together in memory. When a prediction is requested, it only searches within the relevant index grouping.

It accepts RecordIO-protobuf and CSV for input. We must normalize or standardize data before training.

Instance Types & Performance

Training can be done on CPU or GPU (M5, P2). CPU is recommended for inference if our goal is **low latency**, and GPU if goal is higher throughput on large batches.

K-Means Clsutering in SageMaker.

K-Means takes unlabeled data and groups it into k distinct clusters based on feature similarity. SageMaker uses a modified version called **Web-Scale K-Means**. It processes the data in mini-batches, updating the cluster centroids incrementally, which allows it to scale.

It accepts RecordIO-protobuf and CSV for input. We must normalize or standardize data before training.

S3 Data Distribution Types

- **ShardedByS3Key** - dataset is sharded equally among the instances in cluster based on the S3 object keys. Allows to train on massive datasets that are larger than the storage capacity of a single instance. Speeds up training through parallelism.
- **FullyReplicated** - a complete copy of the entire dataset is downloaded to every single training instance in cluster. Every instance sees the whole picture.

Instance Types & Performance

It can be trained on both CPU or GPU, but CPU is recommended. If we use GPU, only one GPU per instance is allowed (P2, P3, G4).

PCA in SageMaker.

PCA takes high-dimensional data and projects it into a lower-dimensional space. SageMaker PCA offers two distinct approaches under the hood, controlled by the `algorithm_mode` hyperparameter:

- **Regular Mode** - calculates the full covariance matrix and its eigenvectors. It is highly accurate but computationally expensive. **Used for datasets with a manageable number of features and rows.**
- **Randomized Mode** - uses an approximation technique. **Use this for massive dataset.**

It accepts RecordIO-protobuf and CSV for input. We must normalize or standardize data before training.

Instance Types & Performance

It can be trained on both CPU or GPU (single GPU per instance).

Factorization Machines in SageMaker.

Factorization Machine is supervised algorithm designed for predicting outcomes when we have sparse, high-dimensional data. It captures **interactions between features** even when there is little to no data for specific pairs.

It accepts only RecordIO-protobuf as input format.

Instance Types & Performance

It can be trained on both CPU or GPU. CPU is recommended, while GPU only works with dense data.

IP Insights in SageMaker.

IP Insights is unsupervised learning of IP address usage patterns. It identifies suspicious behavior from IP addresses. It is explicitly built to catch **fraudulent logins and account takeovers**.

It accepts CSV with two columns only (`entity_id` and `ip_address`), with no header row.

Instance Types & Performance

It can be trained on both CPU or GPU. GPU is recommended, and we can use multiple GPUs.

Cheat Sheets.

Algorithm	Type	Primary Use Case	Required Input Format(s)	Compute (Train)
Linear Learner	Supervised	Reg/Class (Tabular)	RecordIO-pb, CSV	CPU or GPU
XGBoost	Supervised	Reg/Class (Tabular)	CSV *(no header)*, RecordIO-pb, Parquet, LibSVM	CPU or GPU
LightGBM	Supervised	Reg/Class (Tabular)	CSV, Parquet	**CPU** *(Memory Bound M5)*
KNN	Supervised	Reg/Class (Tabular)	RecordIO-pb, CSV	CPU or GPU
Factorization Machines	Supervised	Recommendations, **Sparse Data**	**RecordIO-pb (Float32)**	CPU *(Recommended)* or GPU
DeepAR	Supervised	Time Series Forecasting	JSON Lines, Parquet	CPU *(Start here)* or GPU
BlazingText	Supervised Unsup.	Text Classification Word2Vec (Embeddings)	TXT (with __label__) TXT (Space-separated)	CPU or GPU
Seq2Seq	Supervised	Translation, Summarization	**RecordIO-pb (Integer Tokens)**	**GPU Only** *(Single Machine)*
Object2Vec	Sup/Unsup	Embeddings (Gen. W2V)	JSON Lines (Integer Tokens)	CPU or GPU *(Single Machine)*
Image Classification	Supervised	Image Labeling	RecordIO, Image (JPG/PNG) + LST	**GPU Only**

Algorithm	Type	Primary Use Case	Required Input Format(s)	Compute (Train)
Object Detection	Supervised	Bounding Boxes	RecordIO, Image + JSON	**GPU Only**
Semantic Segmentation	Supervised	Pixel-level Classification	JPG Images + **PNG Masks**	**GPU Only**
K-Means	Unsupervised	Clustering	RecordIO-pb, CSV	CPU or GPU
PCA	Unsupervised	Dimensionality Reduction	RecordIO-pb, CSV	CPU or GPU *(1 GPU per inst)*
Random Cut Forest (RCF)	Unsupervised	**Anomaly Detection**	RecordIO-pb, CSV	**CPU Only**
IP Insights	Unsupervised	Fraud, Suspicious Logins	CSV *(entity_id, ip_address)*	CPU or GPU
LDA	Unsupervised	Topic Modeling	RecordIO-pb, CSV	**CPU Only** *(Single Inst)*
NTM	Unsupervised	Topic Modeling	RecordIO-pb, CSV	CPU or GPU

Model Training, Tuning & Evaluation

▼ **Automatic Model Tuning (AMT) in SageMaker.**

AMT runs multiple training jobs on dataset using the algorithm and ranges of hyperparameters we specify. It uses an **Objective Metric** to evaluate which model is best (it learns as it goes, so it doesnt try every combination).

Warm Start - use one or more previous tuning jobs as a starting point.

There are default limits for number of parallel tuning jobs, number of hyperparameters and number of training jobs per tuning job. To increase these we need to contact AWS support.

Hyperparameter Tuning Approaches

Grid Search - tries every possible combination in our specified range. Only use when have a very small number of categorical/integer hyperparameters.

Random Search - chooses random combinations of hyperparameters. Use this if want to run a massive number of concurrent jobs to save time.

Bayesian Optimization - it treats tuning like a regression problem. It uses results from previous training jobs to guess which hyperparameters will yield the best results next. It balances exploration and exploitation.

Hyperband - a multi-fidelity strategy meant for iterative algorithms. It dynamically allocates resources to promising training jobs and kills poor-performing ones early.

Early stopping only works with Bayesian and Random search. The algorithm **must emit metrics per epoch/iteration to CloudWatch** for early stopping to work.

AMT Best Practices

- Dont optimize too many hyperparameters at once.
- Limit ranges to as small a range as possible.
- Use logarithmic scales when appropriate.
- Dont run too many training jobs conrurrently (limits how well the process can learn as it goes).

SageMaker Autopilot/AutoML.

SageMaker Autopilot automates the machine learning process. We give it a **tabular dataset** (CSV or Parquet) in S3, tell it which column is the target, and it automatically cleans the data, feature engineering, selects algorithms, tunes hyperparameters, and deploys the model.

Autopilot is **fully transparent**. When it runs, it generates two jupyter notebooks.

- **Data Exploration Notebook**: Shows data quality, missing values, and statistics.
- **Candidate Generation Notebook**: Contains the exact python code used to preprocess the data and train the models.

Autopilot Training Modes

- **HPO (Hyperparameter Optimization)** - selects algorithms most relevant to our dataset and best range of hyperparameters to tune our models.
  - If dataset < 100MB, it uses Bayesian optimization.
  - If dataset > 100MB, it uses Multi-fidelity optimization.
- **Ensembling** - trains several base models using AutoGluon library (wider range of models). Models are combined with a stacking ensemble method.
- **Auto** - if the dataset is small, it automatically picks Ensembling. Otherwise it picks HPO. Good for dataset of unknown size.

[Maintenance] SageMaker Debugger.

SageMaker Debugger looks inside that black box during training by capturing tensors (weights, biases, gradients, loss) and system metrics (CPU/GPU usage) in real-time.

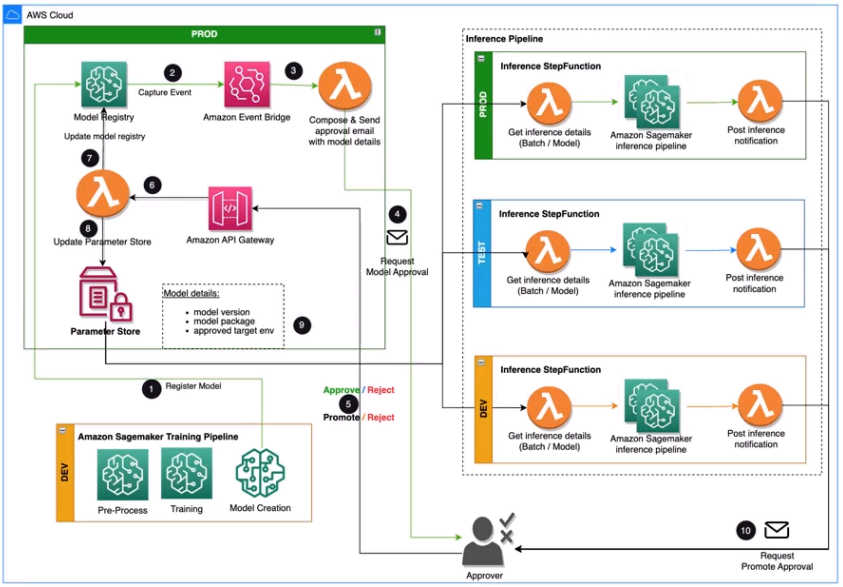
It **logs and fires a CloudWatch event** when the rule is hit. We can set a rule so that if Debugger detects something, EventBridge triggers a Lambda function to stop the training job immediately to save money, or triggers an SNS topic to email the data scientist.

It also comes with built-in rules that automatically monitor for common, mathematical training errors.

Debugger can also track hardware utilization (CPUBottleneck, GPUMemoryIncrease).

SageMaker Model Registry.

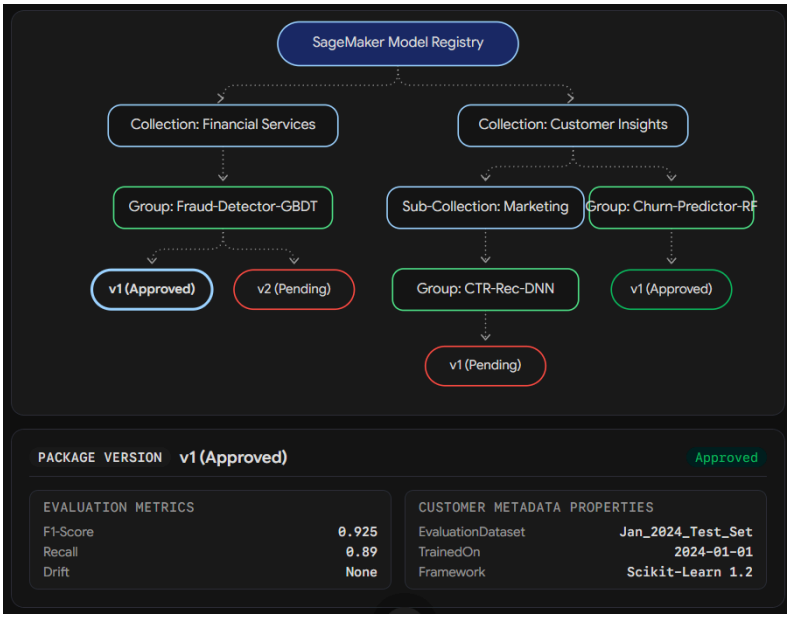
Model Registry is used to catalog our models and manage model versions. We can associate metadata with models, manage approval status of a model, deploy models, automate with CI/CD, etc...



Model Registry Collection - a top-level folder or category used to group related **Model Groups** into a hierarchy. Its purpose is to **improve model discoverability at scale** when an organization has hundreds of models across different teams, domains, or business units.

Model Groups - a container that holds all versioned iterations for one specific ML use-case. We can use Resource Tags here.

Model Package Versions - the numbered iterations inside a Model Group. They are immutable and hold the actual model metadata: Approval Status (Pending, Approved), ModelMetrics, S3 Artifact Paths, and CustomerMetadataProperties.



SageMaker Training at Large Scale.

SageMaker Training Compiler - a feature that automatically optimizes DL models (specifically PyTorch and TensorFlow) to run faster on GPU instances. It compiles model code into a more hardware-efficient format.

We need to use `compiler_config=TrainingCompilerConfig` in our estimator class.

SageMaker Managed Warm Pools - keep the instances alive for a specified period after a job finishes. It is useful if we repeatedly training a model (iterative training or tuning).

We need to set a `KeepAlivePeriodInSeconds`. We are billed for the time the instances stay warm. Usage of a persistent cache to store data across training jobs can help in cost reduction.

SageMaker Checkpointing & Cluster Health Checks.

Checkpointing - the process of saving the state of model (weights, optimizer state) to S3 periodically during training.

To use it, we need to define `checkpoint_s3_uri` and `checkpoint_local_path` in our SageMaker estimator.

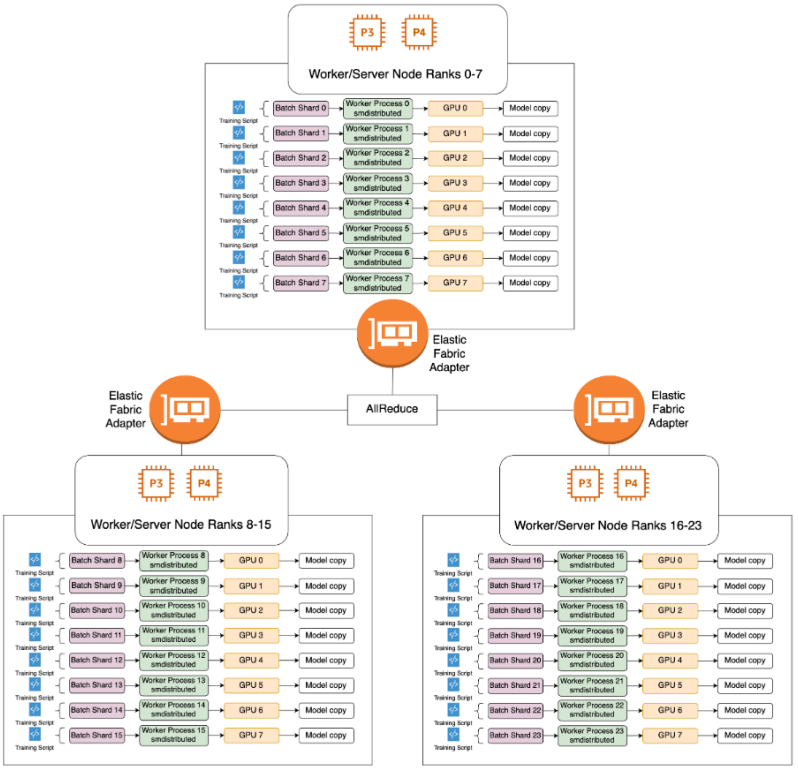
SageMaker performs health checks on the instances and the network during the training job. It prevents a zombie training job. If one node in a massive cluster fails, the whole training job usually hangs.

Automatic Restarts - if a training job fails due to a hardware issue or a network error, SageMaker can automatically restart the job.

SageMaker Distributed Training.

SageMaker Distributed Data Parallel - the model is replicated on every GPU in the cluster. The dataset is sharded. Each GPU gets a different chunk of data. After each pass, the GPUs communicate to average their gradients

Use when our model fits on one GPU, but we have a massive dataset and want to finish training faster.



SageMaker's Distributed Training Libraries

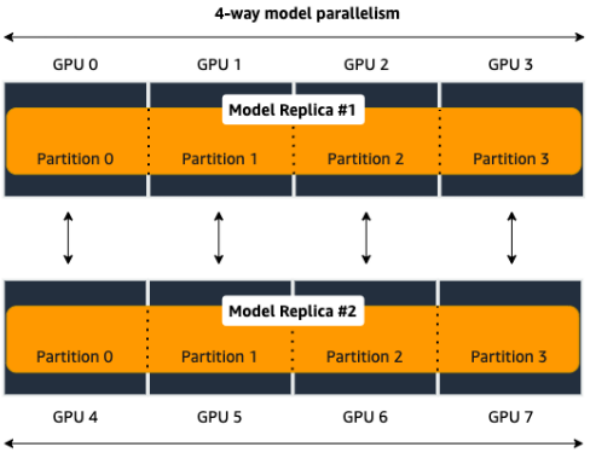
SDTLs solve similar problem as MapReduce/Spark but for distributing computation of gradients in gradient descent.

- **AllReduce** - distributes computation of gradient updates to/from GPUs. It is implemented in SDDP library. We need to specify a backend of **smdpp** to `torch.distributed.init_process_group` in our training scripts.

- **AllGather** - manages communication between nodes to improve performance. Offloads communications overhead to the CPU.

▼ SageMaker Model Parallelism Library.

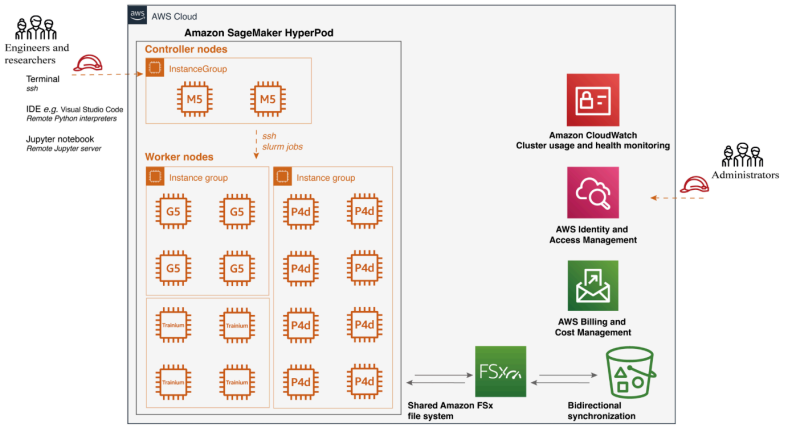
SageMaker Model Parallelism - splits the model itself across multiple GPUs. SMP can automatically decide how to partition model across GPUs so we don't have to do the math.



Use when our model is so huge (billions of parameters) that we get **CUDA Out of Memory Error**.

▼ SageMaker HyperPod.

SageMaker HyperPod - fully managed cluster infrastructure designed specifically for training massive FMs over weeks or months. Standard SageMaker training jobs fail entirely if a single EC2 node or GPU crashes.

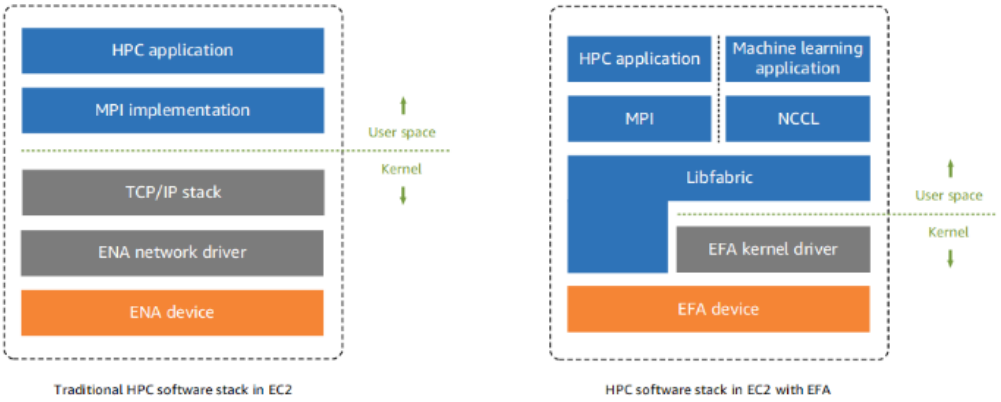


HyperPod **provides resilient training**. If hardware fails during a massive multi-node training job, HyperPod automatically detects the failure, replaces the faulty instance.

▼ Elastic Fabric Adapter & MiCS.

Elastic Fabric Adapter (EFA) - a network interface designed for tightly coupled HPC and ML workloads that require inter-instance communication with low latency and high throughput. **EFA builds on ENA**, offering additional features like hardware-based message passing.

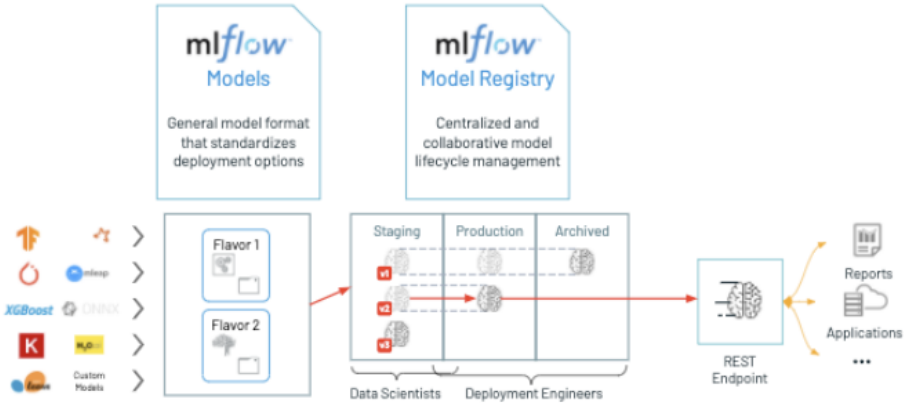
The OS-bypass capabilities of EFAs are not supported on Windows instances. If we attach an EFA to a Windows instance, the instance functions as an ENA without the added EFA capabilities.



Minimize the Communication Scale (MiCS) - same as SageMaker sharded data parallelism provides, but for training models with more than trillion parameters.

MLFlow.

MLFlow is open-source platform for ML/GenAI workflows. It offers observability, evaluations, tracking, tracing, AI gateway. It is integrated with SageMaker AI. It also can manage our models and their deployment.



MLflow Tracking - logs parameters, metrics, code version and tags for each run.

MLflow Artifacts - logs heavy files produced during a run (model weights, requirements.txt, ROC curve, SHAP plots).

MLflow Models - packages a model in a standard directory format containing an MLmodel YAML metadata file that defines model flavors.

MLflow Model Registry - centaral catalog inside MLflow for versioning models, assigning **aliases** or stages, and adding descriptions.

SageMaker Managed MLflow Server - fully managed server hosting the MLflow UI, REST API, and backend. It **can be stopped during off-peak hours** to save compute costs without losing logged runs or S3 artifacts.

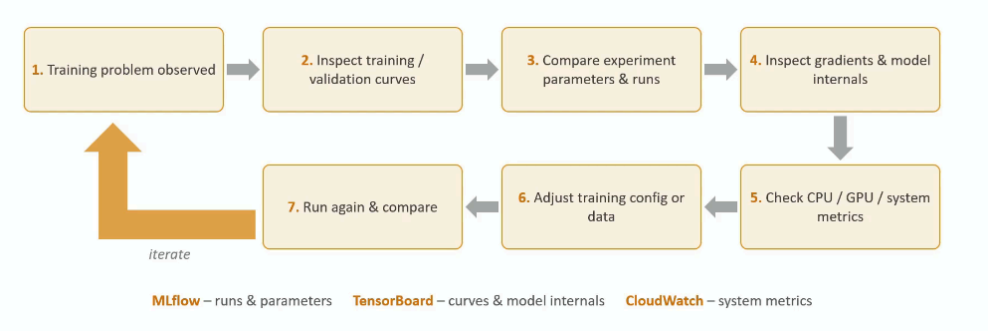
Traditional NLP Metrics.

BLEU - used for machine translation. It focuses on precision (did the words the machine generated appear in the human reference). Requires human references.

ROUGE - used for text summarization. It focuses on recall (did the machine capture all the important words that were in the human reference). Requires human references.

Perplexity - used to measure the general fluency of a Language Model. It calculates how surprised a model is by a sequence of words. Lower is better. Does **not measure factual accuracy or tone**.

▼ **Debugging Model Convergence & Training Performance.**



▼ **ML Troubleshooting Cheat Sheet**

- **Severe Overfitting (High Training Accuracy, Low Validation Accuracy)**
  - **Fixes: Simplify the model** (reduce depth/complexity), add **Dropout**, add **L1/L2 Regularization**, use **Early Stopping**, gather more training data, or increase data augmentation.
- **Underfitting (Low Training Accuracy, Low Validation Accuracy)**
  - **Fixes:** Increase model complexity (add layers/nodes, increase tree depth), decrease regularization, add more features.
- **Slow Convergence / Unstable Training / Gradients Exploding/Vanishing**
  - **Fixes:** Add **Batch Normalization**, implement Gradient Clipping (for exploding gradients), change the activation function.

Generative AI on AWS

▼ **SageMaker JumpStart.**

SageMaker JumpStart helps users quickly deploy and experiment with ML models. Especially generative AI, without needing to build everything from scratch.

We can access to thousands of models from providers like Meta (Llama), Hugging Face, etc... It includes Foundation Models (LLMs), image generation, and vision models.

While SageMaker has built-in algorithms, JumpStart provides a interface to launch them easily with pre-configured settings.

When we fine-tune or deploy a JumpStart model, **our data does not leave VPC.** It is not used to train the base model for other AWS customers.

▼ **AWS Bedrock.**

Bedrock is a fully managed, serverless service that provides access to Foundation Models from third-party providers via a single API.

The Bedrock API Endpoints

- **Control Plane (bedrock)** - used for management tasks. We use this to list available models, manage provisioned throughput, create guardrails, and create custom models.
- **Data Plane (bedrock-runtime)** - used for inference. We use this to send prompts and get responses.

- **Agent Plane (bedrock-agent & bedrock-agent-runtime)** - used specifically to create and invoke Knowledge Bases and Agents.

Security & Authentication

We must use it with IAM user (not root), and user must have relevant Bedrock permissions (AmazonBedrockFullAccess or AmazonBedrockReadOnly).

Before using any base model in Bedrock, we must first request access.

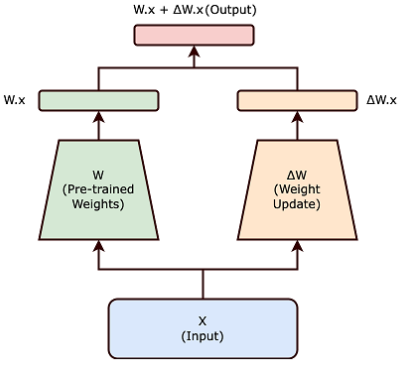
Authentication Methods:

- **IAM Roles / Temporary Credentials** (for production) - uses normal AWS authorization via IAM. Supports the principle of least privilege.
- **Bedrock API Keys** (for dev/exploration) - provides a simpler, bearer-token-style way to call Bedrock APIs.

▼ **Fine-Tuning Foundation Model.**

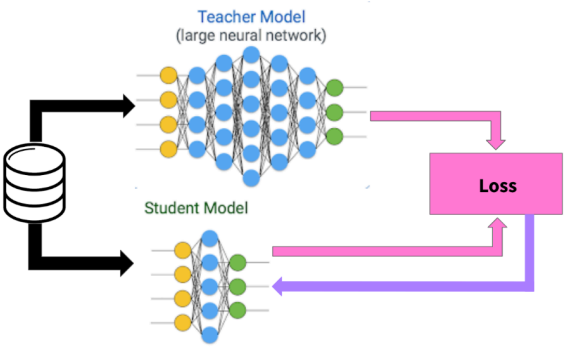
Low-Rank Adaptation (LoRA) - **efficient technique used to fine-tune LLMs** to specific tasks or domain data. Full fine-tuning is expensive and takes a long time. It also can reduce catastrophic forgetting.

It freezes all the original weights of the pre-trained LLM. Then it injects small, trainable low-rank matrices alongside the frozen base model. During the fine-tuning, only the weights in these new matrices are updated using custom training data.



> **Catastrophic Forgetting** - when a model forgets its original, general knowledge after being fully fine-tuned on new, specific data.

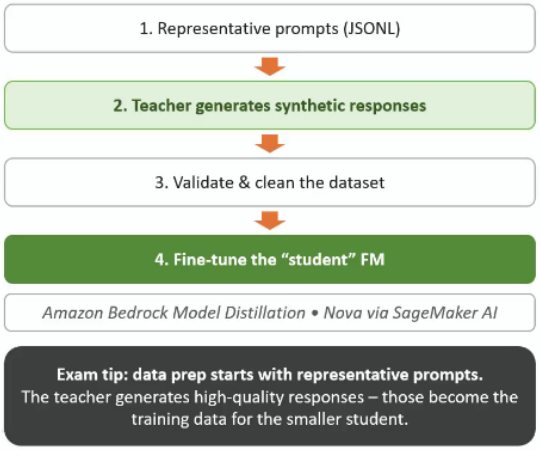
Model Distillation - a compression technique that transfers knowledge from a large, heavy model (the teacher) to a smaller, lighter model (the student). The student model is trained to mimic the exact outputs and behavior of the teacher model.



It creates a much smaller model that is faster and cheaper to run. Also it retains much of the high performance and reasoning capabilities of the teacher model.

Data Preparation for Model Distillation:

1. Pass input data through the teacher model. Instead of just keeping the final prediction, collect the soft labels (the probability scores for all classes).
2. Final training dataset becomes the [Original Input Data] paired with the [Teacher's Soft Labels].



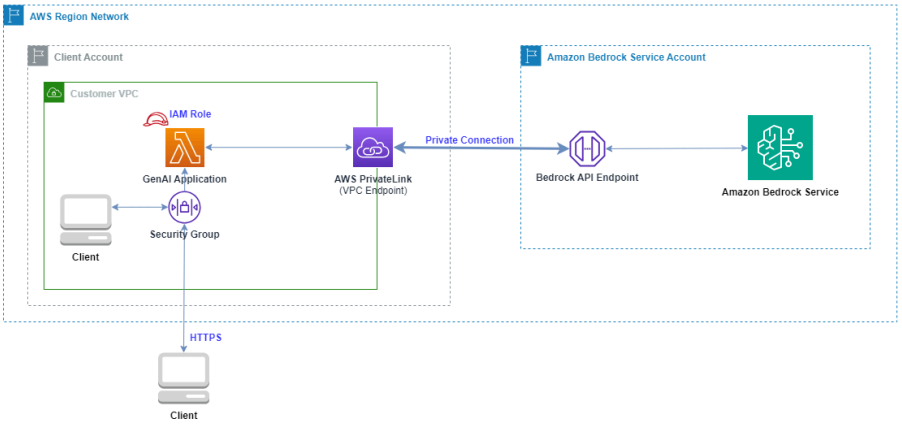
Bedrock Fine-Tuning Custom Models.

Instruction-Based Fine-Tuning - we provide labeled examples of prompts and responses to teach the model a specific style, tone, or format.

```
{"prompt": "Explain the company's sick leave policy.", "completion": "Employees are entitled to 10 days of paid sick leave annually. A medical certificate is required for absences longer than 2 days."}
```

Continued Pre-Training (CPT) - we provide large volumes of raw, unlabeled text to teach the model domain-specific vocabulary that it didn't know before.

Our training dataset must reside in a S3 bucket. The data must be in JSONL format. If we have sensitive data we **should use VPC and PrivateLink**.



RAG with Bedrock.

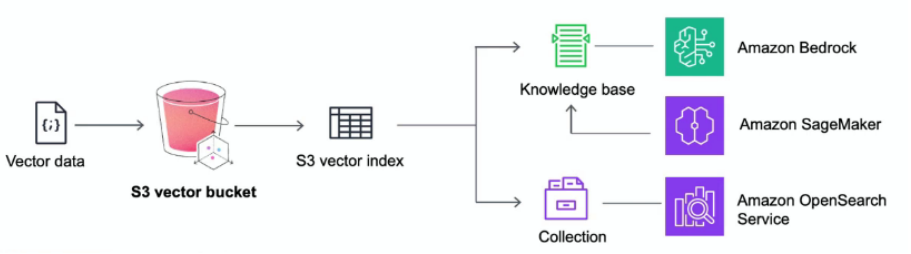
AWS does not have a single standalone vector database. Instead, they added vector capabilities to their existing databases.

Vector Stores

OpenSearch Serverless - best for standalone GenAI applications, massive scale, and serverless architectures.

- **Semantic (Vector) Search** - searching by meaning and intent, rather than exact text matches. It uses vector embeddings and is best for natural language queries.
- **Hybrid Search** - combines semantic search with keyword search.

S3 Vectors - a built-in feature of S3 that lets us store and query vector embeddings directly inside S3. It is 90% cheaper. S3 Vectors are **strongly consistent, but have higher latency**. It is optimized for price, not high-speed performance, so they are best for infrequently queried data.



Aurora PostgreSQL (pgvector) - best for customers who already use relational databases and want ACID compliance alongside vector search.

Amazon MemoryDB for Redis - for ultra-fast, real-time GenAI applications.

Neptune Analytics - best for combining relationships with semantics.

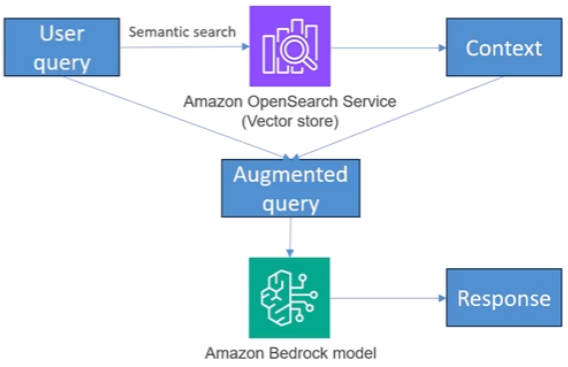
Knowledge Bases

We can upload our documents or structured data (via S3, Confluence, Salesforce, etc) into **Bedrock Knowledge Bases**. Then we must use an embedding model (Cohere or Amazon Titan). After that we need to configure vector store.

We can attach metadata to our documents before vectorizing them. Place `.metadata.json` in S3 right next to source document. It can optimize accuracy and reduce hallucinations. Also, narrows the search space in the vector database, making the retrieval process more efficient.

Using Knowledge Bases

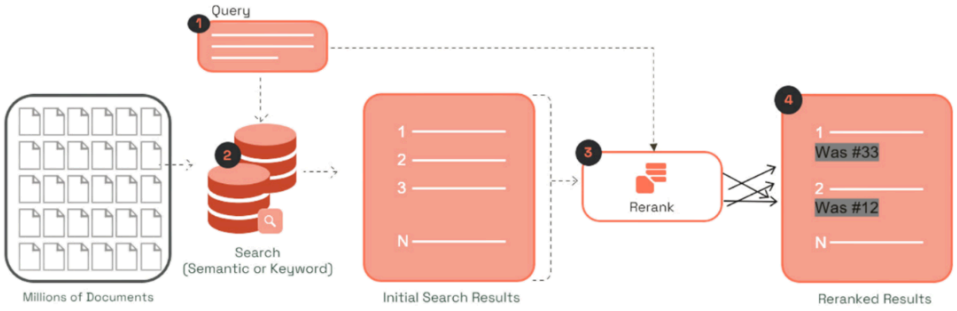
We can chat with our documents (automatic RAG), integrate it into an application directly or incorporate it into an agent.



Re-ranker Models in Bedrock.

Standard vector search is very fast but can sometimes retrieve documents that share similar words or topics but don't actually answer the user's specific question.

Cohere Rerank - a specialized model used in a RAG pipeline to re-evaluate and re-order search results before sending them to the main LLM.



Prompt Management in Bedrock.

Bedrock Prompt Management is a centralized tool within Bedrock to create, test, version, and manage prompts. It allows us to use variables to make prompts dynamic and reusable. We can do version control and API integration too.

Content Filtering with Bedrock Guardrails.

Bedrock Guardrails act as a protective layer between our application users and the foundation model. Guardrails evaluate both the input prompt and the response.

- **Content Filters** - filters out harmful, malicious, or toxic content.
- **Denied Topics** - allows us to define custom business domains that the bot is not allowed to talk about. You provide a natural language description of what to avoid.
- **Sensitive Information Filters** - we can choose either to block the interaction entirely or mask the data.
- **Word Filters** - blocks specific words.
- **Contextual Grounding** (Hallucination Mitigation for RAG) - checks if the model's answer is actually based on source documents or if the model hallucinated.

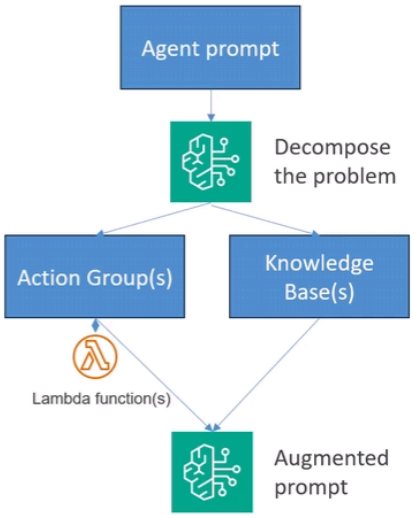
ApplyGuardrail API - a Bedrock API that evaluates text against a Guardrail without needing to invoke a Bedrock FM. We **can filter toxicity on a SageMaker Endpoint, redact PII for a custom hosted LLM or third-party models**, etc...

▼ LLM Agents with Bedrock Agents.

Bedrock Agent is an orchestration framework that allows the LLM to break down a complex user request into a multi-step actions to fulfill the request.

Components of a Bedrock Agent

1. **The Foundation Model**
2. **Instructions (Persona)** - system prompt that tells the agent its job, tone, and constraints.
3. **Action Groups** - how the agent talks to company's backend. It consists of:
   - **OpenAPI Schema** (JSON/YAML) - tells the agent what APIs exist, what parameters they need, and what they do.
   - **Lambda Function** - the actual code that executes the API call.
4. Knowledge Bases - we can link an agent directly to a Bedrock Knowledge Base (RAG).



Return of Control - instead of Bedrock calling the Lambda, Bedrock pauses, sends the formatted payload back to client application, and application makes the API call itself.

Deploying Bedrock Agents

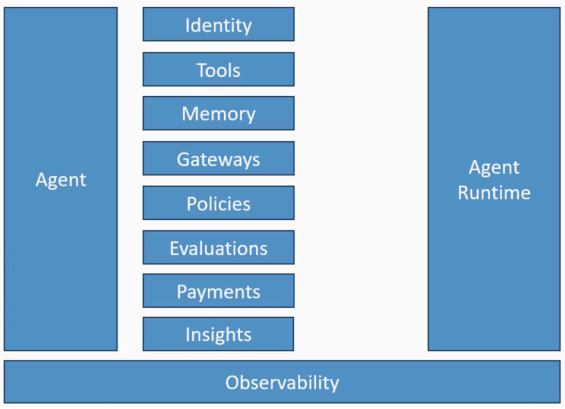
We need to create an alias for an agent (creates a deployed snapshot).

- **On-demand troughtput** - allows agent to run at quotas set at the account level.
- **Provision throughput** - allows us to purchase an increased rate and number of tokens for agent.

▼ Bedrock AgentCore.

AgentCore is a fully managed enterprise **platform** designed to build, deploy, connect, and operate AI agents securely at scale. AgentCore is **model-agnostic and framework-agnostic**. It works with open-source orchestration frameworks.

We can import existing Bedrock Agents and use it as starting point.



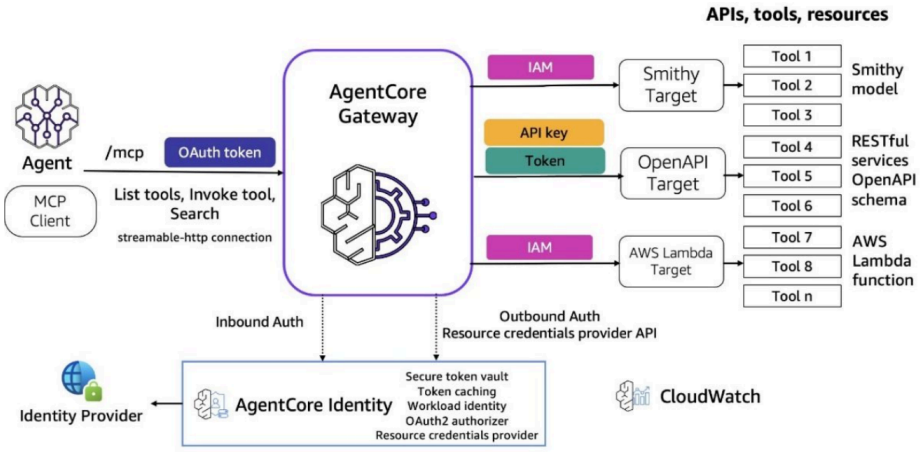
AgentCore Identity - specialized workload identity and credential management for AI agents, allowing them to authenticate and execute tasks securely

AgentCore Runtime - the secure, serverless hosting environment that handles scaling and session isolation for deployed agent code. We can deploy agent to ECR, enhanced with AgentCore capabilities or we can build our own Docker containers. We can have multiple endpoints.

AgentCore Memory - enables agents to retain context and remember user preferences across multiple sessions (short and long term memory).

AgentCore Gateway.

AgentCore Gateway is a fully managed AI gateway that provides a single, secure entry point for agentic traffic, connecting agents to tools, to other agents, and to LLMs.



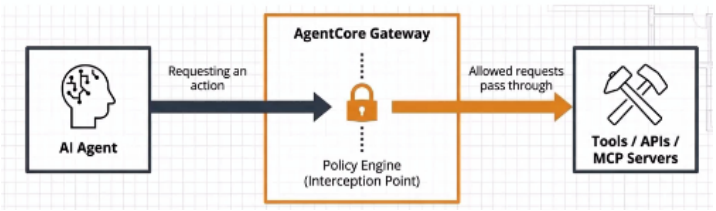
It coverts APIs, Lambda functions, or other services into MCP tools. Agents can then access them through Gateway endpoints.

Gateway manages OAuth authorization to ensure only valid users and agents can access tools and resources.

AgentCore Policies.

AgentCore Policy is the centralized security and compliance layer that governs exactly what an AI agent is allowed to do.

It integrates with AgentCore Gateways, and intercepts tool calls. Then it evaluates against our defined policies.



Developers can write rules in plain English, and then AgentCore automatically translates these natural language instructions into Cedar (authorization language).

All actions are denied unless it is explicitly permitted. If multiple policies apply to a single request and **even one contains a deny, the request is blocked**.

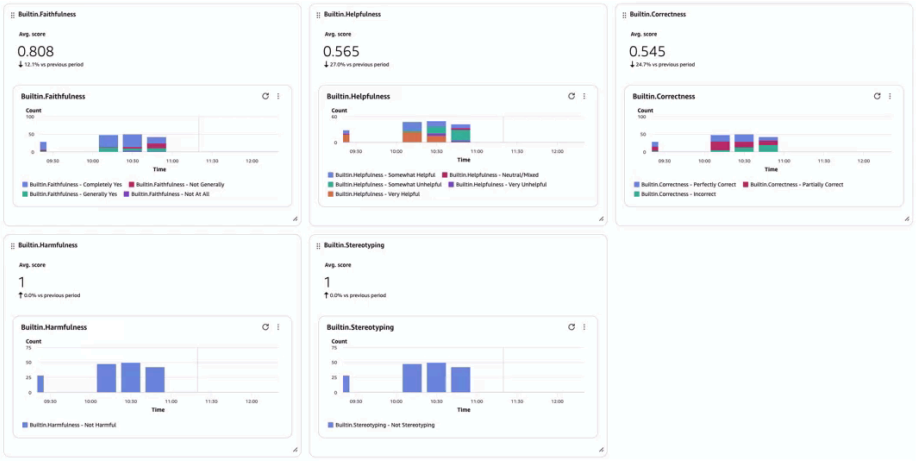
Traditional access controls are stateless (they only look at a single, isolated request). But AI workflows are multi-step and conversational. AgentCore solved it with Temporal Policies.

Temporal Policies - session-scoped rules that reason over the history of actions within a conversation.

> *"The agent is only allowed to call the 'BookFlight' tool if it successfully called the 'GetManagerApproval' tool earlier in this exact same session."*

▼ AgentCore Evaluators.

AgentCore Evaluations asses how well our agent performs tasks, handles edge cases, maintains consistency. We can visualize results in CloudWatch (AgentCore Observability Insights).



It **automatically selects optimal region for processing**. Data remains resident to where the request originated.

Can be integrated with LangGraph, OpenTelemetry, Strands, etc...

Evaluations

- **Correctness** (Builtin.Correctness) - measures if the Agent's response is factually accurate based on an expected reference answer (ground truth). If the answer includes incorrect facts, it fails Correctness.
- **Helpfulness** (Builtin.Helpfulness) - measures how useful the response is from the user's perspective.
- **Faithfulness** (Builtin.Faithfulness) - measures if the LLM's answer was **derived strictly from the Knowledge Base**. If it brings in outside knowledge, it fails Faithfulness.
- **Tool Accuracy** (Builtin.ToolSelectionAccuracy) - measures if the Agent picked the correct API/Lambda function for the user's request.

- **Coherence** (Builtin.Coherence) - measures if the response is free from logical gaps, inconsistencies, or contradictions. If the Agent contradicts itself within the same response, it fails Coherence.

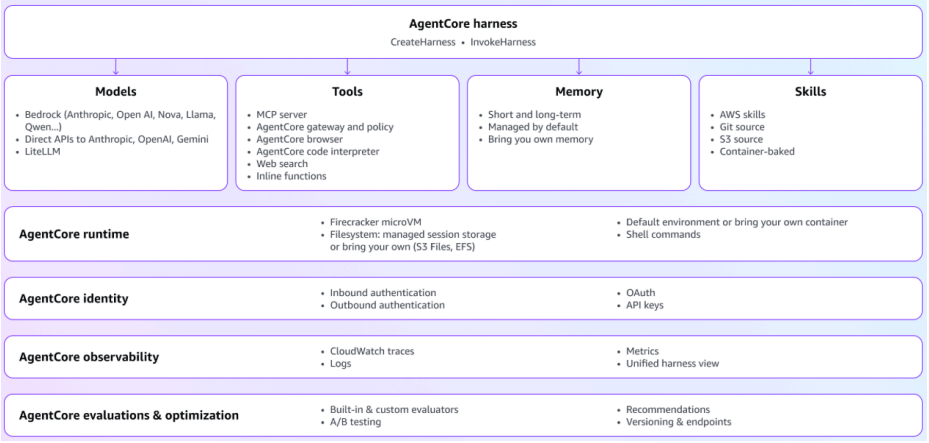
We can create Custom Evaluators. Must define a model to use as a judge and configure it, and define it with a prompt. Then we define scales and evaluation level.

- **Session Level** - evaluates the entire end-to-end conversation (all turns, prompts, responses, and tool calls).
- **Trace Level** - evaluates a specific assistant turn within the conversation. It looks at the current prompt, previous turns, and the current response.
- **Tool Level** - evaluates a single specific tool invocation. It looks at the prompt and the tool calls made. **Tool-level evaluators cannot catch missing tool calls**.

> To detect when an agent fails to call a tool, we must use a **Trace-level evaluator** configured with an LLM-as-a-judge to analyze the full agent turn and confirm that the tool was invoked before generating the response.

▼ AgentCore Harness.

AgentCore Harness is a fully managed agent orchestration layer. With the Harness, we write zero orchestration code. We define your agent purely as a configuration file (JSON/YAML) declaring the foundation model, the system prompt, and the tools.



AgentCore Runtime vs AgentCore Harness

AgentCore Runtime (Bring Your Own Code) - we write the agent logic using a framework. We package it in a container and Runtime provides serverless scaling and isolation.

AgentCore Harness (No Code) - runs inside the Runtime. We provide no code.

▼ AgentCore - Handling Failure Modes.

Coordination Failures - if a worker agent crashes or a network timeout occurs during another agent communication, the entire workflow can stall.

Solutions: Checkpoint state in AgentCore Memory, timeout and retry patterns, use supervisor agent.

Truncated Streaming - streamed responses cut off mid-generation.

Solutions: Check the stop reason, buffer events and resume from the last chunk, fall back to a non-streaming retry.

Tool Failures (AgentCore Gateway) - gateway calls out error or timeout error.

Solutions: Return structured errors the LLM can act on, retry with exponential backoff, circuit breakers.

Bedrock Studio.

Bedrock Studio allows developers to build, test, and share GenAI applications using a visual interface. It is heavily integrated with IAM Identity Center (SSO).

Bedrock Studio Workspaces - we can create a workspace for a specific project and invite team members.

Bedrock Studio vs SageMaker Studio

Feature	Amazon Bedrock Studio	Amazon SageMaker Studio
Target Persona	App Developers, Software Engineers.	Data Scientists, ML Engineers.
Primary Activity	Prompt engineering, building Agents, testing RAG.	Writing Python code, training models, tuning hyperparameters.
Interface	No-code / Low-code UI.	Jupyter Notebooks, IDEs.
Focus	**Using** pre-trained Generative AI.	**Building/Training** custom ML models from scratch.

Cost-Effective Model Selection.

If we are using RAG. The LLM doesn't need to know the answer from its training data. It just needs to read the injected text and summarize it. **Smaller models are highly cost-effective for RAG**.

We **should use small, cheap models to do the grunt work** before passing data to a larger model.

Bedrock Intelligent Prompt Routing - a feature that provides a single serverless endpoint that automatically predicts the complexity of a user's prompt and then routes easy questions to a cheap model and complex questions to a larger model.

Bedrock Model Evaluations.

Bedrock Model Evaluations focuses on grading the raw foundation models.

- **Automated Evaluations** - best for objective tasks where a clear right/wrong answer exists, or for checking safety boundaries. It is used for text summarization, classification and Q&A.

  Use LLM-as-Judge when need to evaluate subjective criteria (tone, relevance, toxicity, helpfulness) rapidly at scale, especially when lack human-written reference answers.

- **Human Evaluations** - best for subjective tasks where tone, and brand voice matter. Bedrock provisions a customized web UI where human workers can rate model outputs side-by-side.

Token Efficiency & Savings.

CountTokens API - before we send a prompt to an expensive model, we need to know if it will fit within the model's context window and how much it will cost. This **API is free**.

CloudWatch Observability Metrics

- **InputTokenCount & OutputTokenCount**
- **TTFT (Time to First Token)** - measures the latency between the user hitting send and the model returning the very first chunk of text.
- **Invocation Throttles** - if we exceed Bedrock quota, Bedrock will throttle us. We can monitor these and use exponential backoff in our application code to handle them.

Context Window Optimization (RAG Cost Savings)

- Limit number of Chunks - instead of retrieving the top 10 chunks from vector db, reduce it to the top 3 or 5 most relevant chunks.

- Metadata Filtering - retrieves fewer, highly relevant chunks, saving tokens.
- Pruning Conversation History - either drop older messages (sliding window) or use a small model to summarize the history before passing it to the main model.

▼ Maximizing Resource Utilization.

If we fine-tune a model in Bedrock (create a customized model), we **must purchase provisioned throughput to deploy it**. We cannot run custom models on on-demand capacity.

In API calls (InvokeModel), if we are using on-demand, we pass the standard model ID. If we are using provisioned throughput, we must pass the specific provisioned model ARN as the modelID.

> If developer bought provisioned throughput but is still getting throttled or billed on-demand, it's because they forgot to update their code to use the provisioned ARN.

Bedrock Batch Inference

Not all GenAI tasks need real-time responses. Instead of sending API requests one by one, bundle prompts into a JSONL and upload it to an S3. **Trigger a batch inference job**, Bedrock processes it asynchronously, and deposits the results into a destination S3 bucket.

Capacity Planning

AWS tracks our GenAI usage using two primary metrics:

- **TPM (Tokens Per Minute)** - limits **how much text we can send and receive per minute**.
- **RPM (Requests Per Minute)** - limits the raw **number of API calls we make per minute**.

If our application hits these limits, we will get a `ThrottlingException`. The fix is to implement exponential backoff and retry in code.

▼ Bedrock Inference Profiles.

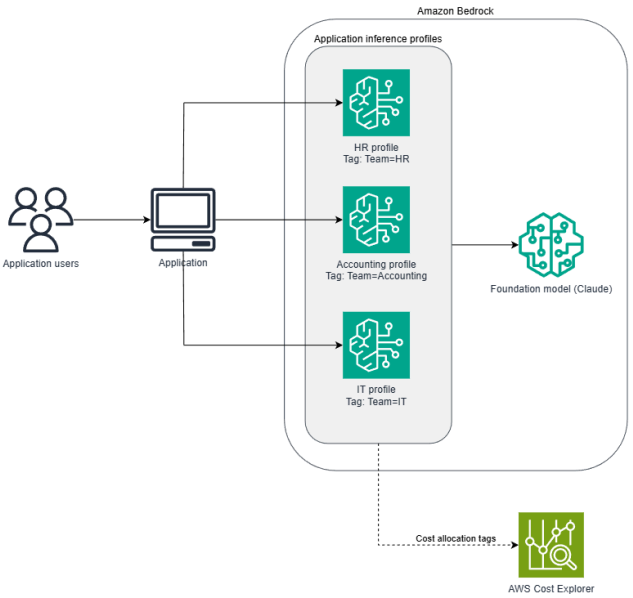
Inference Profile is a logical endpoint (ARN) that we invoke instead of calling the raw FM ID directly. They are used to improve resilience, handle high traffic, and track application costs.

System-Defined Profiles (Cross-Region Inference) - pre-created and managed by AWS. Bedrock automatically and dynamically routes our inference request to the optimal, available AWS region. Used to avoid `ThrottlingException` limits for a FM without requesting quota increases.

CRI compute is global, but Data Storage is local. AWS guarantees that the user's prompt, the generated response, and the CloudWatch Logs are **only stored in the source region**. Data is processed in the destination region in-memory and immediately discarded.

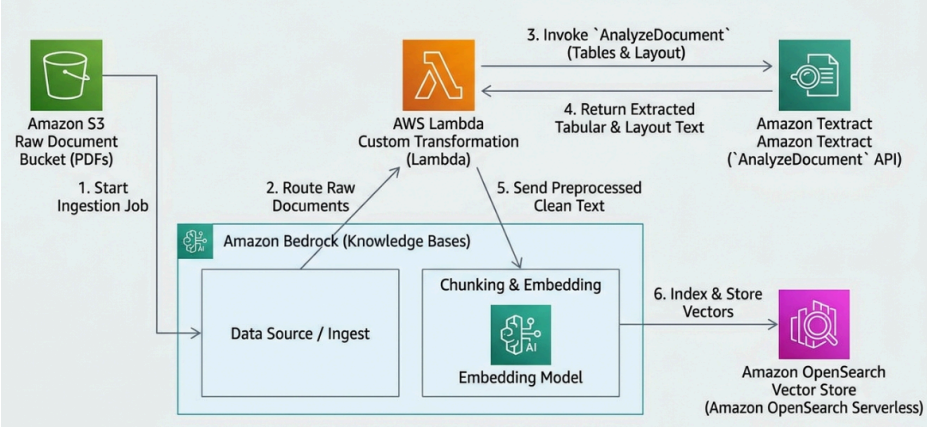
Application Inference Profiles - custom profiles created and managed by us. They can wrap either a single-region model or a system-defined cross-region profile.

By default, all Bedrock model invocations are billed together, making it hard to see which team is spending what. We can create an Application Inference Profile and attach Cost Allocation Tags. Developers then use this ARN in API calls instead of the base model ID.



Bedrock Knowledge Bases - Lambda Custom Parsing.

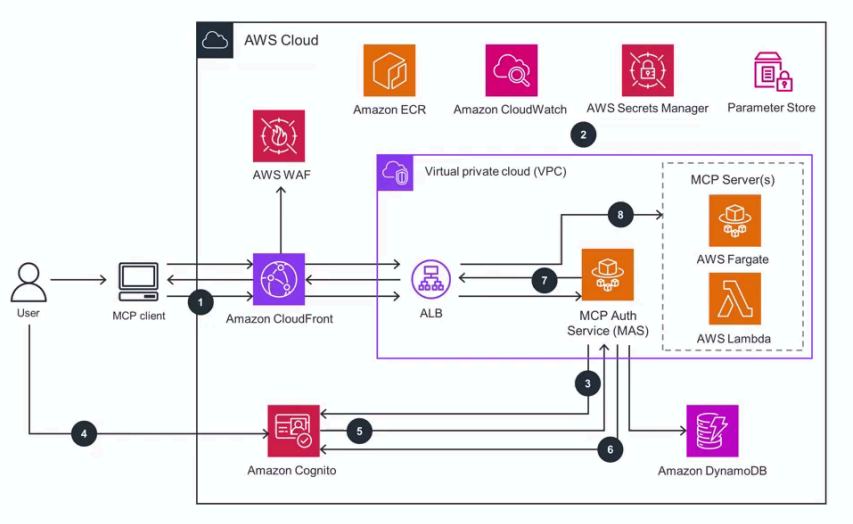
We can configure the Knowledge Base to **route documents to an Lambda function** before they are chunked and embedded. Inside that Lambda function, invoke Amazon Textract (AnalyzeDocument API) to accurately extract the tabular data and layout structure from the PDF, then pass the clean text back to the Knowledge Base.



Solution Architecture - Deploying MCP on AWS

- For stateless and light access we use Lambda function, and for complex tools we use Fargate.
- We can use API Gateway for expsoing endopoints.

> We can also deploy MCP with Bedrock AgentCore.

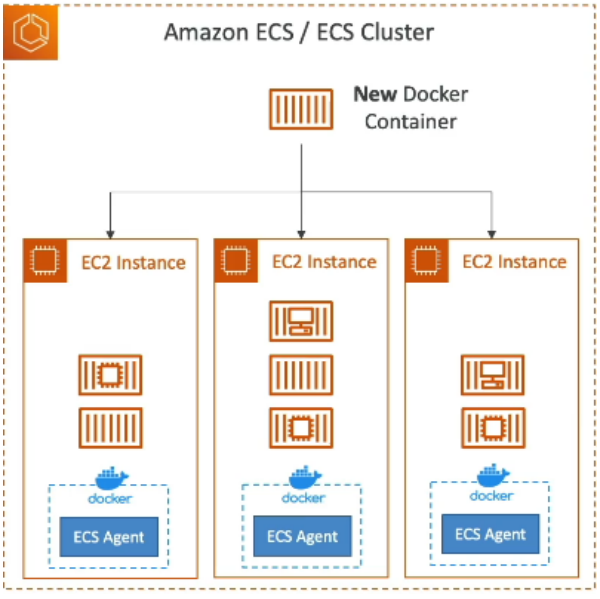


Section III

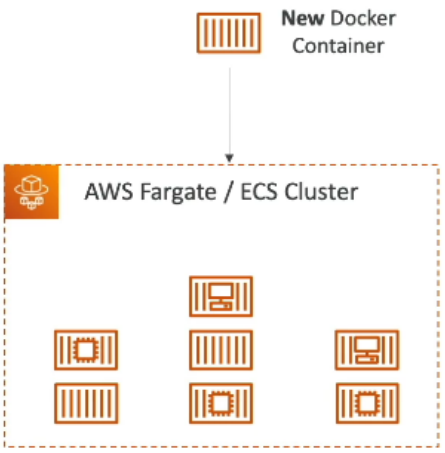
MLOPs with AWS

▼ **ECS & Fargate.**

ECS [EC2 Launch Type] - we must provision & maintain the infrasturcture (EC2 instances). Each EC2 Instance must run the ECS Agent to register in the ECS Cluster. AWS takes care of starting/stopping containers.

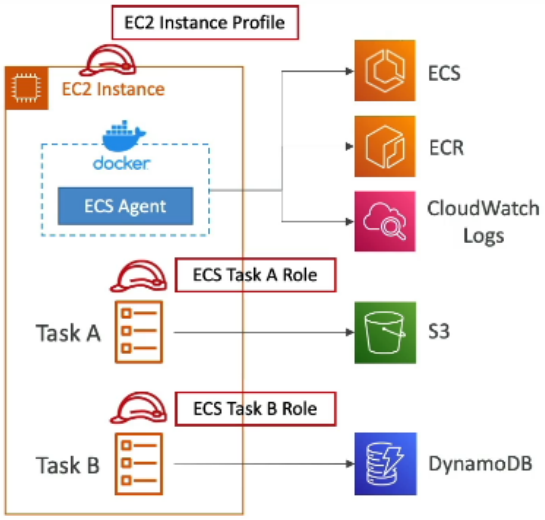


ECS [Fargate Launch Type] - we do not provision the infrasturcture, it's all serverless. We just create task definitions. AWS just runs ECS Tasks for us based on the CPU/RAM we need. To scale, we just need to increase the number of tasks (no more EC2 instances).



IAM Roles for ECS

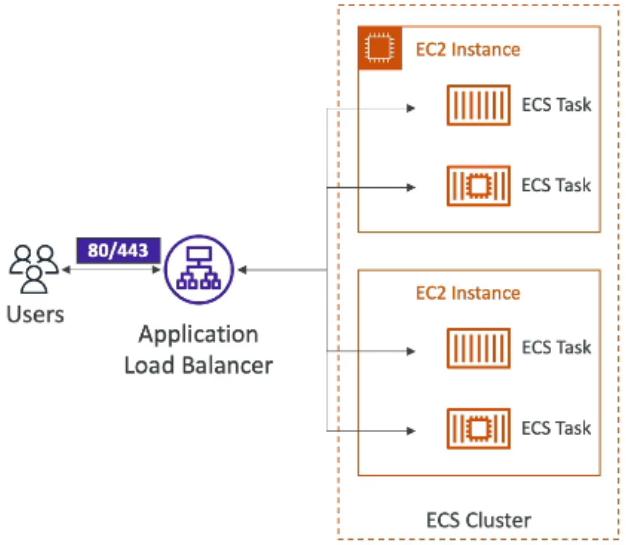
- **EC2 Instance Profile** (EC2 Launch Type only) - used by ECS agent to make API calls to ECS service. We can send container logs to CloudWatch Logs, pull Docker image from ECR, reference sensitive data in Secrets Manager or SSM Parameter Store.
- **ECS Task Role** - allows each task to have a specific role. Each role can use different ECS services.



ECS - ELB Integrations

ALB - supported and works for most use cases.

NLB - recommended only for high throughput/high performance use cases or to pair it with AWS Private Link.



ECS Data Volumes (EFS)

We can mount EFS file systems onto ECS tasks. It works for both EC2 and Fargate launch types.

Tasks running in any AZ will share the same data in the EFS file system. If we use Fargate + EFS it's totally serverless.

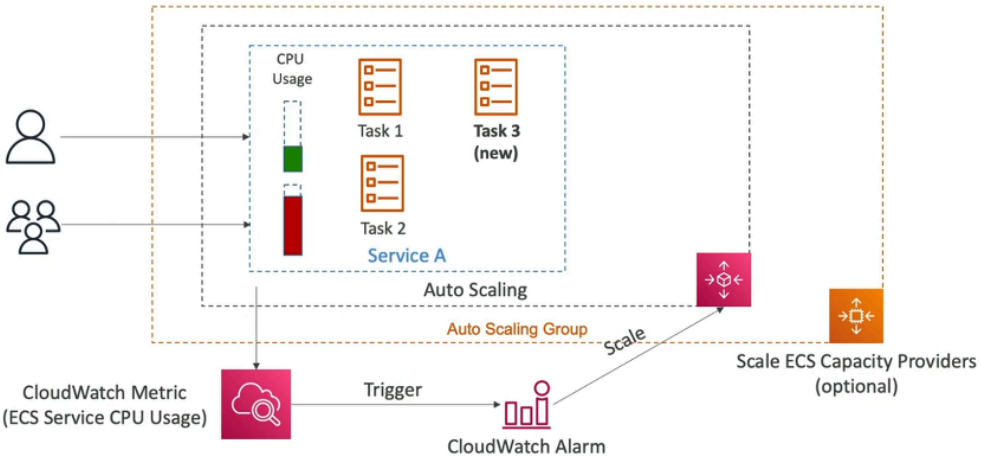
ECS Auto Scaling.

With **ECS Auto Scaling** we can automatically increase/decrease the desired number of ECS tasks. Fargate Auto Scaling is much easier to setup because it is Serverless.

ECS Auto Scaling is similar but not equal to EC2 Auto Scaling (task level vs instance level).

- **Target Tracking Scaling** - scale based on target value for a specific CloudWatch metric.
- **Step Scaling** - scale based on a specified CloudWatch Alarm.
- **Scheduled Scaling** - scale based on a specified date/time (predictable changes).

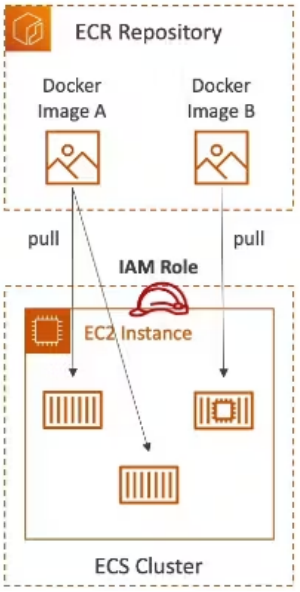
For EC2 launch type we can accommodate ECS scaling by adding underlying EC2 instances (using ASG or ECS Cluster Capacity Provider). ECS Cluster Capacity Provider is better.



ECR.

ECR - private docker registry on AWS. It stores and manages Docker images on AWS. It is fully integrated with ECS and backed by S3.

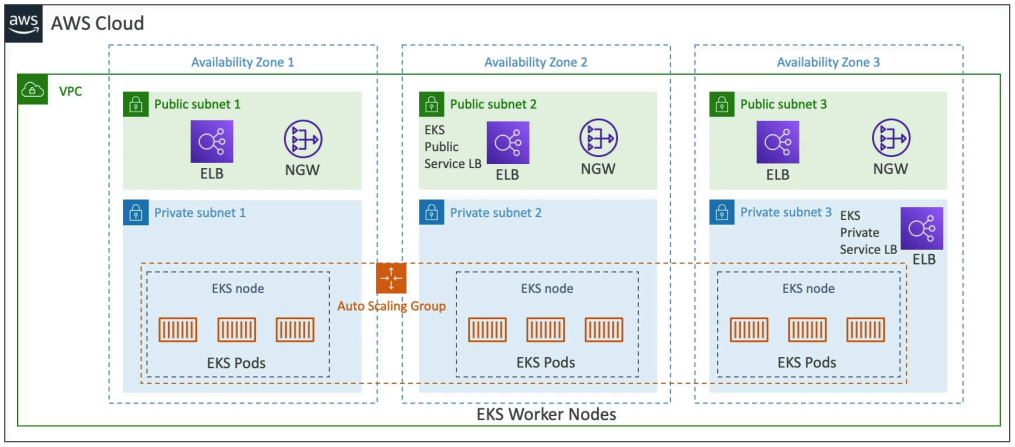
Access is controlled through IAM. It supports image vulnerability scanning, versioning, image tags, image lifecycle...



EKS.

EKS is a managed service that simplifies the process of running Kubernetes on AWS. It is an alternative to ECS, similar goal but different API. Kubernetes is cloud-agnostic (can be used in any cloud).

EKS supports EC2 if we want to deploy worker nodes or Fargate to deploy serverless containers.



We can attach data volumes to our EKS cluster. We need to specify StorageClass manifest on our EKS cluster. It leverages a **Container Storage Interface (CSI)** compliant driver.

We have support for EBS, EFS (works with Fargate), FSx for Lustre, FSx for NetApp ONTAP.

Amazon EKS Connector is a feature that allows us to connect and manage Kubernetes clusters running outside of AWS (whether on-premises or in other cloud environments) within the Amazon EKS console.

Amazon EKS Anywhere is a deployment option for Amazon EKS that enables us to create and operate Kubernetes clusters on our own infrastructure, such as on-premises data centers or other cloud

environments.

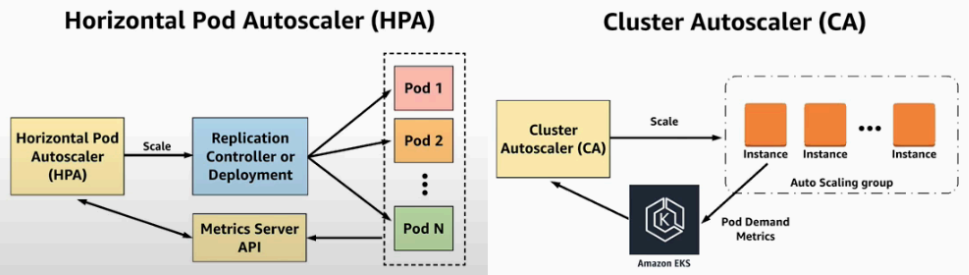
EKS Node Types

- **Managed Node Groups** - creates and manages Nodes (EC2 instances) for us. Nodes are part of an ASG managed by EKS. It supports on-demand or spot instances.
- **Self-Managed Nodes** - nodes created by us and registered to the EKS cluster and managed by an ASG. We can use prebuilt AMI (EKS Optimized AMI). It supports on-demand or spot instances.
- **AWS Fargate** - no maintenance required, no nodes managed.

▼ EKS Auto Scaling.

EKS Metrics Server - lightweight, scalable aggregator of resource usage data in our Kubernetes cluster. The primary use of the Metrics Server is to enable Horizontal Pod Autoscaling (HPA). **HPA** automatically adjusts the number of pod replicas in a deployment, replication controller, or stateful set based on observed CPU utilization.

Kubernetes Cluster Autoscaler automatically adjusts the size of the Kubernetes cluster (e.g. the number of nodes) based on the resource requirements of the workloads running on the cluster.



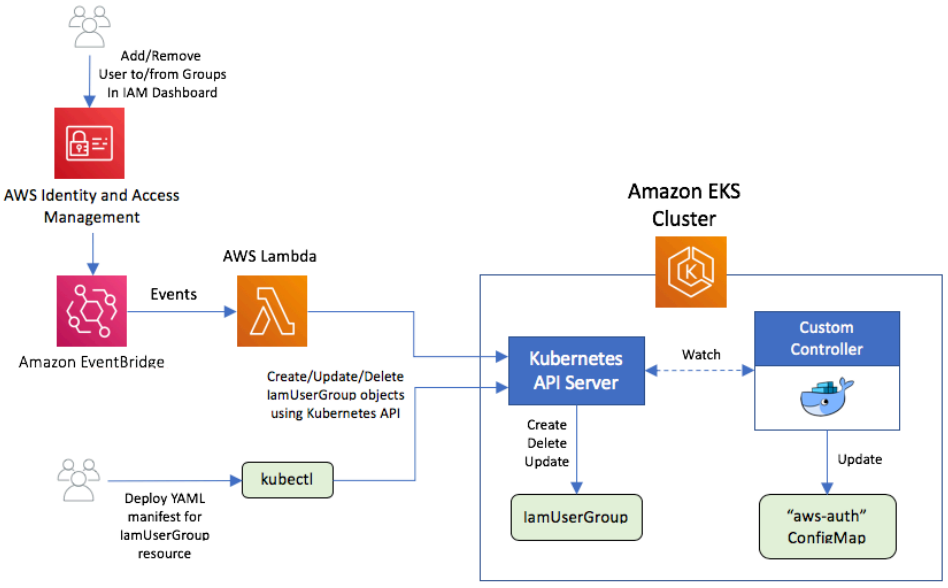
Karpenter - flexible, high-performance Kubernetes cluster autoscaler that launches appropriately sized compute resources, like Amazon EC2 instances, in response to changing application load. It integrates with AWS to provision compute resources that precisely match workload requirements.

▼ EKS Security.

EKS uses IAM to provide authentication to our Kubernetes cluster, but it still relies on native Kubernetes Role-Based Access Control (RBAC) for authorization. This means that IAM is only used for the authentication of valid IAM entities. All permissions for interacting with our Amazon EKS cluster's Kubernetes API are managed through the native Kubernetes RBAC system.

Access to our cluster using IAM entities is enabled by the IAM Authenticator for Kubernetes, which runs on the EKS control plane. The authenticator gets its configuration information from the *aws-auth ConfigMap* (AWS authenticator configuration map).

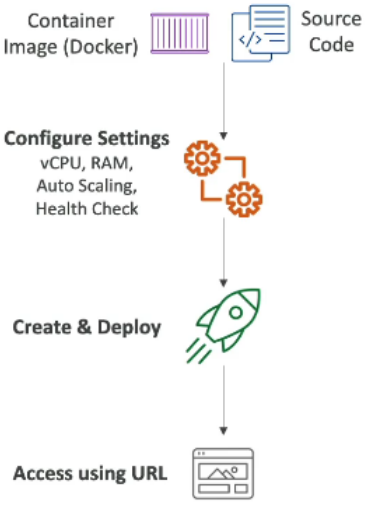
The **aws-auth ConfigMap** is automatically created and applied to our cluster when we create a managed node group or when we create a node group using **eksctl**. It is initially created to allow nodes to join our cluster, but we also use this ConfigMap to add role-based access control (RBAC) access to IAM users and roles.



AWS App Runner.

App Runner is a fully managed service that makes it easy to deploy web applications and APIs at scale. It automatically builds and deploys the web app. We dont need intrastructure experience (start with our source code or container image).

We can connect to database, cache and message queue services. It has automatic scaling, load balancer and encryption.

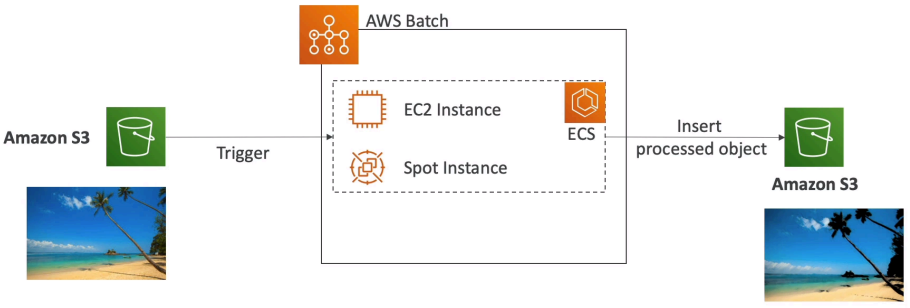


Use cases: web apps, APIs, microservices, rapid production deployments.

AWS Batch.

AWS Batch is a fully managed batch processing service at any scale (efficiently runs 100 000s of computing batch jobs on AWS). A batch job is a job with a start and an end.

Batch jobs are defined as Docker images and run on ECS. Batch will dynamically launch EC2 instances or Spot Instances and will provision the right amount of compute/memory.



▼ **CloudFormation.**

CloudFormation is a declarative way of outlining our AWS Infrastructure, for any resources.

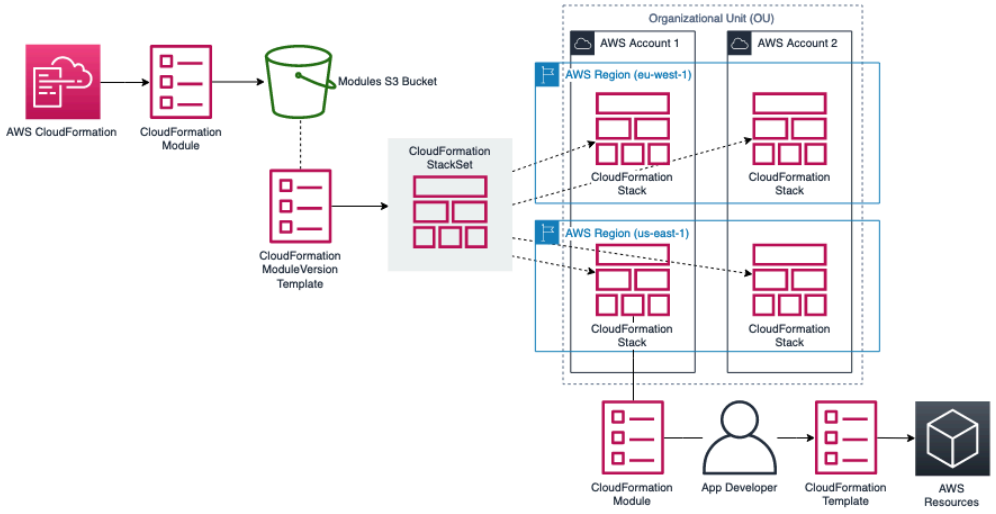
> Example: We want security group, two EC2 instances using that security group, an S3 bucket and ELB in front of these machines. CloudFormation creates all those for us, in the right oreder, with exact configuration that we specify.

Benefits of CloudFormation

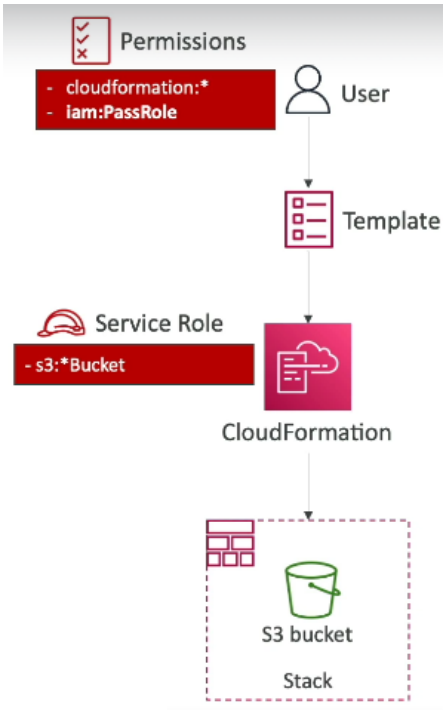
- Infrastructure as code - No resources are manually created, which is excellent for control. Changes to the infrastructure are reviewed through code.
- Cost - Each resources within stack is tagged with an identifier so we can easily see how much a stack costs us. We can estimate the costs of our resources using the CloudFormation template.
- Productivity - Ability to destroy and re-create an infrastructure on the cloud on the fly. We can automate generation of diagrams for our templates.

Application Composer - A visual tool designed to simplify the process of building serverless applications on AWS. It enables developers to drag and drop various AWS services to create an architecture diagram that represents their application.

- **Stacks** are the core unit of deployment in CloudFormation. A stack is a collection of AWS resources that we can manage as a single unit. When you create, update, or delete a stack, CloudFormation automatically provisions or deletes the associated resources in a predictable way.
- **Template** is a JSON or YAML formatted text file that describes the resources AWS CloudFormation will provision in our stack.
- **StackSets** extend the functionality of stacks by allowing us to manage stacks across multiple accounts and regions.



CloudFormation Service Role - IAM role that allows CloudFormation to create/update/delete stack resources on our behalf. It is useful if we want to achieve the least privilege principle, but we dont want to give the user all the required permissions to create the stack resource. User must have iam:PassRole permissions.



▼ **CodeDeploy & CodeBuild.**

CodeDeploy is an AWS **hybrid** service that automates the deployment of applications. It works with EC2 Instances and On-Premises Servers.

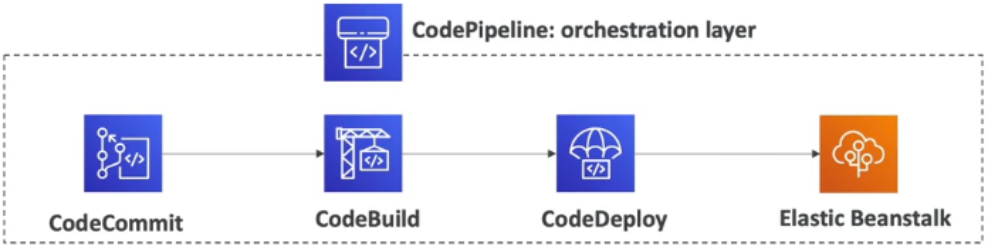
Servers/Instances must be provisioned and configured ahead of time with the CodeDeploy Agent.

CodeCommit is a fully managed source control service that makes it easy for teams to host secure Git repositories. It is scalable and highly available.

CodeBuild is a fully managed build service that compiles source code, runs tests, and produces deployable software artifacts. It is continuously scalable and highly available. We pay only for the build time.

CodePipeline.

CodePipeline is fully managed service that orchestrate the different steps to have the code automatically pushed to production. It is basis for CI/CD.

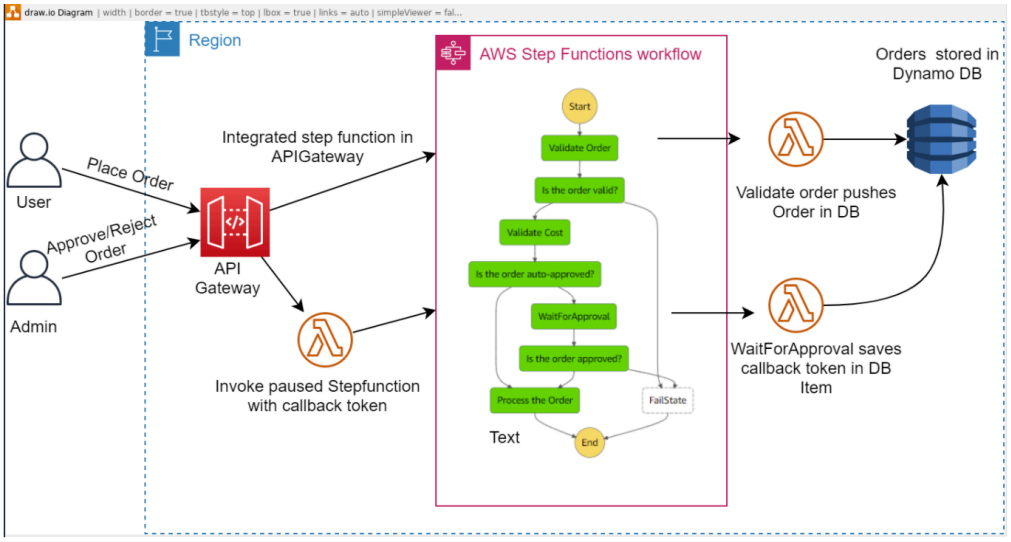


Step Functions.

Step Functions is a serverless orchestration service that allows us to coordinate multiple AWS services into serverless workflows. It orchtestrates our Lambda functions.

It can be integrated with EC2, ECS, on-remises servers, API Gateway, SQS, etc ...

We can use it for implementing human approval feature.



Step Functions Local is a tool that allows us to run and test our Step Functions state machines locally on our development machine. It can help us develop and debug your workflows without needing to deploy them to the AWS cloud, which can save time and costs.

Step Functions States.

Task State - does something with Lambda or other services.

Choice State - adds conditional logic via Choice Rules.

Wait State - delays state machine for a specified time.

Parallel State - adds separate branches of execution.

Map State - iterates over a collection of items and process each one in parallel or sequentially, depending on how we configure it. It allows us to specify a JSON array (or list) as input and then iterate over that array and apply a specified state machine (or sub-state machine) to each item.

The Map state allows us to configure how many iterations are processed in parallel. We can set a MaxConcurrency value to limit the number of parallel executions.

- **Inline Mode** - each item in the array is processed using the same state machine specified directly within the Map state's Iterator. State machine logic defined in the Iterator is directly applied to each item in the array.

```
{
  "Type": "Map",
  "ItemsPath": "$.items",
  "Iterator": {
    "StartAt": "ProcessItem",
    "States": {
      "ProcessItem": {
        "Type": "Task",
        "Resource": "arn:aws:lambda:REGION:ID:function:ProcessItemFunctio
n",
        "End": true
      }
    }
  },
  "End": true
}
```

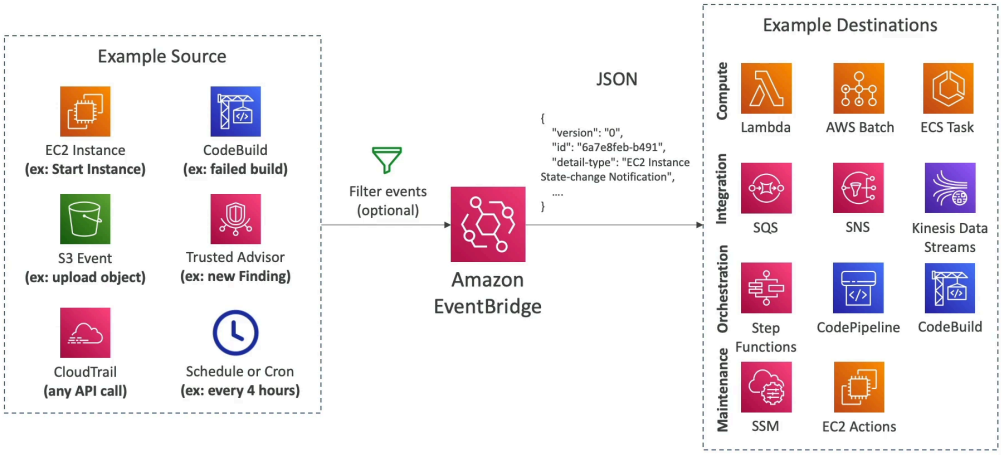
- **Distributed Mode** - the Map state is used in conjunction with a nested state machine or sub-state machine that is applied to each item in the array. It allows a more complex or different workflow to be used for each item. It is designed specifically for **massive parallelism** by breaking up large-scale processing tasks into smaller batches.

```
{
  "Type": "Map",
  "ItemsPath": "$.items",
  "Iterator": {
    "StartAt": "SubMachine",
    "States": {
      "SubStateMachine": {
        "Type": "StateMachine",
        "StateMachineArn": "arn:aws:states:REGION:ID:stateMachine:SubMachin
e",
        "End": true
      }
    }
  },
  "End": true
}
```

EventBridge.

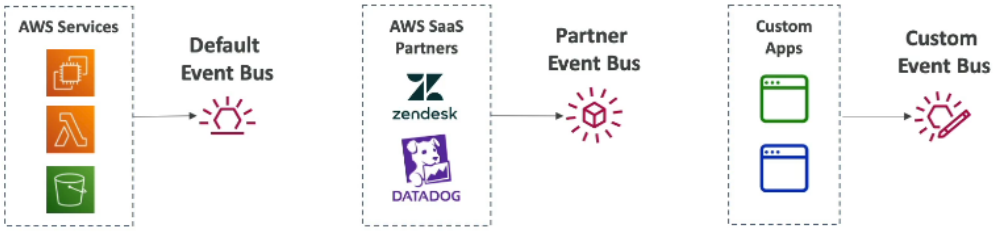
EventBridge is a serverless event bus service that makes it easy to connect applications using data from our own applications, integrated SaaS applications, and AWS services.

We can schedule events with CRON jobs or we can make event rules to react to a service doing something. Also it's possible to trigger Lambda functions, send SQS/SNS messages ...



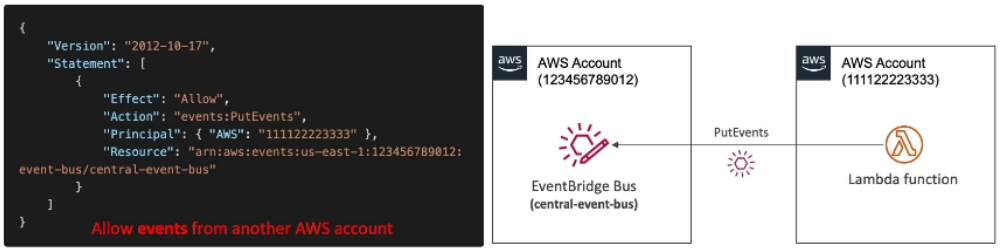
EventBridge Schema Registry - allows us to generate code for our application, that will know in advance how data is structured in the event bus. Schema can be versioned.

Event Bus Types



Event buses can be accessed by other AWS accounts using resource-based policies.

Resource-based Policy - manages permissions for a specific Event Bus. It is used to aggregate all events from our AWS Organization in a single AWS account or AWS region.



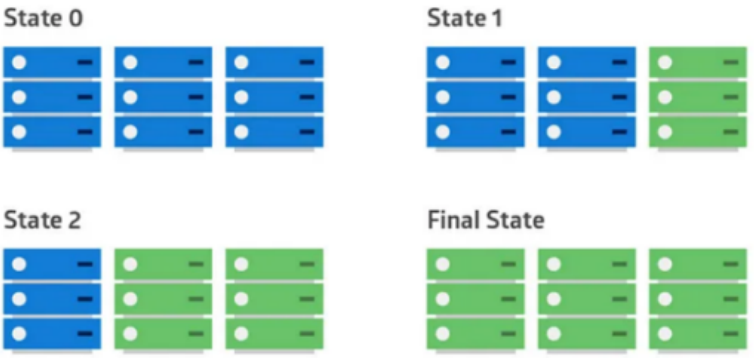
We can archive events sent to an event bus (indefinetely or set period) and we can replay archived events.

▼ **Deployment Strategies.**

All-at-Once (Big Bang) - deploys the new version of the application to all instances at once. It is simple and fast, with no need for complex infrastructure setups. It has high risk of downtime if the new version has issues, no easy rollback. If something goes wrong, the entire system can be affected.

Use Cases: **Non-critical applications where downtime is acceptable or where rollback procedures are straightforward**.

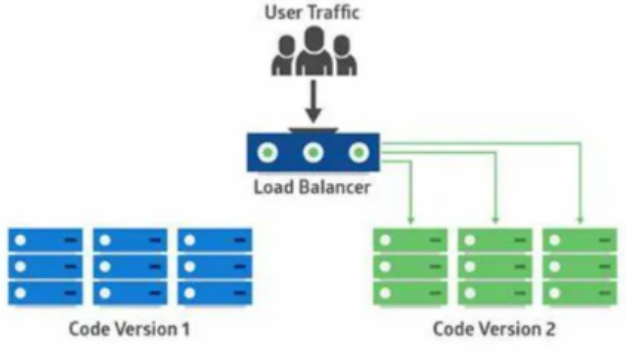
Rolling Deployment - gradually replaces instances of the old version with instances of the new version. Reduces the risk of full-scale failure and can be quickly fixed or rolled back.



Use Cases: **Applications that need to minimize downtime but can tolerate temporary inconsistencies**.

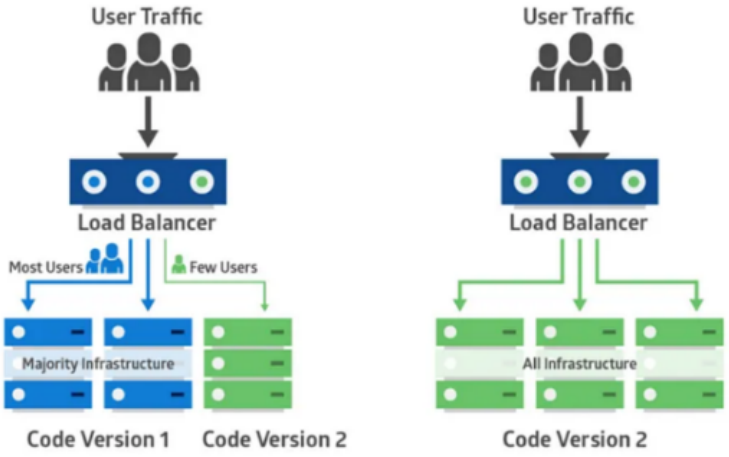
Blue-Green Deployment - two identical environments are maintained: one (blue) running the current version and another (green) running the new version. Traffic is switched from the blue environment to the green one once the new version is ready.

It is zero-downtime deployment, easy rollback by switching traffic back to the blue environment. Testing and validation can be done on the green environment.



Use Cases: **Applications requiring zero downtime and where rollback needs to be swift and reliable**.

Canary Deployment - small portion of the traffic is directed to the new version while the rest remains on the old version. Gradually, more traffic is shifted to the new version as confidence in its stability grows. It allows for real-world testing with minimal risk.



Use Cases: Applications with a **large user base or critical services where early feedback and incremental rollout are important**.

A/B Testing Deployment - similar to Canary deployment, but with a focus on running two or more versions simultaneously to compare performance or user experience. Different users are directed to different versions.

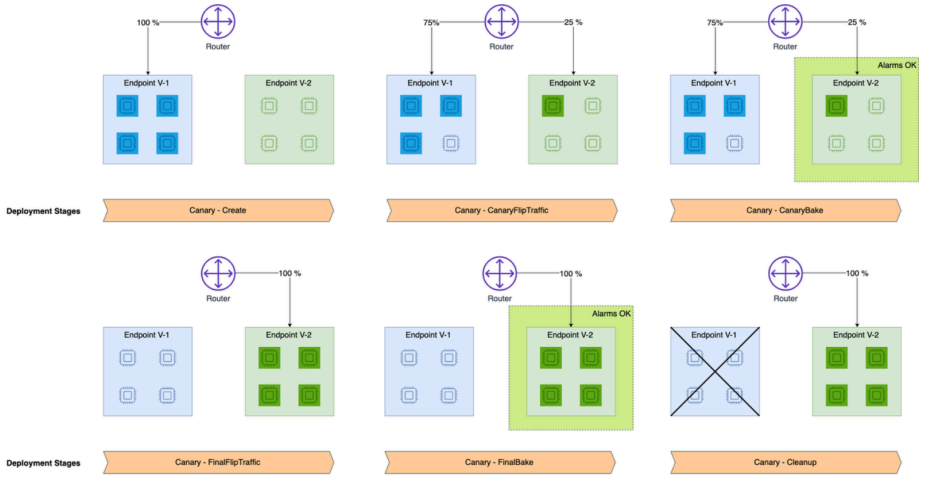
Use Cases: **Testing new features, UI/UX changes, or optimizations**.

Shadow Deployment - new version is deployed alongside the old version, but the new version does not serve production traffic. Instead, it processes a copy of the production traffic to see how it behaves.

Use Cases: Validating major updates or entirely new services without risking production stability.

▼ **SageMaker Deployment Guardrails & Shadow Tests.**

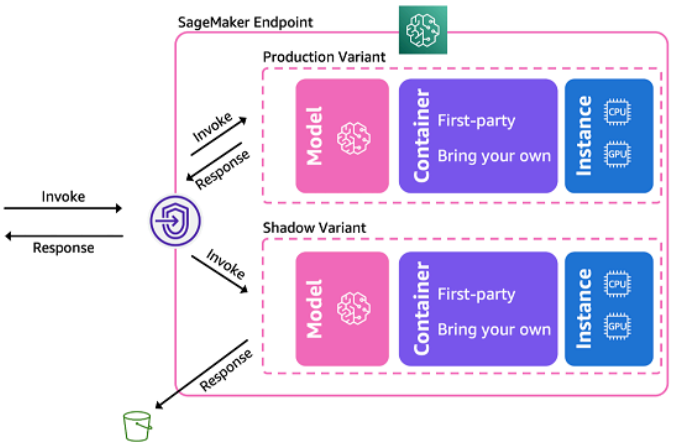
SageMaker Deployment Guardrails - when updating a real-time SageMaker endpoint **use Blue/Green deployments to shift traffic** safely and automatically rollback if things go wrong.



Guardrails rely on CloudWatch Alarms. We configure alarms, and if Model B triggers this alarm during the shifting process, SageMaker **automatically rolls back 100% of traffic to Model A**.

Shadow Testing - allows to test how Model B performs on real production data without the user ever knowing.

When user sends a request to the SageMaker endpoint. SageMaker sends the request to Model A (prod) and duplicates the request to Model B (shadow). Model B generates a prediction, but it is NOT sent to the user. It is just logged to S3/CloudWatch to analyze later.



SageMaker & Docker.

If we bring our own container, we must adhere to SageMaker's strict file structure and networking rules.

```
/opt/ml
├── input
│   ├── config
│   │   ├── hyperparameters.json
│   │   └── resourceConfig.json
│   └── data
│       └── <channel_name>
│           └── <input data>
├── model
│
├── code
│   └── <script files>
│
└── output
    └── failure
```

- `/opt/ml/input/data` - SageMaker automatically downloads training data from S3 here
- `/opt/ml/model` - **traing code must save the final trained model files here**.

Docker container must contain two executable scripts in the PATH:

1. **train** - SageMaker runs this to start training.
2. **serve** - SageMaker runs this to start the inference server when deploying an endpoint.

Behind that URL, we can host multiple Production Variants. A **Production Variant** is a specific combination of a model and an instance type.

We can do **A/B testing on different variants to save money**.

SageMaker Inference.

Real-Time Endpoints - persisent, always-on EC2 instances behind a REST API. Best for ultra-low latency, sustained traffic. The constraint is that maximum payload size is 6MB, and maximum processing time is 60s.

Serverless Inference - pay only for the duration of the inference, not for idle time. Best for unpredictable, or spiky traffic. The problem is that it suffers from cold starts. If the endpoint hasn't been used in a while, the first request will take longer. It doesnt support GPU! Maximum **timeout is 60 seconds**.

We can purchase Provisioned Concurrency to keep a set number of containers warm and avoid cold starts.

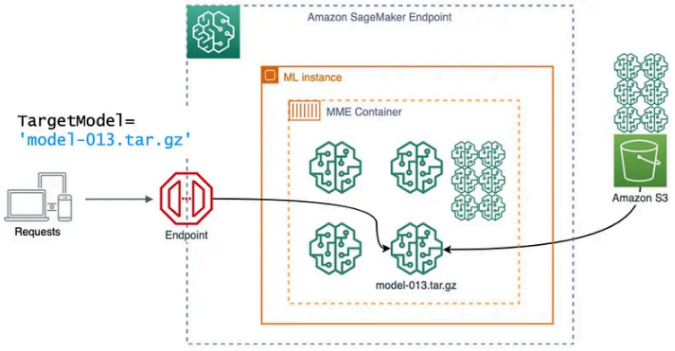
Asynchronous Inference - client sends a request and doesn't wait for the response. Best for large payloads and long processing times. We upload data to S3, then send S3 URI to the async endpoint. The model processes it, and saves the result to S3 and triggers and SNS notification to tell the app it's done.

Async endpoints **can scale down to 0 instances** when there are no requests in the queue.

SageMaker Inference Recommender - takes our model, automatically load-tests it against multiple instance types, and gives us a report showing the trade-offs between latency, throughput, and cost.

SageMaker MME & MCE.

Multi-Model Endpoints allow us to deploy thousands of models behind a single SageMaker endpoint, sharing the same underlying instances and container. It provides massive cost savings compared to provisioning a separate endpoint for every single model.

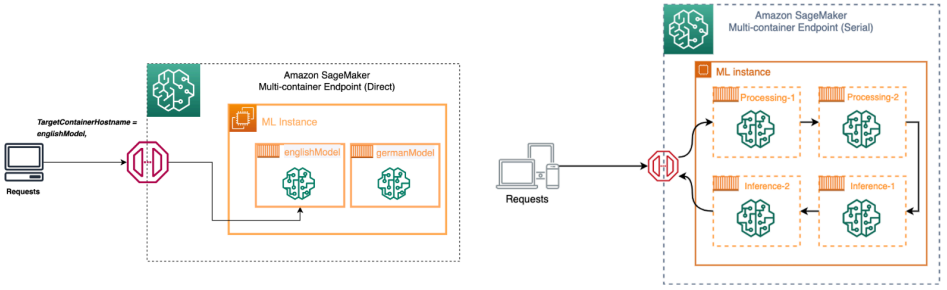


All models on an MME **must use the same ML framework and the same Docker container.**

> If a model is invoked that is not currently in memory, there will be a **cold start latency penalty** while SageMaker downloads it from S3 and loads it into the container.

SageMaker Inference Components - allow us to deploy multiple models to a single SageMaker endpoint, but we **hard-allocate specific compute resources** to each specific model.

Multi-Container Endpoint - up to 15 different containers/frameworks on one endpoint. They can run independently (direct invocation) or sequentially. Goal is to mix different frameworks or build pipelines.



Inference Pipelines - chaining 2 to 15 different containers sequentially on a single endpoint so the client app only has to make one API call.

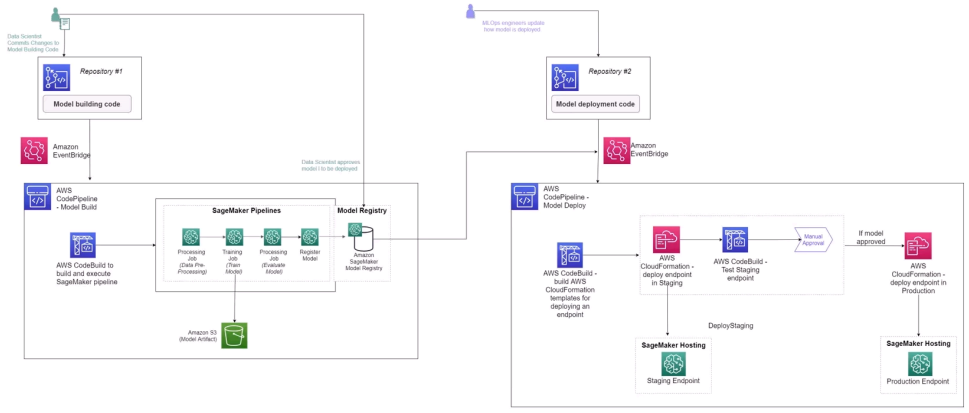
▼ Large Model Inference (LMI) Containers.

LMI Containers - **AWS-provided DL Containers optimized for hosting massive LLMs that cannot fit on a single GPU**. They natively implement Tensor Parallelism and integrate cutting-edge serving libraries like TensorRT-LLM and vLLM to optimize inference speed and memory.

▼ SageMaker Projects.

SageMaker Projects are powered by AWS Service Catalog templates. It helps teams standardize and automate the end-to-end ML workflow using prebuilt templates.

The admin creates a CloudFormation template and publishes it to Service Catalog. When the data scientist clicks "Create Project" in SageMaker Studio, AWS automatically provisions a Git repo, an automated build pipeline, and SageMaker Pipelines.

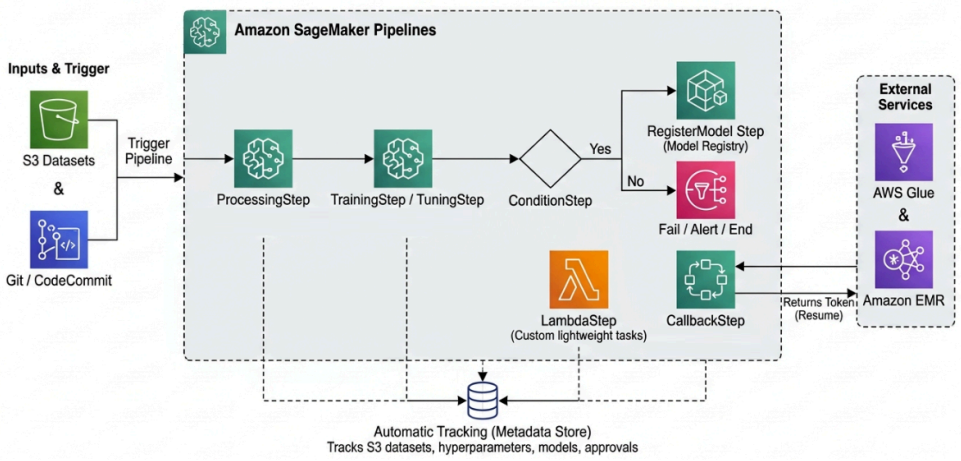


▼ SageMaker Pipelines.

SageMaker Pipelines is fully managed CI/CD service specifically for ML. While CodePipeline is for deploying software/infrastructure, SageMaker Pipelines is for orchestrating ML workflows (CT).

It **automatically tracks everything** (which S3 dataset trained which model, what hyperparameters were used, and who approved it). It also has step caching.

Pipeline Steps

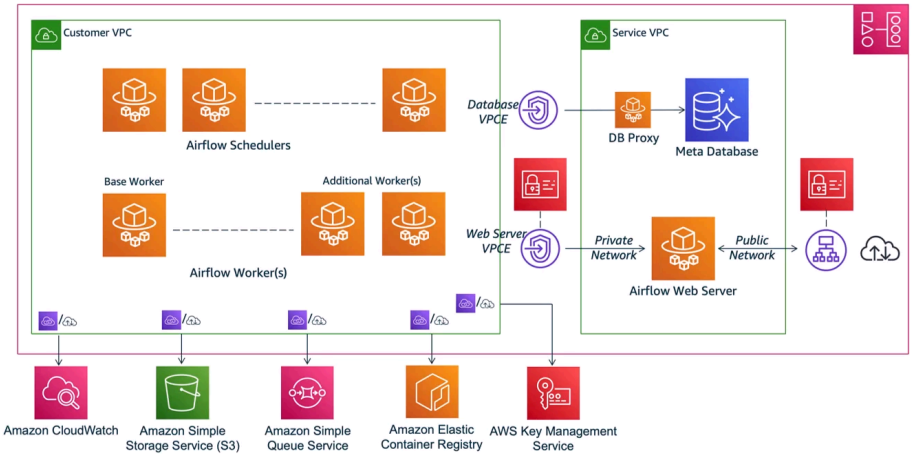


- **ProcessingStep** - runs SageMaker Processing jobs (Data Wrangler, Scikit-Learn data cleaning, custom PySpark feature engineering).
- **TrainingStep / TuningStep** - runs SageMaker Training Jobs or Hyperparameter Tuning Jobs.
- **ConditionStep** - adds branching logic. After a model is trained, the ConditionStep checks if the validation accuracy is acceptable. If yes, it routes to a **RegisterModel** step. If no, the pipeline ends or sends a failure alert.
- **CallbackStep** - **integrates external AWS services** (like Glue or EMR) into the SageMaker Pipeline. It passes a token to the external service and **pauses the pipeline** (saving money) until the external service finishes and returns the token.
- **LambdaStep** - runs a custom Lambda function for lightweight tasks within the pipeline.

▼ Amazon Managed Workflows for Apache Airflow (MWAA).

Apache Airflow is batch-oriented workflow tool. We can use it for develop, schedule, and monitor our workflows. Workflows are defined as python code that creates a DAG.

MWAA provides a managed service for Apache Airflow so we dont have to deal with installing or maintaining it. It runs within a VPC.



DAGs are uploaded into S3. MWAA picks it up and orchestrates and schedules the pipelines defined by each DAG. The S3 bucket must have versioning enabled.

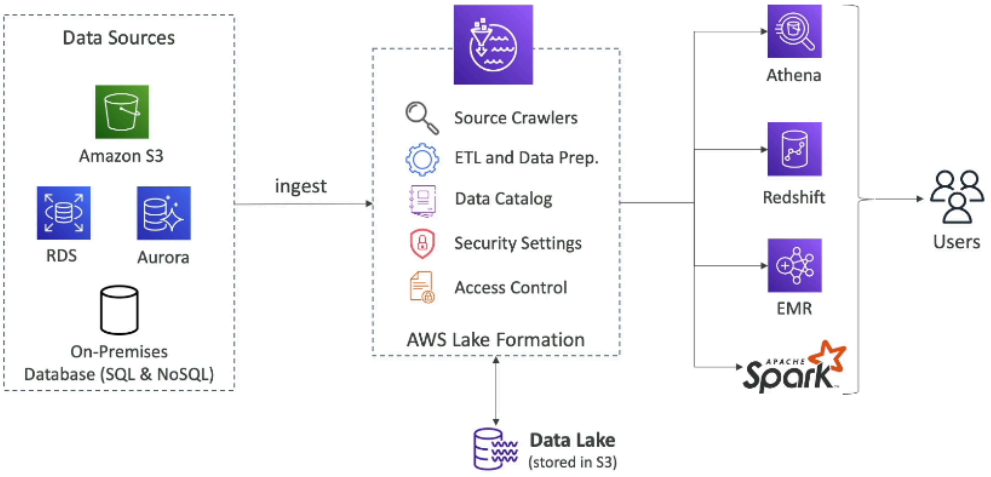
MWAA uses the Celery Executor. We configure a min and max number of workers. MWAA automatically scales out the worker nodes to handle the load.

▼ AWS Lake Formation.

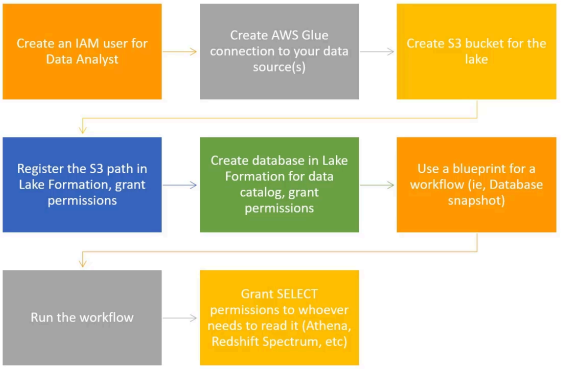
Lake Formation is a fully managed service that makes it easy to setup a data lakes. It is built on top of AWS Glue.

With Lake Formation we can discover, cleanse, transform and ingest data. It combines structured and unstructured data. It has blueprints for S3, RDS, relational & NoSQL DB, etc. Lake Formation Blueprints can help us migrate data from one place to the data lake.

We can have fine-grained access controls for our applications at the row and column level.



Building a Lake



Lake Formation - Centralized Permissions

Lake Formation solves problem of security management by centralizing it. AWS Lake Formation simplifies the security of our data lake, making it easier to enforce data governance policies. We can consolidate data from multiple accounts into a single account.

If we want cross-account permission:

- Recipient must be set up as data lake admin.
- We can use AWS RAM for acfounts external to our organization.
- Add IAM permission for cross-account access.

Lake Formation Governed Tables are new type of S3 tables that support ACID transactions across multiple tables. Works with streaming data too and can query with Athena.

▼ **Lake Formation Fine-Grained Access Control.**

Column-Level Security - hides specific columns from specific users.

Row-Level Security (Data Filters) - filters out specific rows based on who is querying the data.

Cell-Level Security - the combination of row and column level security (restricting a specific column only for specific rows).

▼ **Hardware Selection & GPU/CPU/Custom Silicon.**

NVIDIA GPUs (P-instances, G-instances) - use CUDA/cuDNN/TensorRT libraries. If a container or model code is built using NVIDIA libraries, it must run on an NVIDIA GPU instance.

AWS Custom Silicon (Inferentia & Trainium) - AWS-designed chips that use AWS Neuron SDK. They **do not support NVIDIA/CUDA natively**. To use it, we must recompile the model using Neuron Compiler (neuron-cc).

SageMaker pre-built Deep Learning Containers already include NVIDIA CUDA drivers. We do not need to bring our own container just to use a GPU. We must bring our own image only when we have a highly customized C++ library, unsupported legacy CUDA versions, or a totally custom ML framework.

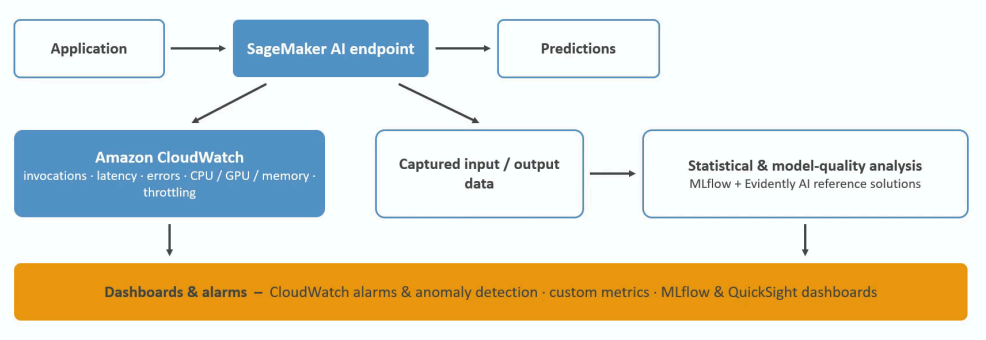
Instance Types & Performance (Inference)

- DL / Transformers (lowest cost) - use Inferentia (Inf1/Inf2).
- DL (must keep existing NVIDIA/CUDA code) - use GPU instances (G4dn, G5). G-instances are cost-effective for inference.
- Tabular Data / Trees - use CPU instances (M-family for memory-bound, C-family for compute).

Instance Types & Performance (Training)

- Massive DL / LLMs (lowest cost) - use Trainium (Trn1).
- Massive DL/ LLMs (multi-node network) - use heavy GPU instances (P3, P4, P5) with EFA.

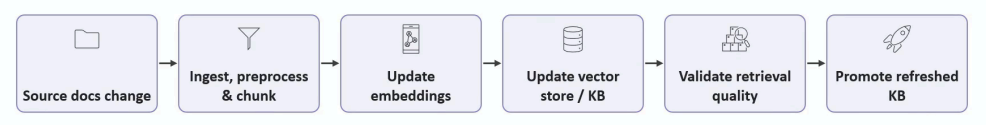
▼ Solution Architecture - Model Monitoring



▼ Solution Architecture - Automating Knowledge Base Updates

When source document change, the retrieval index must stay in sync. Bedrock Knowledge Bases sync is incremental (StartIngestionJob re-processes added, modified & deleted docs). Because it is incremental we get cost efficiency from this.

Refresh succeeds only when ingestion completes and retrieval quality is validated.



▼ Solution Architecture - CloudFormation for SageMaker MLOps

We must use CloudFormation for repeatable provisioning across multiple environments (Dev, Test, Prod).

> Should not use Jupyter Notebooks or imperative scripts to provision production infrastructure, as they lead to config drift and are hard to maintain.

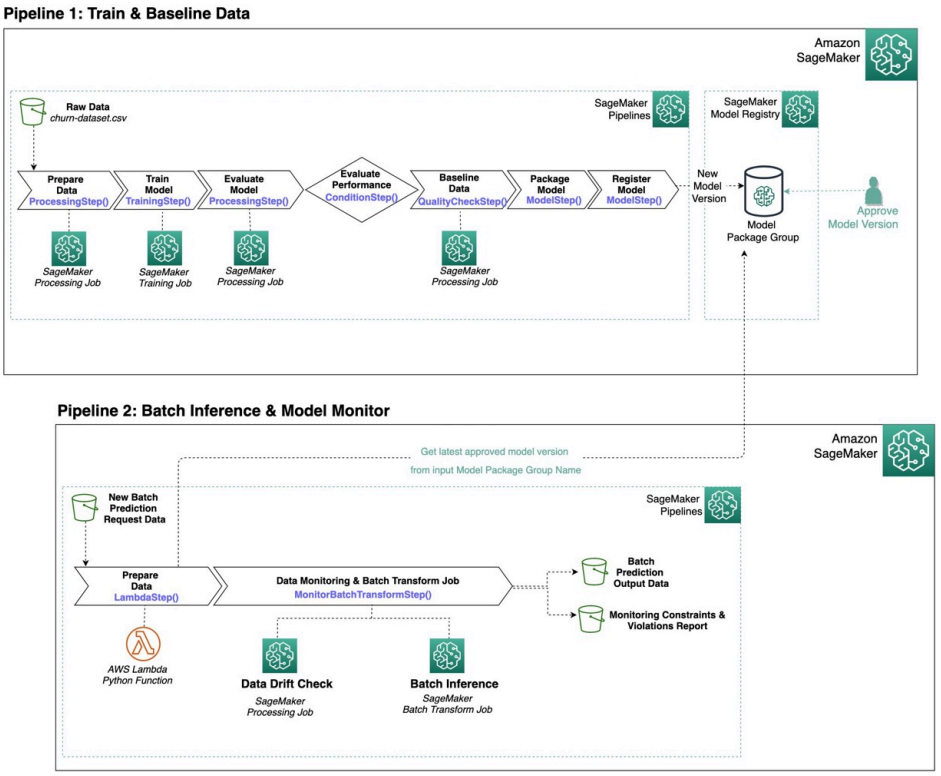
To deploy a SageMaker Endpoint via CloudFormation, we must define three distinct resources in template:

1. `AWS::SageMaker::Model`

2. `AWS::SageMaker::EndpointConfig` (define the Production Variant, which dictates the instance type and starting capacity/instance count).
3. `AWS::SageMaker::Endpoint`

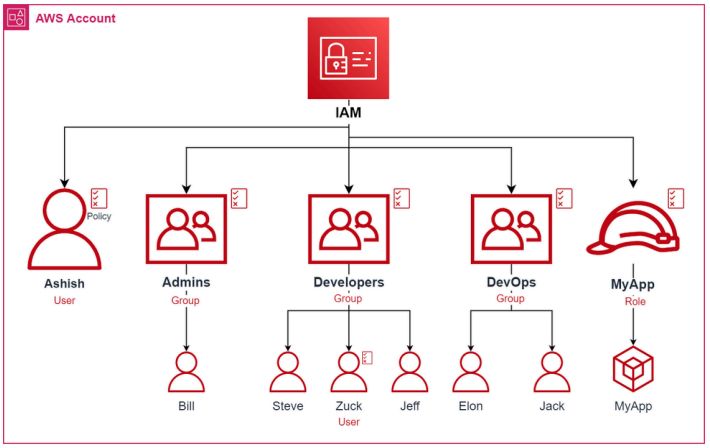
In complex CI/CD pipelines, we often have multiple CloudFormation stacks. To pass values from the Compute Stack to the Workflow Stack without hard-coding or manual edits, we must use **CloudFormation Cross-Stack References**.

▼ **Solution Architecture - Model Retraining with SageMaker Pipelines**



Security, Identity & Compliance

▼ **IAM Users, Groups and Policies.**



Root Account - created by default, shouldn't be used or shared.

IAM Users - people within our organization and they can be grouped. Users dont have to belong to a group and one user can belong to multiple groups. Each user has a unique name and credentials (password or access keys).

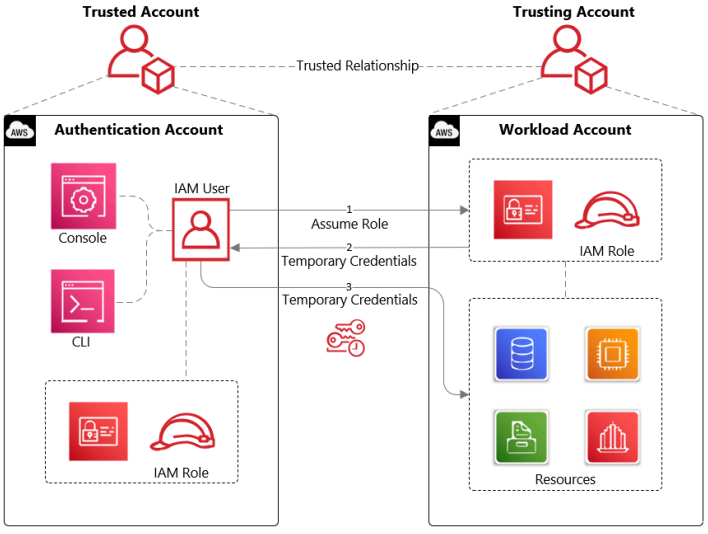
IAM Groups - collections of users. Instead of attaching policies to individual users, we can assign permissions to groups, making it easier to manage access based on job function or role.

Users or Groups can be assigned JSON documents called **policies** which define the permissions of the user. In AWS we apply the **least privilege principle** (dont give more permissions than a user needs).

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": "ec2:Describe*",
            "Resource": "*"
        },
        {
            "Effect": "Allow",
            "Action": "elasticloadbalancing:Describe*",
            "Resource": "*"
        },
        {
            "Effect": "Allow",
            "Action": [
                "cloudwatch:ListMetrics",
                "cloudwatch:GetMetricStatistics",
                "cloudwatch:Describe*"
            ],
            "Resource": "*"
        }
    ]
}
```

There are also **IAM Roles** which give permissions to AWS services. Unlike a user, a role doesn't have long-term credentials. Instead, it can be assumed temporarily by trusted entities.

Cross-account access using IAM roles in AWS allows one AWS account to grant permissions to users or services in another AWS account.



AWS Console Simultaneous Sign-In

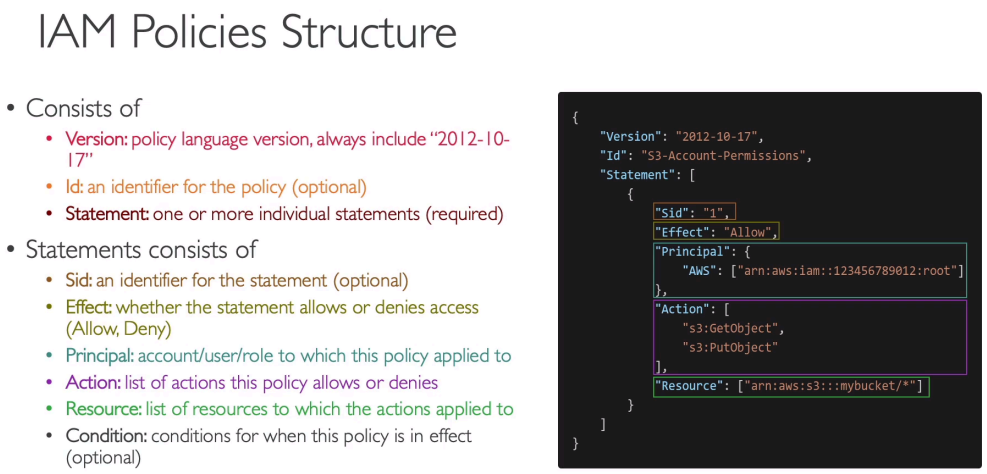
Console Simultaneous Sign-In is a feature that lets us manage and control the number of concurrent logins to the AWS Management Console for a specific IAM user or role.

By default, an IAM user can log in to the AWS Console from multiple devices/browsers at the same time. We can set a limit on how many concurrent sessions can be established. This is useful for controlling access, improving security, and enforcing policies around user behavior.

IAM Policies Structure.

IAM Policy Components

- Statement - main element of an IAM policy, and it consists of one or more individual statement.
- Sid - identifier for the statement.
- Effect - specifies whether the statements allows or denies the specified action.
- Principal (Optional) - Specifies the entity (user, account or role) that the policy is applied to.
- Action - lists the specific actions that the policy allows or denies. Actions are represented as strings, often in the format "**[service]:[action]**".
- Resource - specifies the AWS resources.
- Condition (Optional) - defines conditions under which the policy statement is in effect.



IAM MFA.

In AWS we can set minimum password length, specific character types as requirement, allow IAM users to change their passwords after some time and prevent password re-use.

MFA (Multi Factor Authentication) = password + security device.

- Virtual MFA (Google Authenticator, Authy)

  Support for multiple tokens on a single device.

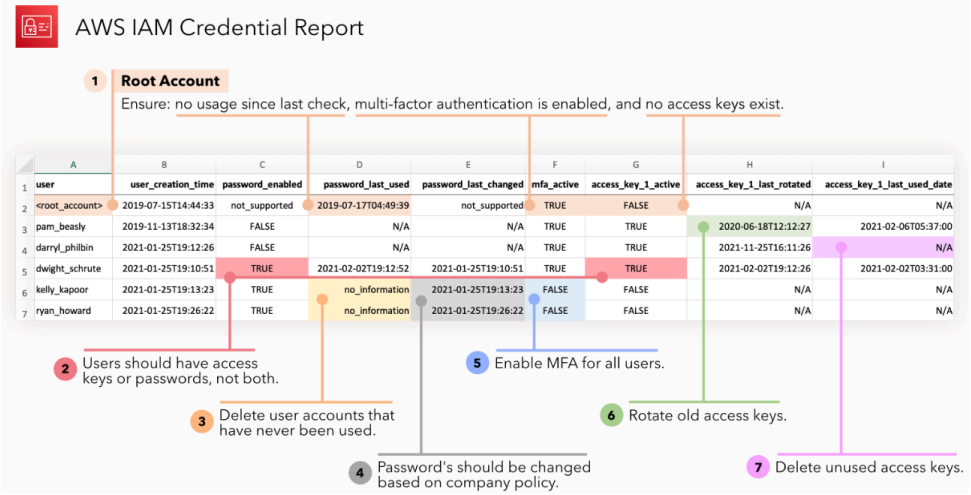
- U2F (Universal 2nd Factor) (YubiKey)

  Support for multiple root and IAM users using a single security key.

- Hardware Key Fob MFA Device (Gemalto)
- Hardware Key Fob MFA Device for AWS GovCloud (SurePassID)

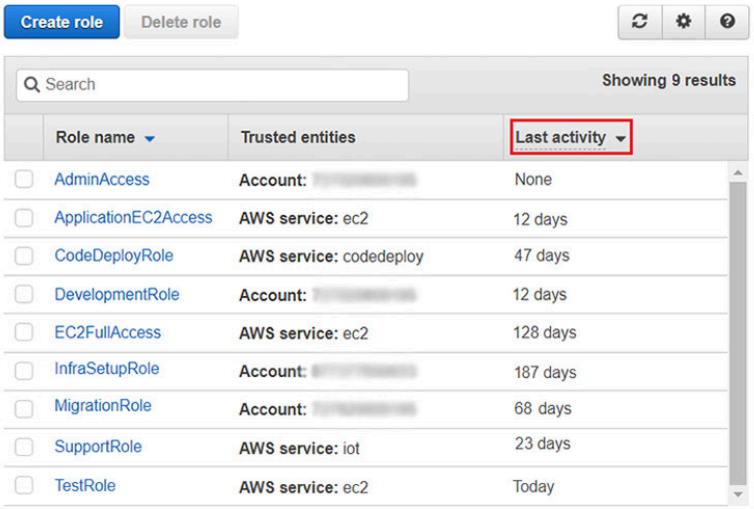
IAM Security Tools.

- **IAM Credential Report** (**account-level**) - Report that lists all account's users and the status of their credentials.



user	user_creation_time	password_enabled	password_last_used	password_last_changed	mfa_active	access_key_1_active	access_key_1_last_rotated	access_key_1_last_used_date
<root_account>	2019-07-15T14:44:33	not_supported	2019-07-17T04:49:39	not_supported	TRUE	FALSE	N/A	N/A
pam_beasly	2019-11-13T18:32:34	FALSE	N/A	N/A	TRUE	TRUE	2020-06-18T12:12:27	2021-02-06T05:37:00
darryl_philbin	2021-01-25T19:12:26	FALSE	N/A	N/A	TRUE	TRUE	2021-11-25T16:11:26	N/A
dwight_schrute	2021-01-25T19:10:51	TRUE	2021-02-02T19:12:52	2021-01-25T19:10:51	TRUE	TRUE	2021-02-02T19:12:26	2021-02-02T03:31:00
kelly_kapoor	2021-01-25T19:13:23	TRUE	no_information	2021-01-25T19:13:23	FALSE	FALSE	N/A	N/A
ryan_howard	2021-01-25T19:26:22	TRUE	no_information	2021-01-25T19:26:22	FALSE	FALSE	N/A	N/A

- **IAM Access Advisor** (**user-level**) - Shows the service premissions granted to a user and when those services were last accessed. This information can be used to revise policies.

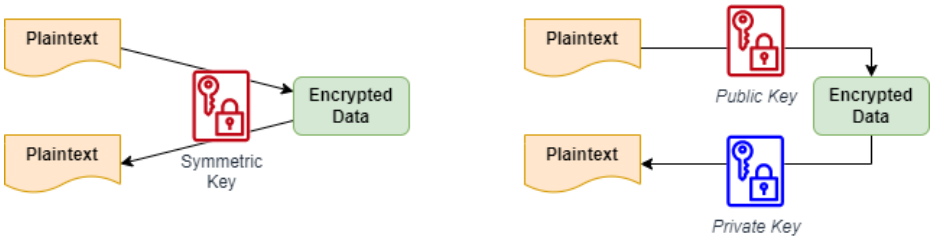


▼ KMS.

AWS KMS (Key Management Service) is a managed service that allows us to create and manage encryption keys used to encrypt and decrypt our data. It integrates with many other AWS services to simplify the encryption process.

Keys Types

- **Symmetric (AES-256 Keys)** - single encryption key that is used to encrypt and decrypt. AWS services that are integrated with KMS use Symetric CMKs. We never get access to the KMS key unencrypted (must call KMS API to use).
- **Asymmetrict (RSA & ECC Key Pairs)** - public (encrypt) and private (decrypt) key pair. Used for encrypt/decrypt or sign/verify operations. The public key is downloadable, but we cant access the private key unencrypted. It is used for encryption outside of AWS by users who cant call the KMS API.



Symmetric Encryption - the same key is used for both encrypting and decrypting data

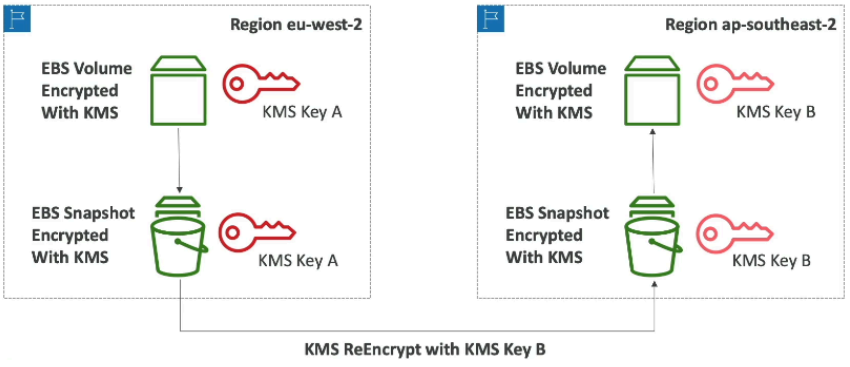
Asymmetric Encryption - the public key is used for encryption and the private key is used for decryption

Types of KMS Keys

- **Customer Managed Key (CMK)** - created, managed and used by the customer. Possibility of rotation policy (new key generated every year). Possibility to bring our own key.
- **AWS Managed Key** - created, managed and used on the customer's behalf by AWS. Used by AWS services.
- **AWS Owned Key** - collection of CMKs that an AWS service owns and manages to use in multiple accounts. AWS can use those to protect resources in our account (we cant view the keys).

Automatic key rotation is automatic every 1 year for AWS managed keys. For created CMKs it must be enabled (can be automatic or on-demand), and for imported KMS keys, only manual rotation is possible using alias.

KMS keys are scoped per region. If we want to to move encrypted EBS volume to another region we need to take snapshot of that EBS volume. Then we need to copy snapshot to another region, we neeed to re-encrypt the snapshot using a different KMS key and then we can restore EBS volume from that snapshot.



KMS Key Policies - control access to KMS keys (similar to S3 bucket policies).

- **Default KMS Key Policy** - created if we dont provide a specific KMS Key Policy. Root user has complete access to the key.
- **Custom KMS Key Policy** - define users, roles that can access the KMS key and who can administer the key. Useful for cross-account access of our KMS key.

To copy snapshot across accounts we need to create a snapshot encrypted with our own KMS key. Then attach a KMS Key Policy to authorize cross-account access and share the encrypted snapshot. In target account we need to create a copy of the snapshot, encrypt it with CMK in our account and then create a volume from snapshot.

▼ KMS Multi-Region Keys.

Multi-Region Keys are identical KMS keys in different regions that can be used interchangeably. They have the same key ID, key material and automatic rotation. We can encrypt in one region and decrypt in

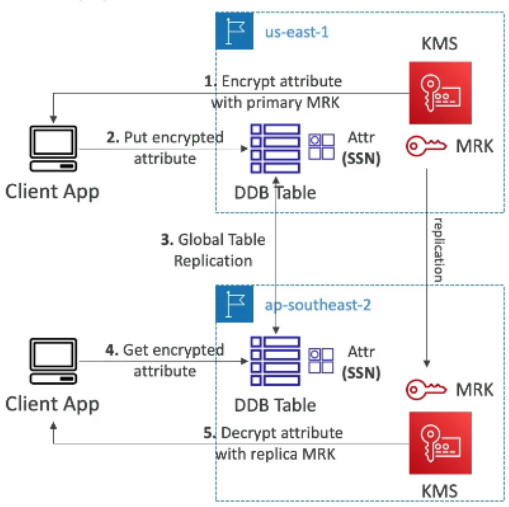
other regions (no need to re-encrypt or make cross-region API calls).

KMS Multi-Region is not global (primary + replicas). Each Multi-Region key is managed idependently.

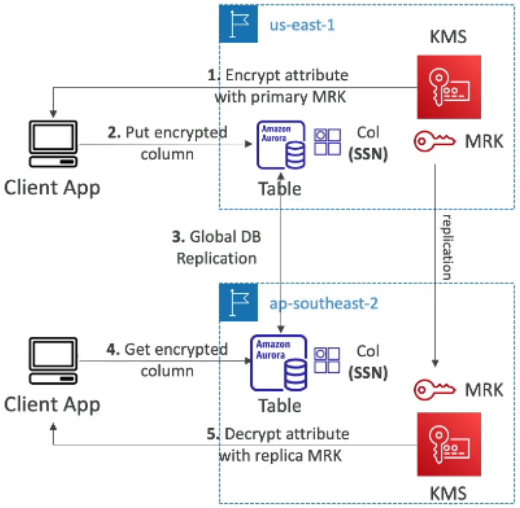
DynamoDB Global Tables/Global Aurora & KMS MRK Client-Side Encryption

We can encrypt specific attributes client-side in our DynamoDB table using DynamoDB Encryption Client. Combined with Global Tables, the client-side encrypted data is replicated to other regions. If we use a MRK replicated in the same region as the DynamoDB Global table, then clients in these regions can use low-latency API calls to KMS in their region to decrypt the data client-side.

Using client-side encryption we can protect specific fields and guarantee only decryption if the client has access to an API key.

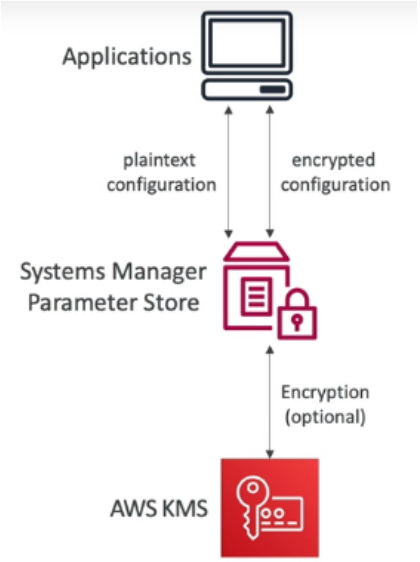


We can encrypt specific attributes client-side in our Aurora table using the AWS Encryption SDK. Combined with Aurora Global Tables, the client-side encrypted data is replicated to other regions. We can protect specific fields even from database admins.



▼ **SSM Parameter Store.**

SSM Parameter Store - it is a secure storage for configuration and secrets (API keys, passwords, configurations ...). It is serverless, scalable and durable. Also it is secure because we control access permissions using IAM.



Parameters Policies - allow us to assign a TTL to a parameter to force updating or deleting sensitive data such as passwords.

Expiration (to delete a parameter)

```
{
    "Type":"Expiration",
    "Version":"1.0",
    "Attributes":{
        "Timestamp":"2020-12-02T21:34:33.000Z"
    }
}
```

ExpirationNotification (EventBridge)

```
{
    "Type":"ExpirationNotification",
    "Version":"1.0",
    "Attributes":{
        "Before":"15",
        "Unit":"Days"
    }
}
```

NoChangeNotification (EventBridge)

```
{
    "Type":"NoChangeNotification",
    "Version":"1.0",
    "Attributes":{
        "After":"20",
        "Unit":"Days"
    }
}
```

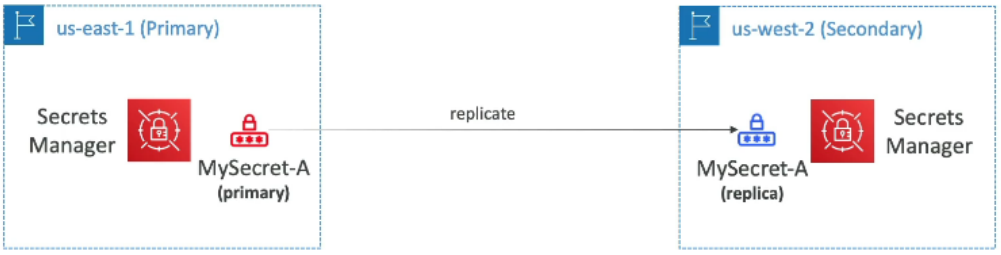
▼ AWS Secrets Manager.

ASM is a newer service which is used for storing secrets. It has capability to force rotation of secrets every X days. We can automate generation of secrets on rotation using Lambda. All secrets are encrypted using KMS.

Can be integrated with Amazon RDS (MySQL, Postgres, Aurora). Mostly meant for RDS integration.

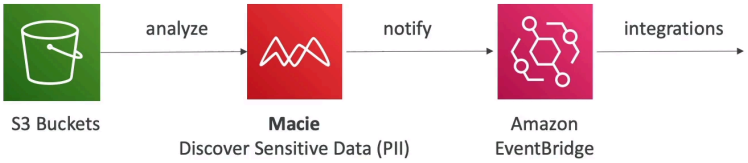
We can replicate secrets across multiple regions. Secrets Manager keeps read replicas in sync with the primary secret. There is an ability to promote a read replica secret to a standalone secret.

Use cases: multi-region apps, disaster recovery strategies, multi-region databases...



▼ Amazon Macie.

Amazon Macie is a fully managed data security and data privacy service that uses ML and pattern matching to discover and protect our sensitive data in AWS. Macie helps idenfity and alert us to sensitive data in our **S3 Buckets**, such as personally identifiable information (PII).



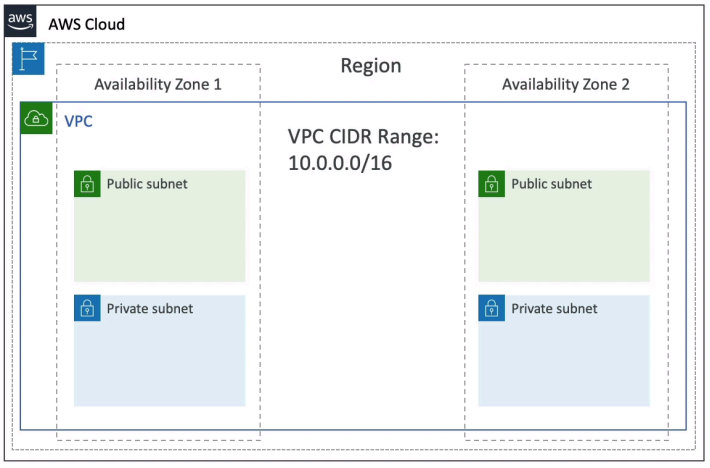
VPC & Subnet.

VPC (Virtual Private Cloud) is a private network where we can deploy our resrouces (regional resource). All new AWS accounts have a default VPC. New EC2 instances are launched into the default VPC if no subnet is specified. Default VPC has internet connectivity and all EC2 instances inside it have public IPv4 addresses.

Our VPC CIDR should not overlap with our other networks (e.g. corporate).

Subnets allow us to partition our network inside our VPC (AZ resource).

- Public subnet - subnet that is accessible from the internet.
- Private subnet - subnet that is not accesible from internet

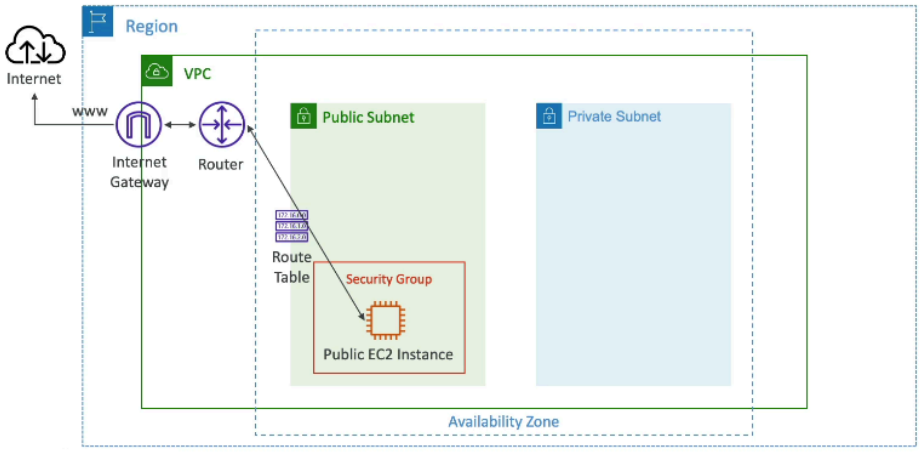


Internet Gateways & Route Tables.

Internet Gateway is a horizontally scaled, redundant, and highly available VPC component that allows instances within our VPC to connect to the internet, and for the internet to initiate connections with instances within our VPC (if allowed by security rules).

One VPC can only be attached to one IGW and vice versa.

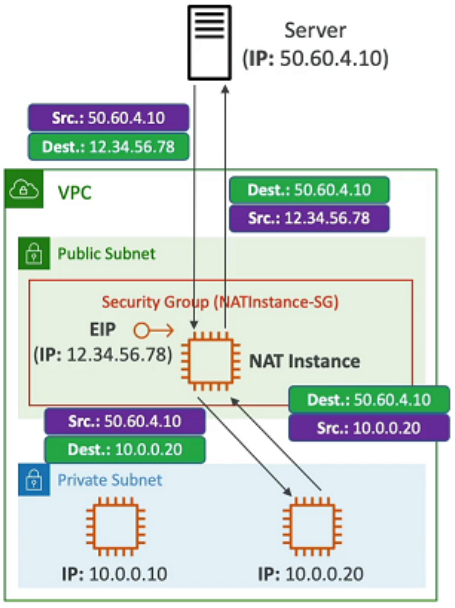
Routing Table is a set of rules (routes) that determine where network traffic is directed. Every subnet in our VPC must be associated with a route table.



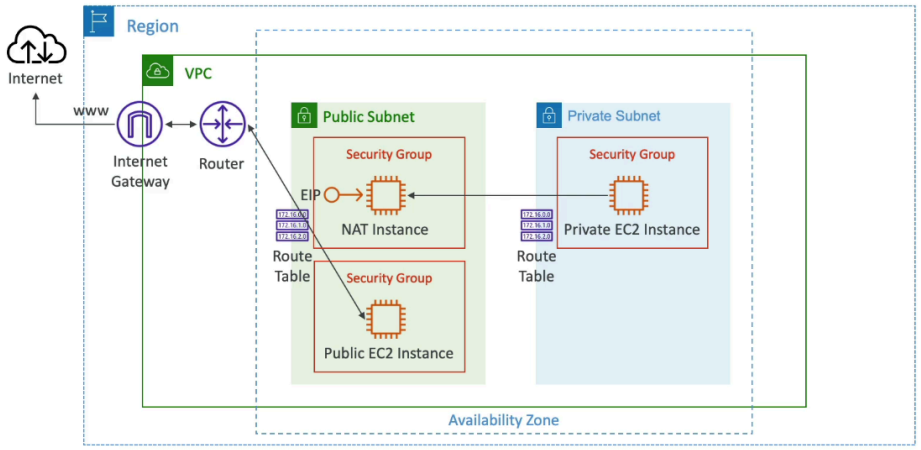
▼ **NAT Instances & NAT Gateways.**

NAT Instance allows EC2 instance in private subnets to connect to the internet. They must be launched in a public subnet and we must disable EC2 source/destination check setting and have Elastic IP attached to it.

Route Tables must be configured to route traffic from private subnets to the NAT Instance.

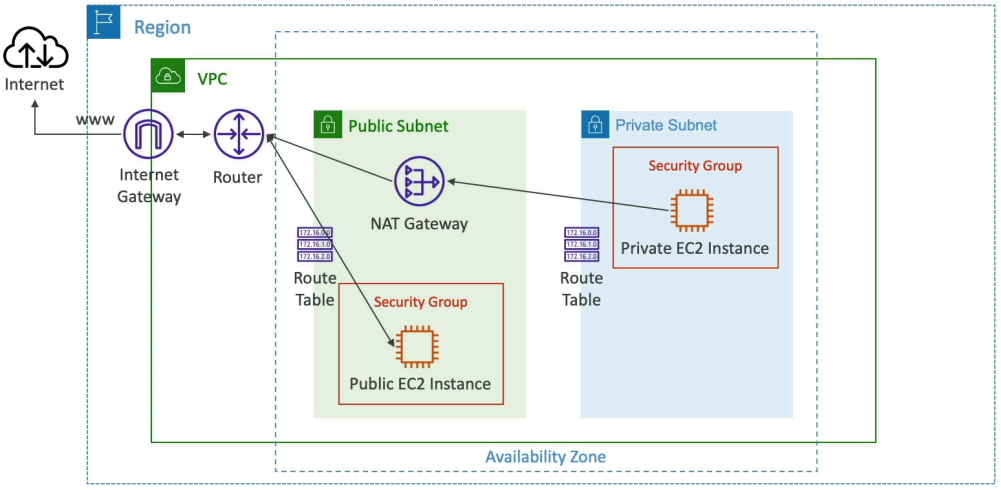


There is a pre-configured Amazon Linux AMI for NAT Instance. NAT instances are not highly available, we need to create an ASG in multi-AZ and internet traffic bandwidth depends on EC2 instance type.

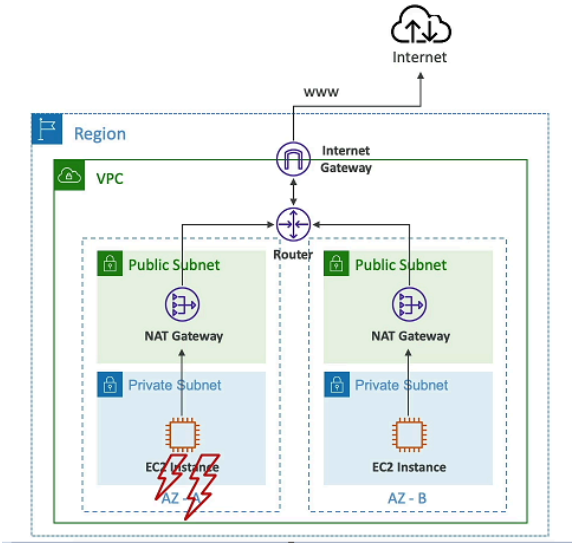


NAT Gateway is AWS managed NAT with higher bandwidth, high availability and no administration. We pay per hour for usage and bandwidth. NAT Gateway is created in a specific AZ and uses an Elastic IP.

It cant be used by EC2 instance in the same subnet (only from other subnets). It requires an Internet Gateway.



NAT Gateway is resilient within a single AZ. We must create multiple NAT Gateways in multiple AZs for fault-tolerance. There is no cross-AZ failover needed because if an AZ goes down it doesnt need NAT.



NAT Instance vs NAT Gateway

	NAT Gateway	NAT Instance
Availability	Highly available within AZ (create in another AZ)	Use a script to manage failover between instances
Bandwidth	Up to 100 Gbps	Depends on EC2 instance type
Maintenance	Managed by AWS	Managed by you (e.g., software, OS patches, ...)
Cost	Per hour & amount of data transferred	Per hour, EC2 instance type and size, + network $
Public IPv4	✓	✓
Private IPv4	✓	✓
Security Groups	✗	✓
Use as Bastion Host?	✗	✓

▼ NACL & Security Groups.

NACL - firewall which controls traffic from and to subnet. It can have ALLOW and DENY rules. Rules only include IP addresses. Newly created NACLs will deny everything. We can have one NACL per subnet, new subnets are assigned the Default NACL.

Default NACL accepts everything inbound/outbound with the subnets it is associated with. We should not modify the Default NACL, instead we should create custom NACLs.

Default NACL for a VPC that supports IPv4

Inbound Rules

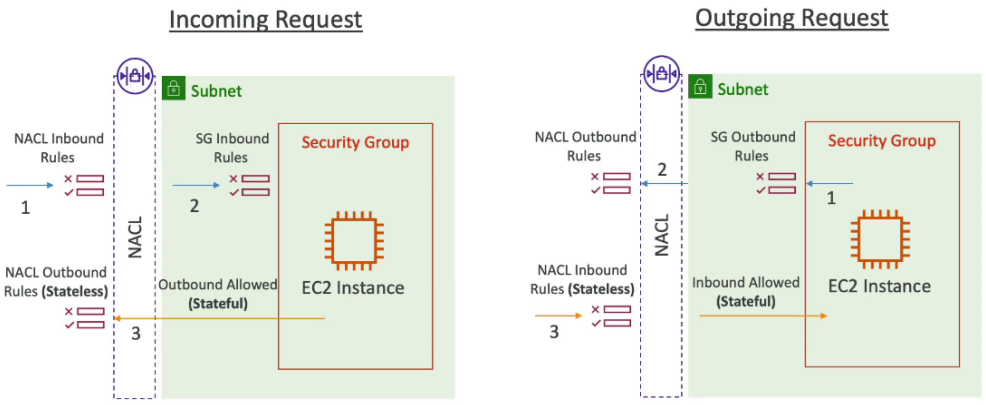
Rule #	Type	Protocol	Port Range	Source	Allow/Deny
100	All IPv4 Traffic	All	All	0.0.0.0/0	ALLOW
*	All IPv4 Traffic	All	All	0.0.0.0/0	DENY

Outbound Rules

Rule #	Type	Protocol	Port Range	Destination	Allow/Deny
100	All IPv4 Traffic	All	All	0.0.0.0/0	ALLOW
*	All IPv4 Traffic	All	All	0.0.0.0/0	DENY

Rules have a number (1-32766), higher precedence = lower number. First rule match will drive the decision.

Security Groups - firewall that controls traffic to and from an EC2 instance. It can have only ALLOW rules. Rules include IP addresses and other security groups.



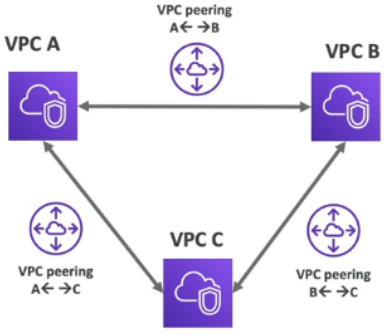
Security Group vs NACL

Security Group	NACL
Operates at the instance level	Operates at the subnet level
Supports allow rules only	Supports allow rules and deny rules
Stateful: return traffic is automatically allowed, regardless of any rules	**Stateless:** return traffic must be explicitly allowed by rules (think of ephemeral ports)
All rules are evaluated before deciding whether to allow traffic	Rules are evaluated in order (lowest to highest) when deciding whether to allow traffic, first match wins
Applies to an EC2 instance when specified by someone	Automatically applies to all EC2 instances in the subnet that it's associated with

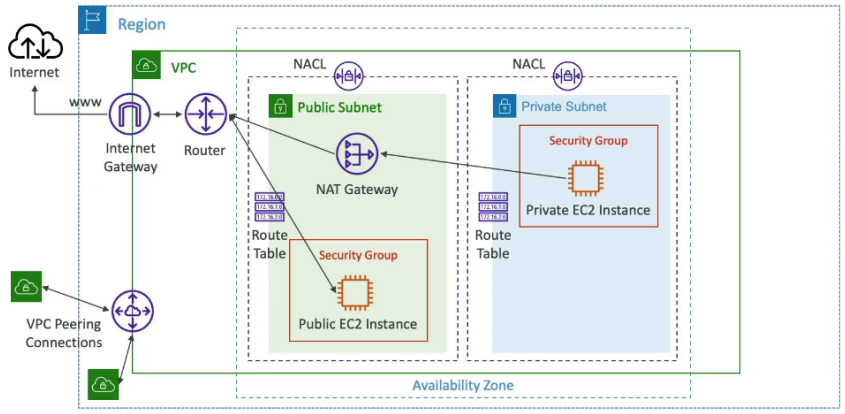
▼ VPC Peering.

VPC Peering connects two VPCs privately using AWS network. It makes two VPCs behave as if they were in the same network. They must not have overlapping CIDR (IP address range).

VPC Peering connection is not transitive (must be established for each VPC that need to communicate with one another).

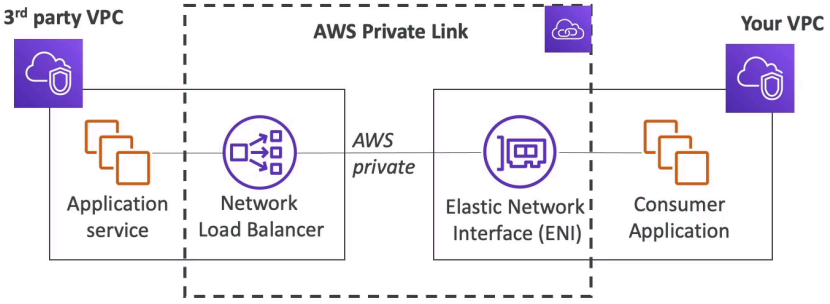


We must update route tables in each VPC's subnets to ensure EC2 instances can communicate with each other. We can create VPC Peering connection between VPCs in different AWS accounts/regions. Also we can reference a security group in a peered VPC (works cross accounts in same region).



VPC Endpoints & AWS PrivateLink.

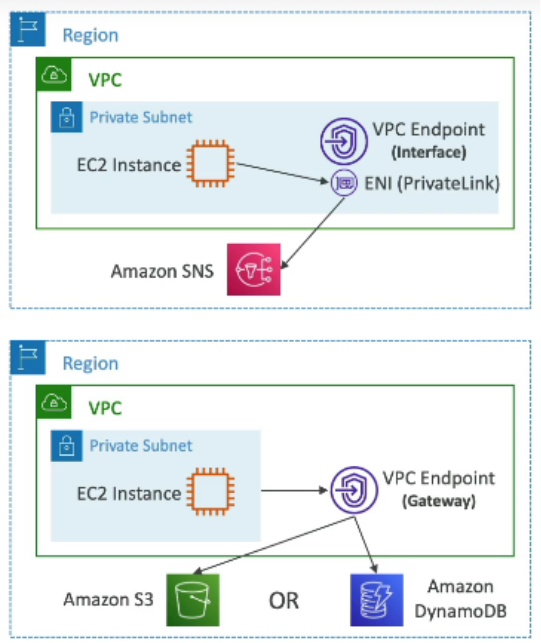
AWS PrivateLink allows us to privately access services hosted on AWS, including our own applications, third-party services, and AWS services, without exposing our traffic to the public internet. It simplifies the network architecture, improves security. Does not require VPC peering, internet gateway, NAT, route tables, but it requires a NLB on service VPC and ENI on customer VPC.



VPC Endpoints (powered by AWS PrivateLink) allow us to connect to AWS Services using a private network instead of the public internet. It gives us enhanced security and lower latency to access AWS services.

- **Interface Endpoints** (powered by PrivateLink) - provisions an ENI (private IP address) as an entry point (must attach a Security Group). It supports most AWS services. We pay per hour and per GB of data processed.
- **Gateway Endpoints** - provisions a gateway and must be used as a target in a route table (does not use security groups). It supports S3 and DynamoDB and it is free.

  We can attach an endpoint policy that controls access to the service to which we are connecting. We can modify the endpoint policy attached to our endpoint and add or remove the route tables used by the endpoint. An endpoint policy does not override or replace IAM user policies or service-specific policies.



VPC Flow Logs.

VPC Flow Logs capture information about IP traffic going into our interfaces. There are also Subnet Flow Logs and ENI Flow Logs. VPC Flow logs data can go to S3, CloudWatch Logs and Kinesis Data Firehose.

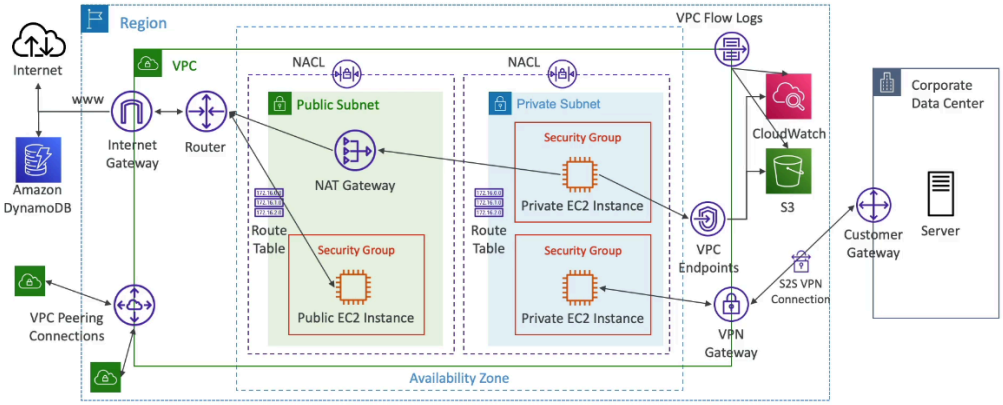
Site-to-Site VPN, VGW & CGW.

Site-to-Site VPN is used to connect an on-premises VPN to AWS. The connection is automatically encrypted and it goes over the public internet.

To establish Site-to-Site VPN:

- **On-premises** must use a **Customer Gateway (CGW)** - a physical device or software application on the customer's side of the VPN connection. It can be a hardware VPN device, such as a router or firewall, or it can be software-based.
- **AWS** must use a **Virtual Private Gateway (VGW)** - the VPN concentrator on the Amazon side of the VPN connection within the AWS infrastructure. It is a logical entity that represents a VPN gateway capable of handling multiple VPN connections simultaneously.

On-premises should use public internet-routable IP address for its CGW device or if it's behind a NAT device that is enabled for NAT traversal (NAT-T), it should use the public IP address of the NAT device.



Direct Connect.

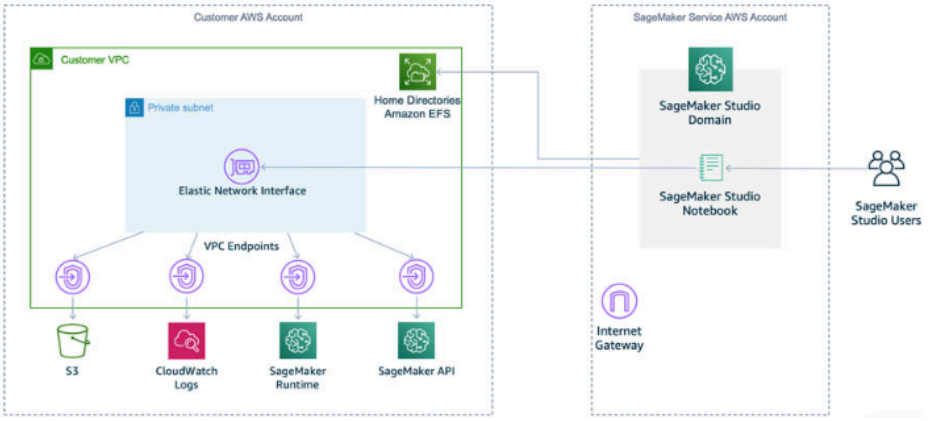
Direct Connect provides a dedicated **private** connection from a remote network to our VPC. Dedicated connection must be setup between our data center and AWS DX locations. We need to setup a VGW on our VPC. With DX we can access both public and private resources on same connection.

Use cases:

- Increase bandwidth throughput (**working with large data sets**).
- More consistent network experience (applications using real-time data feeds).
- Hybrid environments.

SageMaker & VPC Integration.

Training jobs run in a VPC. We can use a private VPC for even more security (we need to set up S3 VPC endpoints).



Notebooks are internet-enable by default. This can be security hole. We can disable it, but then our VPC needs an interface endpoint (PrivateLink) or NAT Gateway, and allow outbound connections.

Training and inference containers are also internet-enable by default.

Network Isolation Mode - removes the network card from the container entirely. Used to protect licensed code or marketplace models so they cannot phone home or leak data anywhere.

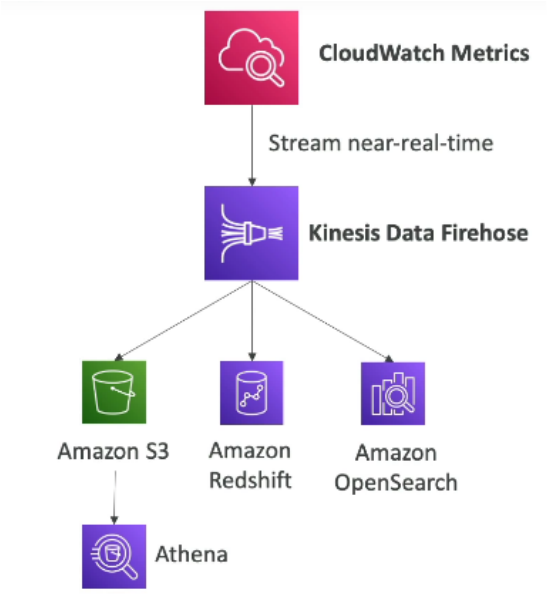
Management & Governance

CloudWatch Metrics.

CloudWatch provides metrics for every service in AWS. Metric is a variable to monitor (CPU Utilization, Network In, etc). We can create custom metrics too (**Memory Utilization**).

Each metric has timestamps and dimension. **Dimension** is an attribute of a metric (instance id, environment, etc.). CloudWatch can create dashboards of metrics.

CloudWatch Metric Streams - continually stream CloudWatch metrics to a destination of our choice, with near real-time delivery and low latency. We can use KDF or third party service provider (Datadog, Dynatrace, etc). We have option to filter metrics to only stream a subset of them.



CloudWatch Logs.

CloudWatch Logs is a logging service that monitors, stores, and access log files from AWS resources, applications, and services. Logs are encrypted by default.

- Log groups - arbitrary name, usually representing an application.
- Log stream - instance within applications/log files/containers.

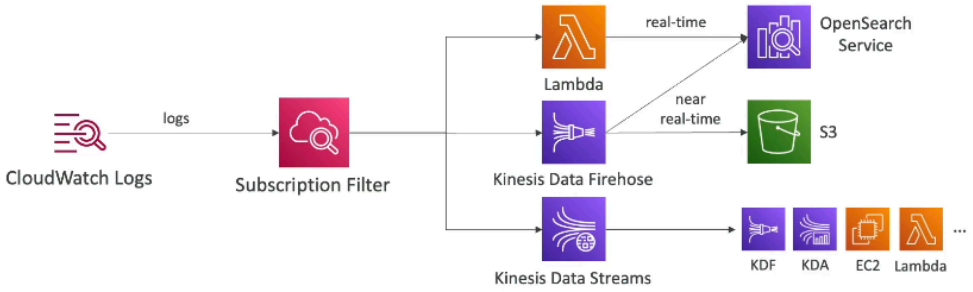
CloudWatch Logs can send logs to: S3, KDS, KDF, AWS Lambda, OpenSearch.

Logs can be collected from: Elastic Beanstalk, ECS, AWS Lambda, CloudTrail, CloudWatch Log agents (on EC2 instances or on-premises servers), Route53.

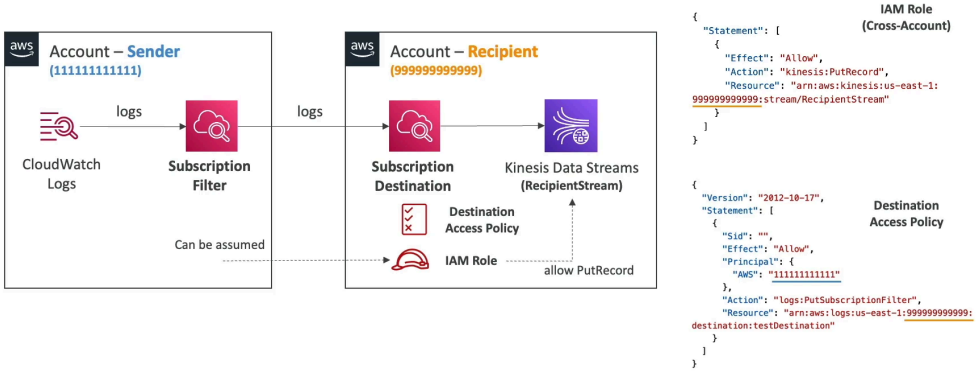
CloudWatch Logs Insights - search and analyze log data stored in CloudWatch Logs. Provides a purpose-built query language. It automatically discovers fields from AWS services and JSON log events. We can save queries and add them to CloudWatch Dashboard. It's query engine, not a real-time engine.

Log data can take up to 12h to become available for export. It is not near real-time or real-time, if we want that we should use Logs Subscriptions instead.

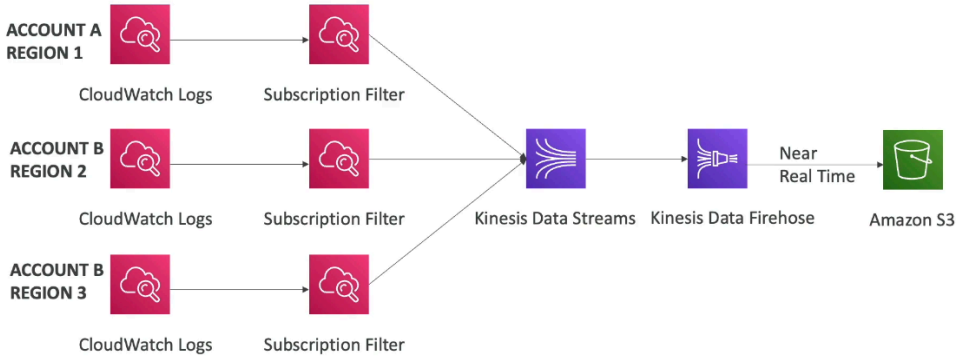
CloudWatch Logs Subscriptions - lets us get a real-time log events from CloudWatch Logs for processing and analysis. We can send to KDS, KDF or AWS Lambda. There is Subscription Filter which can filter which logs will be delivered to our destination.



We can have Cross-Account Subscription which can send log events to resources in different AWS account (KDS, KDF).

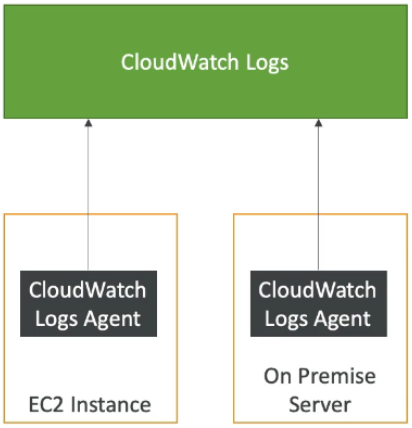


▼ **Logs Aggregation (Multi-Account & Multi Region).**



▼ **CloudWatch Agent & CloudWatch Logs Agent.**

By default, no logs from our EC2 instance will go to CloudWatch, we need to run a CloudWatch Agent on EC2 to push the log files we want (need to make sure IAM permissions are correct).



CloudWatch Unified Agent - it collects additional system-level metrics such as RAM, processes, etc. It collect logs to send them to CloudWatch Logs. It has centralized configuration using SSM Parameter Store.

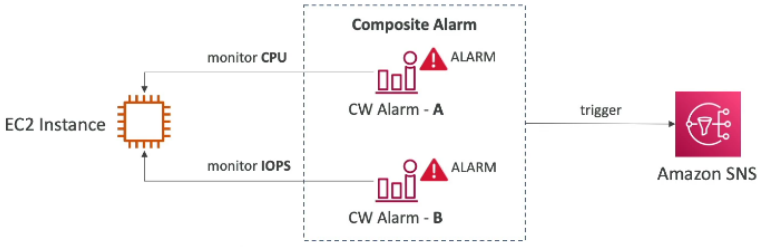
CloudWatch Alarms.

CloudWatch Alarms - used to trigger notifications for any metrics. It has various options (sampling, percentage, max, min, etc). With CloudWatch Alarm we can stop, terminate, reboot or recover an EC2 instance, trigger auto scaling action or send notification to SNS.

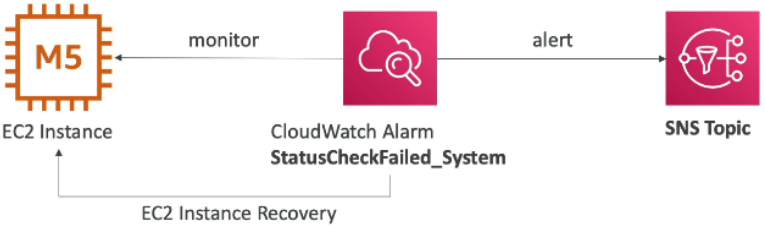
Alarm States: OK, INSUFFICIENT_DATA, ALARM.

Period - length of time in seconds to evaluate the metrics.

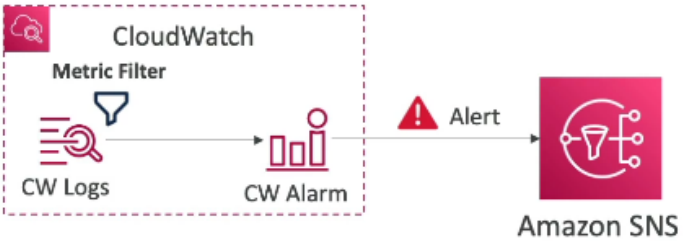
Composite Alarms - monitoring the states of multiple other alarms and we can use AND and OR conditions to merge them.



There is a status check to check the EC2 and we can define a CloudWatch Alarm to monitor a specific EC2 instance. In case the alarm is being breached, we can start an EC2 instance recovery or send an alert to our SNS topic.



Alarms can be created based on CloudWatch Logs Metrics Filters.



AWS X-Ray.

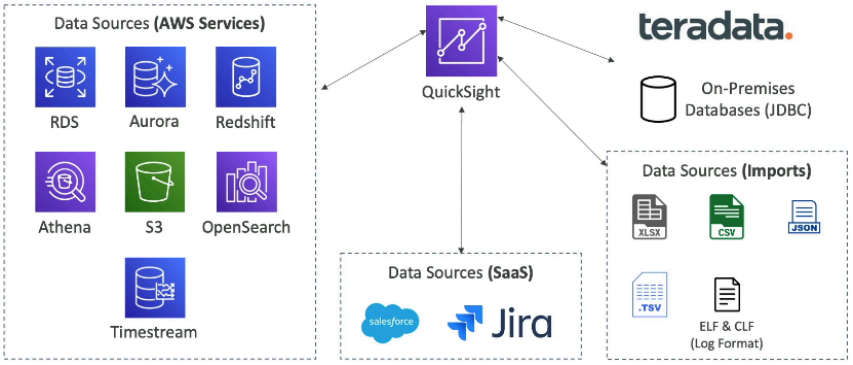
AWS X-Ray is a distributed tracing service that helps developers analyze and debug production, distributed applications, such as those built using a microservices architecture. It provides end-to-end visibility into the performance of our applications by tracing requests as they travel through our application.

QuickSight.

QuickSight is a serverless ML-powered business intelligence service to create interactive dashboards. It is fast, automatically scalable, embeddable, with per-session pricing. Can be integrated with RDS, Aurora, Athena, Redshift ...

It has in-memory computation capabilities using SPICE engine. Each user gets 10GB of SPICE. In enterprise edition we have possibility to setup Column-Level Security (CLS) to prevent some columns to be displayed to users who dont have permission.

Usage: business analytics, building visualizations, perform ad-hoc analysis ...



In QuickSight we can define users (standard versions) and groups (enterprise version). These users and groups only exist within QuickSight, not IAM!

Dashboard - read-only snapshot of an analysis that we can share. It preserves the configuration of the analysis.

We can share the analysis or the dashboard with users or groups. Users who see the dashboard can also see the underlying data.

QuickSight Tools

QuickSight ML Insight - features that automatically analyze our data and surface patterns, anomalies, forecasts, and explanations.

QuickSight Q - a natural-language querying capability that lets users ask questions about data in language and receive visual answers.

QuickSight Paginated Reports - formatted reports designed for printing, PDF generation, and operational reporting use cases where layout precision matters.

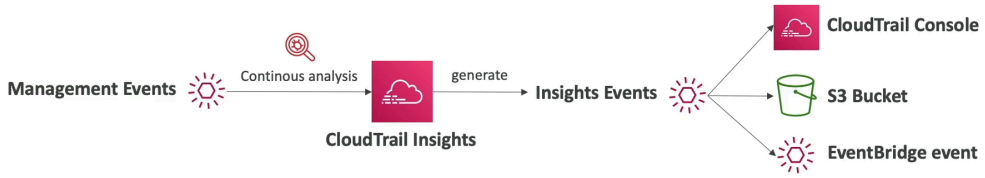
CloudTrail.

AWS CloudTrail provides governance, compliance and audit for our AWS account. It is enabled by default. We can get history of events and API calls made within our account. It's possible to put logs from CloudTrail into CloudWatch Logs or S3. Can be applied to all regions (default) or a single region. By default, CloudTrail event log files are encrypted using Amazon S3 server-side encryption (SSE).

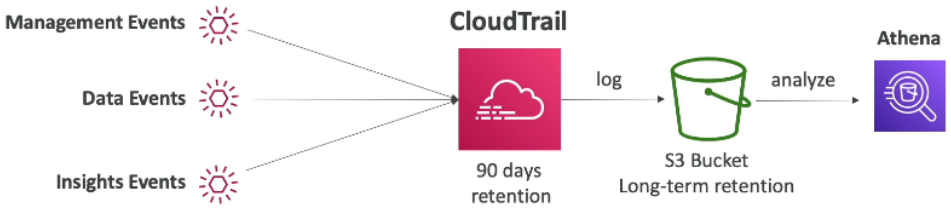
CloudTrail Events

- **Management Events** - operations that are performed on resources in our AWS account. By default, trails are configured to log management events. We can separate read events from write events.
- **Data Events** - S3 object-level activity and Lambda function execution activity (get, delete, put). By default, data events are not logged because high volume operations. It can separate read events from write events.
- **CloudTrail Insights Events**

CloudTrail Insights is a feature within AWS CloudTrail that helps us detect unusual or unexpected activity in your AWS accounts. It continuously analyzes our CloudTrail event logs to identify patterns and anomalies, enabling us to quickly respond to potential security issues.



Events are stored for 90 days in CloudTrail. To keep events beyond this period we should log them to S3 and use Athena.



Config.

AWS Config helps us with auditing and recording compliance of our AWS resources. It can record configurations and changes over time. It is possible to store the configuration data into S3 (analyzed by Athena).

Questions that can be solved by AWS Config:

- Is there unrestricted SSH access to my security groups?
- Do my buckets have any public access?
- How has my ALB configuration changed over time?

Config is a per-region service and can be aggregated across regions and accounts. We can receive alerts (SNS notifications) for any changes.

Config Rules

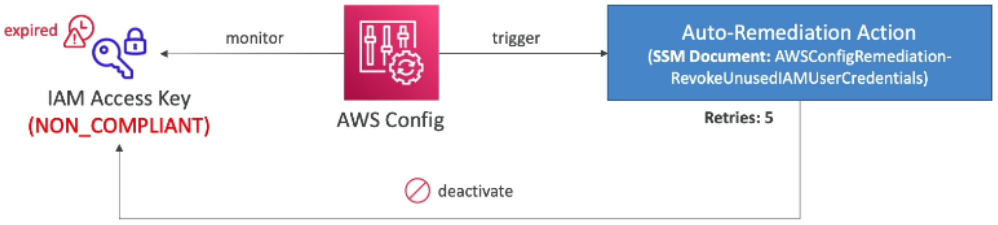
We can use AWS managed config rules or make custom config rules (**must be defined in AWS Lambda**). Rules can be evaulated/triggered for each config change and/or at regular time intervals.

Config Rules does not prevent actions from happening (**no deny**).

Config Remediations

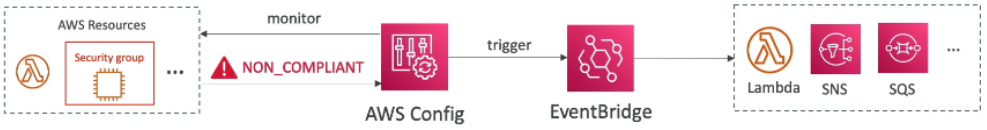
We can automate remediation of non-compliant resource using SSM Automation Documents. It is possible to create custom Automation Documents that invokes Lambda function.

We can set Remediation Retries if the resource is still non-compliant after auto-remediation.

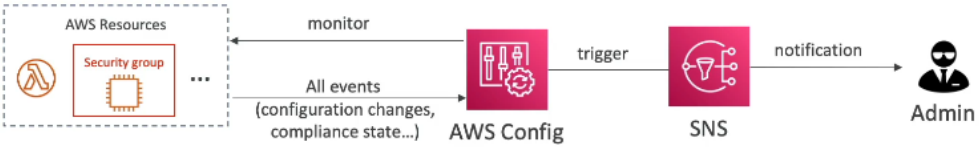


Config Notifications

We can use EventBridge to tigger notifications when AWS resources are non-compliant.



There is ability to send configuration changes and compliance state notifications to SNS.



Cost Explorer.

Cost Explorer - allows users to visualize, understand, and manage their AWS costs and usage over time. We can create custom reports that analyze cost and usage data on high level (total cost and usage across all accounts).

With Cost Explorer we are able to access our optimal savings plan. Also we can forecast usage up to 12 months based on previous usage.

AWS Budgets.

AWS Budgets - allows us to set custom budgets that alert us when our AWS costs or usage exceed our budgeted amount. There are four types of budgets: Usage, Cost, Reservation, Savings Plans.

We can track utilization of our reserved instances (supports **EC2, ElastiCache, RDS, Redshift**).

First two budgets are free, then $0.02/day/budget.

Trusted Advisor.

Trusted Advisor is a service that helps customers optimize their AWS environments, improve performance, and save money. It provides real-time guidance to help users follow AWS best practices.

It lets us analyze our AWS accounts and provides recommendation on six categories:

- Cost optimization.
- Performance.
- Security.
- Fault tolerance.
- Service limits.
- Operational Excellence.

For Business & Enterprise Support plan:

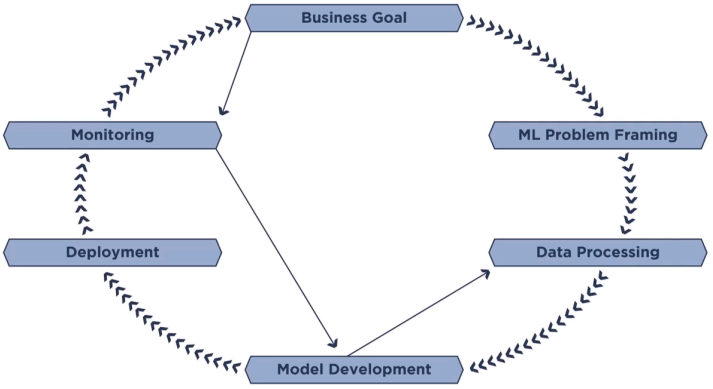
- Full Set of Checks.
- Programmatic Access using AWS Support API.

ML Best Practices

AWS ML Design Principles.

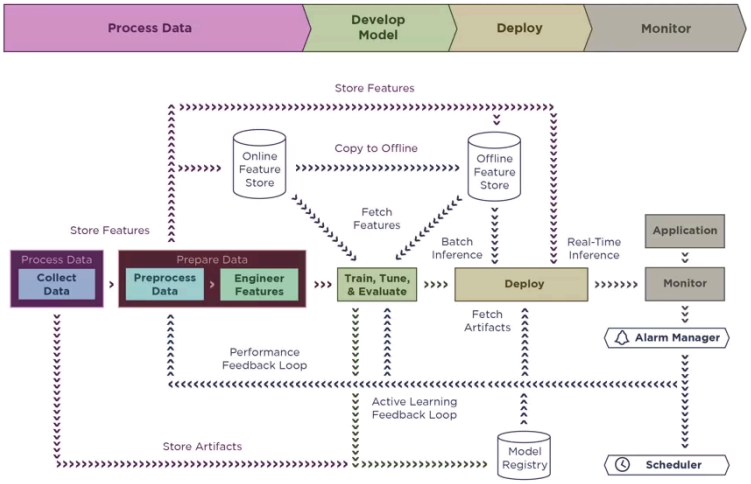
1. Assign ownership.
2. Provide protection.
3. Resiliency.
4. Reusability.
5. Reproducibility (version control).
6. Optimize resources & reduce cost.
7. Automation (CI/CD/CT).
8. Continous monitoring & analysis.
9. Minimize environmental impact.

ML Lifecycle



Framing ML Problem Best Practices.

1. Establish ML roles and responsibilities (SageMaker Role Manager).
2. Prepare an ML profile template (document resources required).
3. Establish model improvement strategies (SageMaker Experiments, AutoML).
4. Establish a lineage tracker system (SageMaker Lineage Tracking, Pipelines, Studio, Feature Store, Model Registry).
5. Establish feedback loops across ML lifecycle phases (CloudWatch).
6. Review fairness and explainability.
7. Design data encryption and obfuscation (Glue DataBrew).
8. Use APIs to abstract change from model consuming applications (SageMaker + API Gateway).
9. Use purpose-built AI and ML services and resources (SageMaker, JumpStart, Marketplace).
10. Define relevant evaluation metrics.
11. Identify if ML is the right solution.



Data Processing Best Practices.

1. Profile data to improve quality (Data Wrangler, Glue, Athena, ...).
2. Create tracking and version control mechanism.
3. Ensure least privilege access.

4. Secure data and modeling environment (IAM, KMS, Secrets Manager, VPCs).
5. Protect sensitive data (Macie).
6. Keep only relevant data.
7. Use a data catalog (Glue).
8. Use data pipeline (SageMaker Pipeline).
9. Automate managing data changes (MLOps).
10. Use modern data architecture.
11. Use managed data labeling.
12. Enable feature reusability (Feature Store).

▼ **Model Development Best Practices.**

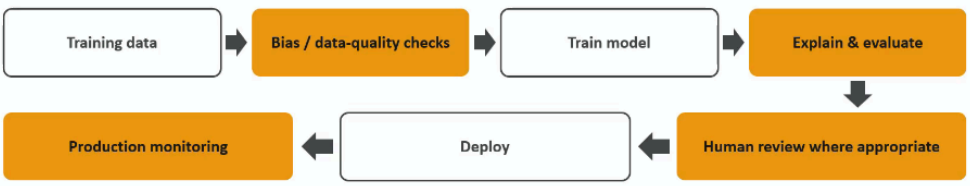
1. Automate operations through MLOps (CloudFormation, CDK, SageMaker Pipelines, Step Functions).
2. Establish reliable packaging patterns to access approved libraries (ECR, CodeArtifact).
3. Secure governed ML environments.
4. Secure inter-node cluster communication.
5. Protect against data poisoning threats.
6. Enable CI/CD/CT automation.
7. Ensure feature consistency across training and inference (Feature Store).
8. Ensure model validation with relevant data (SageMaker Experiments).
9. Establish data bias detection and mitigation.
10. Establish a model performance evaluation pipeline.
11. Establish feature statistics (Data Wrangler).
12. Detect performance issues when using transfer learning (SageMaker Debugger).
13. Select optimal computing instance size.
14. Select local training for small scale experiments.
15. Use managed training capabilities (SageMaker, Managed Spot Instances).
16. Use distributed training.
17. Use warm-start and checkpointing hyperparameter tuning.
18. Setup budget and use resource tagging to track costs.

▼ **Model Monitoring Best Practices.**

1. Enable model observability and tracking.
2. Sync architecture and configuration, and check for skew across environments.
3. Monitor human interactions with data for anomalous activity (Logging, GuardDuty, Macie).
4. Allow automatic scaling of the model endpoint.
5. Ensure a recoverable endpoint with a managed version control.
6. Establish an automated re-training framework (SageMaker Pipelines, Step Functions).
7. Include human-in-the-loop monitoring.
8. Monitor usage and cost by ML activity.

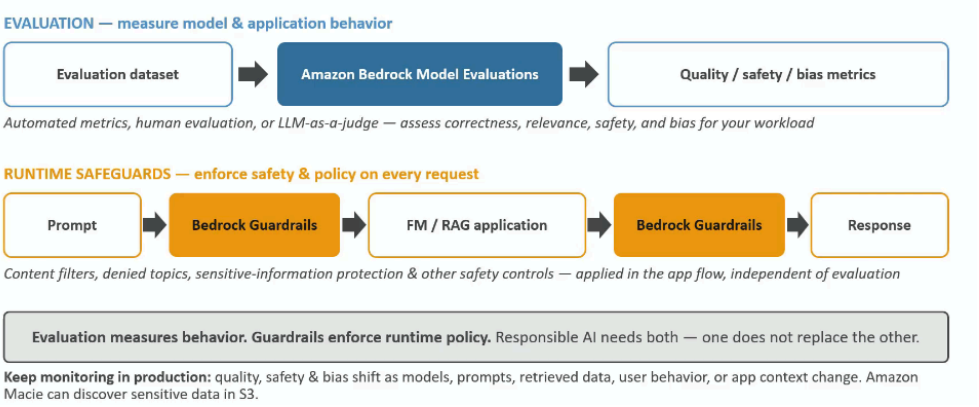
AWS Tools for Reponsible Model.

Responsible AI - Traditional ML



- **Bias & fairness metrics** — standardized metrics quantify disparities in datasets and model outcomes (the individual metrics are covered elsewhere in this course)
- **SHAP** — explains individual or aggregate predictions using feature-attribution values
- **SageMaker AI MLflow** — tracks experiment / model metrics and artifacts for reproducibility, comparison, and auditability
- **Human review** — a general workflow pattern when consequences, uncertainty, or risk justify it (not tied to any single service)
- **Production monitoring** — CloudWatch, QuickSight, and AWS-published monitoring solutions

Responsible AI - GenAI



Good To Know

Boto3

Boto3 - AWS SDK for Python. It lets developers write Python code to create, manage, and delete AWS resources.

Evidently AI

Evidently AI is an **open-source Python library** (and observability platform) used to evaluate, test, and monitor ML models and LLM systems in production. It can detect Data Drift and track data quality and model performance.